

Project Title

AI-Driven Smart LMS

By

Sarfraz

Roll No: RBSIT-21-13

Session: 2021-2025

Supervisor: Mr. Ghulam Ghos



**Department of Information Technology, Bahauddin Zakariya
University Multan**

January 2026

BAHAUDDIN ZAKARIYA UNIVERSITY, MULTAN

Department of Information Technology

AI-Driven Smart Learning Management System

(AI-Driven Smart LMS)

Degree of Bachelor of Science in Information Technology

By

SARFRAZ

Roll No: BSIT-21-13

Session: 2021 – 2025

Supervisor: Mr. Ghulam Ghos



January 2026

Certificate of Approval

This is to certify that the project titled "**AI-Driven Smart Learning Management System**" submitted by **SARFRAZ (Roll No: RBSIT-21-13)** in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology has been examined and approved.

Name: Mr. Ghulam Ghos

Name: Dr. Maroof Pasha

Signature: _____

Signature: _____

Date: _____

Date: _____

Declaration

I hereby declare that this project titled "**AI-Driven Smart Learning Management System**" is my own work and has not been submitted elsewhere for any other degree or qualification

Name: SARFRAZ

Roll No: RBSIT-21-13

Signature: _____

Date: _____

Acknowledgements

All praise, thanks, glorification, and gratitude are for **ALLAH** Almighty. I achieved my goal by **ALLAH's** infinite mercy and grace. I say Alhamdulillah for HIS uncountable blessings. After **ALLAH**, thanks to last Prophet **Muhammad** (ﷺ) who guided us toward the right path. Beloved prophet (ﷺ) says “Education is equally important for men and women”.

Very special and humble thanks to my father. He is my main supporter to continue the studies. My father instilled iron determination in me. I just want to say, he is great, he is genius. I also express special thanks to my mother. She always prayed for me and improves my confidence to continue the higher study. My mother has always been very caring about my health, studies, and career. I would like to pay my heartiest gratitude to my sister and brothers for being a great source of support and encouragement to complete this work. Especially, thanks to my brothers who guided and helped me all along the way.

I would like to express my sincere gratitude to my supervisor **Mr. Ghulam Ghos** for the continuous support of my research work, for their patience, motivation, enthusiasm, and immense knowledge. Their guidance helped me during all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for my research work.

Secondly, I would like to say thanks to **Mam Aliya Bodla**, principle and director of Regional Institute of Allied Health Sciences and Higher Education, Mian Channu, whose support and concern towards students is highly valuable.

In addition, I would like to acknowledge important role of **Mr. Usman**, whose help and assistance remained a source of motivation for me throughout this task.

SARFRAZ
RBSIT-21-13

Abstract

This paper presents an AI-Driven Smart Learning Management System...

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Project Motivation	3
1.4 Aims and Objectives.....	3
1.5 Project Scope.....	5
1.6 Document Structure.....	7
CHAPTER 2: LITERATURE REVIEW & BACKGROUND STUDY.....	9
2.1 Introduction to Domain.....	9
2.2 Existing Systems Analysis	10
2.3 Comparative Analysis of Existing Systems.....	11
2.4 Identified Research Gaps	12
2.5 Proposed Solution Overview.....	14
2.6 Summary of Related Research Papers	15
CHAPTER 3: REQUIREMENT ANALYSIS & SPECIFICATION	17
3.1 Introduction to Requirements.....	17
3.2 Stakeholder Analysis	17
3.3 Functional Requirements	18
3.4 Non-Functional Requirements	24
3.5 Use Case Modeling.....	24
3.6 User Stories	30
3.7 Hardware & Software Requirements	30
3.8 Assumptions and Constraints.....	32
CHAPTER 4: SYSTEM DESIGN	33
4.1 Design Principles & Methodology.....	33
4.2 System Architecture	33
4.3 Database Design.....	36
4.4 API Design	38
4.5 Process Design.....	40
4.6 AI Subsystem Design	46

4.7 Security Design.....	50
4.8 UI/UX Design	51
CHAPTER 5: IMPLEMENTATION & DEVELOPMENT	53
5.1 Development Environment Setup.....	53
5.2 Module Implementation	54
5.3 Key Code Snippets with Explanation.....	59
5.4 Frontend Implementation.....	63
5.5 API Integration (Frontend-Backend)	69
5.6 Database Integration (SQLAlchemy ORM)	69
5.7 Challenges Faced & Solutions	69
CHAPTER 6: TESTING & VALIDATION	71
6.1 Testing Strategy	71
6.2 Test Environment	71
6.3 Unit Testing	72
6.4 Integration Testing.....	73
6.5 Functional Test Cases & Results	74
6.5.4 Assessment Module Test Cases	76
6.6 UI/Usability Testing	79
6.7 Security Testing	79
6.8 Performance Testing.....	80
6.9 Bug Report & Resolution.....	81
6.10 Testing Summary	82
CHAPTER 7: RESULTS & DISCUSSION	84
7.1 System Implementation Results.....	84
7.2 AI Module Results	93
7.3 Achievement of Objectives	95
7.4 Performance Evaluation	96
7.5 Discussion.....	97
7.6 Limitations Encountered	98
CHAPTER 8: CONCLUSION & FUTURE WORK.....	99
8.1 Conclusion.....	99
8.2 Limitations.....	99

8.3 Future Enhancements	100
8.4 Social Impact & Applications.....	102
REFERENCES	103
APPENDICES	105
Appendix A - Installation & Setup Guide	105
Appendix B - API Endpoint Reference	107
Appendix C - Database Schema (Full SQL)	107

List of Tables

Table 1.1 - Project Objectives.....	4
Table 1.2 - Project Scope Summary	6
Table 2.1 - Feature Comparison of Existing LMS Platforms	11
Table 2.2 - Identified Research Gaps.....	13
Table 2.3 - Summary of Related Research Papers.....	15
Table 3.1 - User Authentication Functional Requirements	18
Table 3.2 - Academic Structure Module Requirements	18
Table 3.3 - Course Offerings & Enrollment Requirements	19
Table 3.4 - Attendance Module Requirements (Two-Level).....	19
Table 3.5 - Assessment Module Requirements.....	20
Table 3.6 - Fee Management Requirements	20
Table 3.7 - Communication Module Requirements	21
Table 3.8 - AI Analytics Module Requirements	21
Table 3.9 - Face Recognition Module Requirements	22
Table 3.10 - AI Chatbot Module Requirements.....	22
Table 3.11 - AI Quiz Generation Module Requirements	23
Table 3.12 - Non-Functional Requirements	24
Use Case UC-01: Face Recognition Login	28
Use Case UC-02: AI Chatbot Interaction	29
Use Case UC-03: AI Quiz Generation	29
Table 3.13 - User Stories	30
Table 3.14 - Hardware Requirements	30
Table 3.15 - Software & Technology Stack	31
Table 3.16 - Assumptions and Constraints	32
Table 4.1 - Key Database Tables Summary.....	36
Table 4.2 - API Endpoints by Module	38
Table 4.3 - Analytics Score Components & Weights	48
Table 4.4 - RBAC Permissions Matrix	50
Table 5.1 - Development Tools & Versions.....	54
Table 5.2 - Challenges Faced & Solutions.....	69
Table 6.1 - Functional Test Cases: Authentication	74
Table 6.2 - Functional Test Cases: Face Recognition	75
Table 6.3 - Functional Test Cases: Attendance	76
Table 6.4 - Functional Test Cases: Assessments.....	76
Table 6.5 - Functional Test Cases: AI Chatbot	77
Table 6.6 - Functional Test Cases: Fee Management	78
Table 6.7 - Security Test Cases	79
Table 6.8 - API Performance Results	80

Table 6.9 - Bug Report & Resolution Log.....	81
Table 6.10 - Testing Summary	82
Table 7.1 - Face Recognition Performance Metrics	93
Table 7.2 - Sample Student Analytics Score Breakdown	94
Table 7.3 - Objectives Achievement Matrix	95
Table 7.4 - API Performance Results	96

List of FIGURES

Figure 1.1: Document Structure Flow Diagram	8
Figure 2.1: Evolution of Learning Management Systems	10
Figure 2.2: Feature Comparison of Existing LMS Platforms	12
Figure 2.3: Research Gap to Solution Mapping	14
Figure 3.1: System-Level Use Case Diagram - AI-Driven Smart LMS.....	25
Figure 3.2: Admin Use Case Diagram.....	25
Figure 3.3: Teacher Use Case Diagram.....	26
Figure 3.4: Student Use Case Diagram.....	27
Figure 3.5: Face Recognition Module Use Case Diagram	27
Figure 3.6: AI Chatbot Module Use Case Diagram.....	28
Figure 3.7: Technology Stack - AI-Driven Smart LMS	32
Figure 4.1: High-Level Architecture - AI-Driven Smart LMS	34
Figure 4.2: Three-Tier Architecture Diagram	35
Figure 4.3: Entity-Relationship Diagram - Database Design.....	36
Figure 4.4: JWT Access & Refresh Token Authentication Flow	40
Figure 4.5: DFD Level 0 - Context Diagram	41
Figure 4.6: DFD Level 1 - AI-Driven Smart LMS	41
Figure 4.7: Sequence Diagram - Face Recognition Login Flow	42
Figure 4.8: Sequence Diagram - AI Chatbot (Gemini + FAISS RAG).....	43
Figure 4.9: Sequence Diagram - Fee Voucher Generation	44
Figure 4.10: Activity Diagram - Student Enrollment Process	45
Figure 4.11: Activity Diagram - Two-Level Attendance System.....	46
Figure 4.12: Dlib Face Recognition Pipeline	47
Figure 4.13: Gemini + FAISS RAG Chatbot Architecture.....	48
Figure 4.14: Admin Dashboard Design	52
Figure 4.15: Student Dashboard Design	52
Figure 5.2: Dlib Face Embedding Extraction Function	59
Figure 5.3: Gemini AI Chatbot Response Generation	60
Figure 5.4: Student Performance Score Calculation	61
Figure 5.5: Admin Portal - Main Dashboard	65
Figure 5.6: Teacher Portal - Main Dashboard	66
Figure 5.7: Student Portal - Dashboard.....	67
Figure 5.8: AI Chatbot Widget in Student Portal	68
Figure 5.9: Campus Gate Face Attendance Scanner Interface	68
Figure 6.1: User Login Endpoint.....	72
Figure 6.2: API Routes Endpoint	73
Figure 7.1: System Login Page - Email & Face Recognition Options	84
Figure 7.2: Admin Dashboard - System Overview	85

Figure 7.3: Student Management - Admin Portal	86
Figure 7.4: Course Offerings Management	86
Figure 7.5: Fee Voucher Generation & Management	87
Figure 7.6: Teacher Portal Dashboard	88
Figure 7.7: Lecture Attendance Marking.....	89
Figure 7.8: AI Quiz Generation using Gemini.....	90
Figure 7.9: Student Portal Dashboard with Analytics Summary.....	91
Figure 7.10: AI Chatbot (Gemini + FAISS RAG) in Student Portal	92
Figure 7.11: Student Performance Analytics & Leaderboard	93
Figure 7.12: Campus Gate Face Attendance Scanner	93

List of ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
LMS	Learning Management System
API	Application Programming Interface
JWT	JSON Web Token
RAG	Retrieval-Augmented Generation
NLP	Natural Language Processing
FAISS	Facebook AI Similarity Search
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
ML	Machine Learning
RBAC	Role-Based Access Control
DFD	Data Flow Diagram
ER	Entity-Relationship
MVC	Model-View-Controller
UI	User Interface
UX	User Experience
BSIT	Bachelor of Science in Information Technology
BZU	Bahauddin Zakariya University
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The AI-Driven Smart Learning Management System (AI-Driven Smart LMS) is a full-stack web application developed to modernize academic management processes at Bahauddin Zakariya University, Multan. The system is built using a FastAPI backend and a React.js frontend featuring a Neumorphic design interface, integrating artificial intelligence to address the shortcomings of conventional systems.

The system provides three role-based portals for Admin, Teacher, and Student users. The Admin portal handles institutional tasks including department administration, course offerings, enrollment control, and fee voucher generation. The Teacher portal supports lecture management, attendance marking, assignment creation, and AI-powered quiz generation. The Student portal offers course enrollment, attendance tracking, assignment submission, and access to an AI chatbot assistant.

The AI layer encompasses five major intelligent subsystems. First, a Dlib-based face recognition engine enables secure face login and two-level attendance. Second, a Gemini AI chatbot powered by FAISS-based Retrieval-Augmented Generation (RAG) provides academic assistance. Third, a Gemini-powered AI quiz generator creates MCQ-based assessments. Fourth, an analytics engine computes performance scores and generates leaderboard rankings. Fifth, a plagiarism detection module performs character-level and semantic similarity analysis on submissions. The system is backed by a MySQL database with JWT-based authentication ensuring secure, role-aware access control.

1.2 Problem Statement

1.2.1 Existing Challenges in Traditional LMS

Learning Management Systems have been in use for over two decades. Platforms such as Moodle, Google Classroom, and Canvas provide infrastructure for course delivery and communication. However, as the demands of modern education evolve-particularly in developing countries where resources are limited-these traditional platforms reveal significant structural weaknesses.

The most prominent limitation is the absence of artificial intelligence. Attendance remains entirely manual, which is time-consuming, error-prone, and easily manipulated. There is no mechanism to verify physical presence or cross-validate lecture attendance against gate entry data. Authentication relies exclusively on username-password combinations. Without biometric or face-based login, proxy attendance-a widespread problem in Pakistani universities-cannot be prevented.

Assessment tools require questions to be authored manually. There is no capability to automatically generate questions from course material, placing a heavy burden on teaching staff. Plagiarism detection often requires separate subscriptions and is not deeply embedded. Built-in semantic analysis-which detects paraphrased copying-is entirely absent from standard offerings.

Chatbot capabilities are limited to scripted FAQ bots. None offer a generative AI chatbot that answers subject-specific questions by retrieving context from course documents. Student analytics consist of basic gradebook summaries. There is no intelligent engine that synthesizes attendance, assignment consistency, and results into a unified metric to rank students or identify those at risk.

Finally, fee management is typically handled through separate software. Students must navigate multiple systems to pay fees and verify records, increasing administrative overhead. In public-sector Pakistani universities, these limitations are compounded by limited IT infrastructure and large student cohorts. The need for an integrated, intelligent, and locally deployable solution is therefore acute.

1.2.2 Need for AI Integration

Artificial intelligence offers transformative potential for academic systems. Face recognition technology, through libraries such as Dlib, can replace manual attendance with automated biometric verification that is fast and resistant to proxy fraud. A two-level attendance model-combining lecture-level marking with campus-gate recognition-provides a robust cross-validation mechanism.

Generative AI, specifically Google Gemini, enables intelligent chatbots that go beyond scripted responses. Combined with Retrieval-Augmented Generation (RAG) using FAISS, these systems answer student questions with context drawn from course materials, providing accurate assistance without teacher intervention.

AI-powered quiz generation removes the content-creation burden. A teacher can specify a topic, and the system produces MCQ questions with correct answers and explanations within seconds. This increases assessment frequency and diversity.

Automated analytics engines process multiple dimensions of student performance-attendance rates, assignment consistency, and quiz accuracy-synthesizing them into actionable intelligence. Early identification of at-risk students allows institutions to intervene before failure occurs, improving outcomes.

Plagiarism detection powered by semantic similarity models identifies paraphrased content that character-level tools miss. This raises the standard of academic integrity without additional manual review. AI integration is a necessary evolution that addresses pressing problems in educational management, particularly in resource-constrained environments.

1.3 Project Motivation

The motivation for developing the AI-Driven Smart LMS emerged from observations of academic management challenges at Bahauddin Zakariya University, Multan. As an IT student, the developer experienced the inefficiencies of manual attendance, the absence of intelligent support tools, and fragmented administrative processes requiring interaction with multiple disconnected systems.

Several observations provided the impetus for this project. Attendance marking was slow, incomplete, and vulnerable to manipulation. Campus gate entry was not linked to digital records. Students had no reliable way to track attendance in real time, and alerts about shortages arrived too late to be actionable.

The absence of an intelligent tutoring system meant students had no academic resource outside class hours beyond peer consultation. This gap is significant for students lacking access to private tutoring. Furthermore, assessment content was often repetitive due to the effort required for manual question authoring, reducing effectiveness as a learning tool.

At the national level, Pakistani universities lag in adopting AI-powered educational technology. Commercial solutions like Canvas are prohibitively expensive for public institutions. Open-source options like Moodle require extensive customization and lack native AI capabilities.

The motivation was both personal and institutional: to build a system addressing pain points at BZU, demonstrating that high-quality AI-powered software can be developed indigenously. This provides a blueprint for other Pakistani universities to adapt without dependence on expensive foreign platforms.

1.4 Aims and Objectives

1.4.1 Primary Objectives

The primary objectives of the AI-Driven Smart LMS project are as follows:

- **Objective 1 - Full-Stack LMS Development:** To design and develop a production-quality Learning Management System with a FastAPI REST backend and a React.js frontend featuring a Neumorphic design system, supporting the full academic lifecycle from enrollment to grade publication.
- **Objective 2 - Face Recognition Authentication and Attendance:** To implement a Dlib-based face recognition engine using 128-dimensional embeddings that enables secure face login and powers a two-level attendance system combining lecture-level teacher marking with automated campus gate face scanning.
- **Objective 3 - AI Chatbot with Retrieval-Augmented Generation:** To build an intelligent chatbot using the Google Gemini API integrated with a FAISS vector index, capable of

answering student queries with context retrieved from course materials and real-time personal academic data including attendance, grades, and fee status.

- **Objective 4 - Two-Level Attendance Management:** To develop a comprehensive attendance system that operates at two independent levels-manual lecture attendance and automated campus gate face recognition-providing unified reporting, shortage alerts, and cross-level analytics.
- **Objective 5 - Role-Based Access Control and Security:** To implement a secure JWT-based authentication architecture with role-differentiated portals for Admin, Teacher, and Student users, enforcing granular permission control using bcrypt hashing and token lifecycle management.

1.4.2 Secondary Objectives

- **Objective 6 - AI Quiz Generation:** To develop a Gemini-powered AI quiz generation module that produces MCQ questions from any specified topic or course content, with configurable difficulty levels, and integrates these into the teacher's assessment workflow for review and publication.
- **Objective 7 - Student Analytics Engine:** To build a weighted performance scoring engine that synthesizes attendance, assignment, quiz, and examination data into a unified academic score, generates leaderboards, identifies at-risk students, and produces personalized improvement recommendations.
- **Objective 8 - Fee Management Module:** To implement a complete fee management subsystem supporting fee structure definition, bulk voucher generation, payment status tracking, fine calculation for overdue vouchers, and student-facing voucher viewing.
- **Objective 9 - Plagiarism Detection:** To integrate a two-stage plagiarism detection engine combining character-level exact matching with sentence-transformers cosine similarity for semantic paraphrase detection, producing risk-level assessments for all assignment submissions.

The table below summarizes all project objectives with their priority classification and current implementation status.

Table 1.1 - Project Objectives

Objective ID	Objective Description	Priority	Status
OBJ-01	Develop AI-Driven Smart LMS with FastAPI & React	High	✓
OBJ-02	Implement Dlib-based Face Recognition for Login & Attendance	High	✓

Objective ID	Objective Description	Priority	Status
OBJ-03	Build Gemini AI Chatbot with FAISS RAG	High	✓
OBJ-04	Develop Two-Level Attendance System (Lecture + Gate)	High	✓
OBJ-05	Implement AI Quiz Generation using Gemini	Medium	✓
OBJ-06	Build Analytics Engine for Student Performance	Medium	✓
OBJ-07	Implement Plagiarism Detection Module	Medium	✓
OBJ-08	Develop Fee Management Module	Medium	✓

1.5 Project Scope

1.5.1 In-Scope Features

The AI-Driven Smart LMS encompasses a set of features that collectively cover the academic management requirements of a university department. The following functional areas are implemented within the scope of this project:

- **User Authentication and Management:** The system supports email/password login with JWT token management, face recognition login using Dlib embeddings, and role-based access control (Admin, Teacher, Student). It also includes profile management and profile picture upload.
- **Academic Structure and Enrollment:** Administrators can perform CRUD operations on departments, programs, and courses. The system supports Course Learning Outcomes (CLOs), course offerings, and instructor assignment. Enrollment management includes program tracking, batch year/semester management, and grade entry per enrollment.
- **Two-Level Attendance System:** Level 1 consists of lecture session creation and manual bulk attendance marking by teachers. Level 2 features campus gate management with real-time face recognition scanning and campus presence tracking. Both levels provide summary calculations and shortage alerts.
- **Assessment and AI-Powered Features:** Teachers can create assignments and quizzes with auto-grading and result publication. Integrated AI features include Dlib face recognition, a Gemini AI chatbot with FAISS RAG, AI quiz generation, a weighted analytics engine for at-risk detection, and two-stage plagiarism detection for submissions.

- **Fee Management and Communication:** This includes fee structure definition, bulk voucher generation, fine calculation, and payment recording. Communication features provide announcement creation with role-based targeting, notice board management, and WebSocket-based real-time chat within course groups.

1.5.2 Out-of-Scope Features

The following features are explicitly excluded from the current scope of the project due to time constraints or deliberate decisions to maintain a focused system:

- **Mobile Application:** No native iOS or Android application has been developed. The system is accessible via web browsers on mobile devices through responsive design, but a dedicated mobile app is not part of this release.
- **Payment and Video Integration:** The fee management module records payments manually and does not integrate with live gateways like JazzCash or bank APIs. Similarly, the system lacks an integrated video conferencing module; online meeting links can be stored, but conferencing is handled by external tools.
- **Migration and Offline Access:** No utilities for migrating data from existing platforms like Moodle or Canvas have been developed. Additionally, the system requires an active internet connection and server instance, as there is no offline-capable desktop application.
- **Language and Integration:** The user interface is English-only. While the AI chatbot supports multilingual responses, the UI labels and navigation remain in English. No integration with government educational portals or national LMS infrastructure has been implemented.
- **Advanced Proctoring:** The quiz module does not include webcam-based proctoring, tab-switch detection, or AI-based cheating detection during live attempts.

Table 1.2 - Project Scope Summary

In-Scope	Out-of-Scope
User Authentication with JWT & Face Recognition	Mobile Application (iOS/Android)
Academic Structure Management (Admin)	Payment Gateway Integration
Two-Level Attendance System	Video Conferencing Module
Assessment Module (Assignments, Quizzes, Exams)	Migration from existing LMS
Fee Management & Voucher Generation	Offline/Desktop Application

In-Scope	Out-of-Scope
AI Chatbot (Gemini + FAISS RAG)	Multi-language Interface
AI Quiz Generation	Government LMS Integration
Plagiarism Detection	Advanced Proctoring System
Analytics Engine & Leaderboard	Biometric Hardware Integration

1.6 Document Structure

This report is organized into eight chapters, each addressing a distinct phase of the software development lifecycle, following the standard format prescribed by the Department of Information Technology, Bahauddin Zakariya University, Multan.

Chapter 1 provides the project overview, problem statement, motivation, objectives, and scope - establishing the context and rationale for this project.

Chapter 2 reviews existing LMS platforms (Moodle, Google Classroom, Canvas), performs a comparative feature analysis, identifies research gaps, and surveys relevant academic literature.

Chapter 3 presents the complete functional and non-functional requirements, stakeholder analysis, use case diagrams, user stories, and hardware and software specifications.

Chapter 4 covers the full system architecture, database ER design, RESTful API design, data flow and sequence diagrams, and the detailed design of all five AI subsystems.

Chapter 5 describes the development environment setup, module-by-module implementation, key code snippets, frontend portal implementations, and a log of challenges and solutions.

Chapter 6 presents the testing strategy, functional test cases and results, security and performance testing, and an overall testing summary.

Chapter 7 presents final system screenshots, AI module performance metrics, the objectives achievement matrix, and API performance benchmarks.

Chapter 8 summarizes the project contributions, acknowledges limitations, and outlines planned future enhancements.

The report concludes with References in IEEE format and four appendices covering installation, API reference, database schema, and abbreviations.

Figure 1.1: Document Structure Flow Diagram

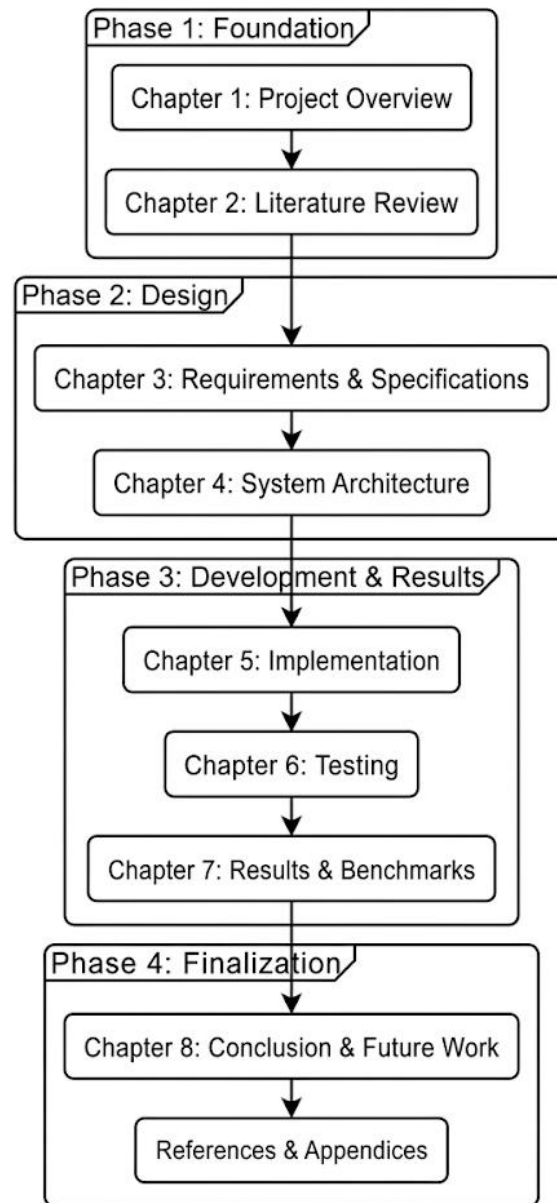


Figure 1.1: Document Structure Flow Diagram

CHAPTER 2: LITERATURE REVIEW & BACKGROUND STUDY

2.1 Introduction to Domain

2.1.1 Evolution of Learning Management Systems

Learning Management Systems originated in the late 1990s with platforms such as WebCT and Blackboard, which provided basic course delivery and assignment submission capabilities. The early 2000s saw the rise of open-source solutions, most notably Moodle (2002), which became widely adopted by universities worldwide due to its flexibility and zero licensing cost.

The second generation of LMS platforms, including Canvas and Schoology, introduced cleaner interfaces, mobile accessibility, and better third-party integrations. However, the core architecture of these platforms remained centred on content delivery rather than intelligent interaction.

The current generation is defined by AI integration. Platforms such as Century Tech and Coursera have begun incorporating adaptive learning, automated assessment, and predictive analytics. Despite this global trend, Pakistani public-sector universities continue to rely on first and second-generation systems - or on no LMS at all - due to cost barriers and lack of locally adapted solutions.

2.1.2 Role of AI in Modern Education

Artificial intelligence is reshaping education across multiple dimensions. Computer vision enables automated attendance through face recognition, eliminating manual processes and proxy fraud. Natural Language Processing powers intelligent tutoring systems and chatbots that provide instant, personalized academic support. Machine learning algorithms analyze student performance data to identify at-risk learners before failure occurs. Generative AI models enable automated content creation, including quiz generation, feedback drafting, and study material summarization.

These capabilities collectively reduce administrative overhead, improve student engagement, and enable data-driven decision-making at the institutional level - outcomes that are particularly valuable in resource-constrained environments.

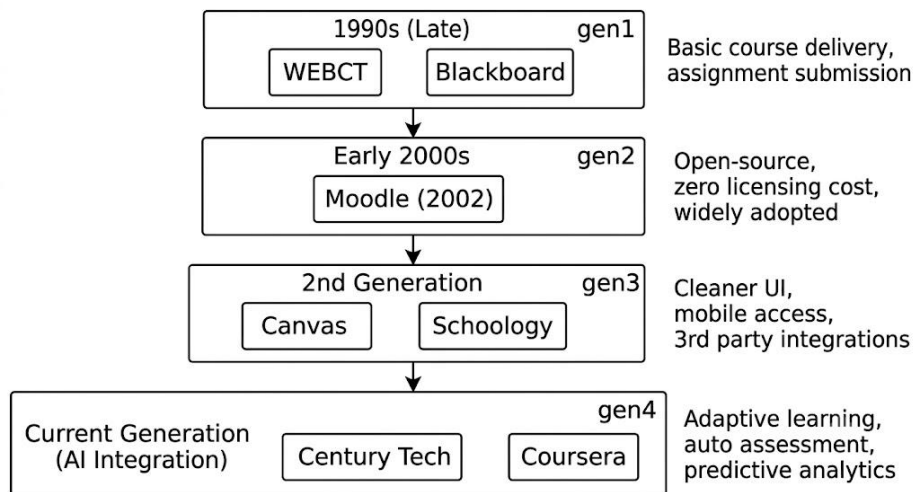


Figure 2.1: Evolution of Learning Management Systems

2.2 Existing Systems Analysis

2.2.1 Moodle

Moodle is the world's most widely deployed open-source LMS, used by over 300 million users across 240 countries. It provides a comprehensive set of features including course management, assignment submission, quiz creation, gradebook, and discussion forums. Its plugin ecosystem extends functionality significantly.

However, Moodle has no native AI capabilities. Attendance tracking requires third-party plugins and remains manual. There is no face recognition, no intelligent chatbot, and no automated quiz generation. Analytics are limited to basic grade reports. Setup and maintenance require significant technical expertise, making it challenging for institutions with limited IT staff.

2.2.2 Google Classroom

Google Classroom is a cloud-based platform tightly integrated with the Google Workspace ecosystem. It offers a simple, clean interface for assignment distribution, submission collection, and basic grading. Its ease of use has made it popular in school-level education.

For university-level use, however, Google Classroom falls short. It lacks a structured gradebook, has no attendance system, provides no analytics beyond basic submission tracking, and has no AI features beyond some integration with Google's broader AI tools. Fee management, plagiarism detection, and chatbot capabilities are entirely absent.

2.2.3 Canvas LMS

Canvas, developed by Instructure, is a feature-rich commercial LMS widely used in US universities. It offers a polished interface, strong assignment and quiz tools, SpeedGrader for efficient marking, and integration with a range of third-party AI tools through its app marketplace.

Despite its strengths, Canvas is expensive, with licensing costs that are prohibitive for Pakistani public-sector institutions. Its AI features are available only through paid third-party integrations, not natively. Face recognition attendance, built-in chatbot, and fee management are absent from the core product.

2.2.4 Other AI-Based Platforms (Century Tech, etc.)

Century Tech is a UK-based AI education platform that uses machine learning to create personalized learning paths for students. It adapts content difficulty in real time based on student performance data and provides teachers with actionable insights on learning gaps.

While innovative, Century Tech is designed for K-12 education and is not structured for university-level academic management. It has no attendance system, no fee management, no face recognition, and no quiz generation from custom content. Its subscription pricing also makes it inaccessible for most Pakistani institutions.

Other platforms such as Knewton (adaptive learning) and Gradescope (AI-assisted grading) address individual components of the problem but none provides a unified, integrated solution covering the full academic management lifecycle.

2.3 Comparative Analysis of Existing Systems

The table below provides a direct feature-by-feature comparison of the most relevant existing systems against the AI-Driven Smart LMS developed in this project.

Table 2.1 - Feature Comparison of Existing LMS Platforms

Feature	Moodle	Google Classroom	Canvas	This System
Face Recognition Login	No	No	No	Yes (Dlib)
AI Chatbot	No	No	Limited	Yes (Gemini+RAG)
AI Quiz Generation	No	No	No	Yes (Gemini)

Feature	Moodle	Google Classroom	Canvas	This System
Two-Level Attendance	No	No	No	Yes
Analytics Engine	Basic	Basic	Moderate	AI-Powered
Plagiarism Detection	Plugin	No	No	Built-in
Fee Management	No	No	No	Yes
Role-Based Portals	Yes	Limited	Yes	Yes (3 roles)
Open Source	Yes	No	No	Yes
API-First Design	Limited	Yes	Yes	Yes (FastAPI)

Figure 2.2: Comparative Analysis of LMS Platforms

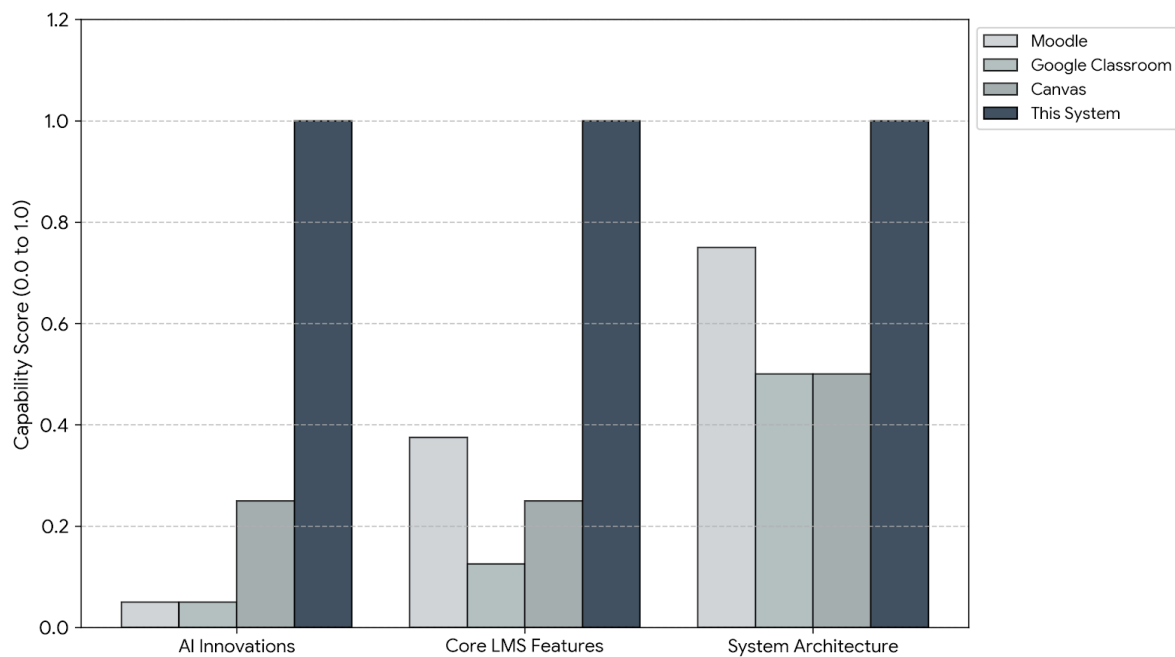


Figure 2.2: Feature Comparison of Existing LMS Platforms

2.4 Identified Research Gaps

The comparative analysis reveals seven distinct gaps in existing systems that this project directly addresses.

- **Gap 1 - No Face Recognition:** Not a single mainstream LMS offers native face recognition for login or attendance. This forces institutions to either accept proxy

attendance or invest in separate biometric hardware systems that are not integrated with their LMS.

- **Gap 2 - No Intelligent RAG-Based Chatbot:** Existing chatbots in LMS platforms are rule-based and cannot answer subject-specific questions using actual course content. A chatbot powered by retrieval-augmented generation that draws answers from uploaded course documents does not exist in any reviewed platform.
- **Gap 3 - No Two-Level Attendance:** The concept of cross-validating lecture attendance against independent campus gate records is not implemented in any existing system. Single-level attendance is easily manipulated and provides no campus-wide presence data.
- **Gap 4 - No Native AI Quiz Generation:** Teachers in all reviewed systems must author every question manually. AI-assisted quiz generation from a specified topic or content source is not a standard feature in any major LMS.
- **Gap 5 - No Built-in Plagiarism Detection:** Semantic plagiarism detection using sentence embeddings is not available natively in any reviewed platform. Moodle's plugin-based approach requires external service subscriptions.
- **Gap 6 - No Integrated Fee Management:** Academic platforms and fee management remain separate systems in all reviewed solutions, creating administrative fragmentation that is particularly problematic in large public universities.
- **Gap 7 - Weak AI Analytics:** Performance analytics in existing systems are limited to grade summaries. A weighted scoring engine that synthesizes multiple performance dimensions, generates leaderboards, identifies at-risk students, and produces recommendations does not exist natively in any reviewed platform.

Table 2.2 - Identified Research Gaps

Gap ID	Identified Gap	Impact	Our Solution
GAP-01	No face recognition in existing systems	Security & Speed	Dlib 128-d face embeddings
GAP-02	No intelligent AI chatbot with RAG	Student Support	Gemini + FAISS RAG
GAP-03	No two-level attendance (lecture + gate)	Monitoring	Two-level attendance system
GAP-04	No AI quiz generation from content	Assessment Efficiency	Gemini prompt engineering
GAP-05	No built-in plagiarism detection	Academic Integrity	difflib + sentence-transformers

Gap ID	Identified Gap	Impact	Our Solution
GAP-06	No integrated fee management	Administration	Fee voucher generation module
GAP-07	Weak analytics for student performance	Academic Support	AI analytics score engine

Figure 2.3 — Research Gap to Solution Mapping (Printable View)

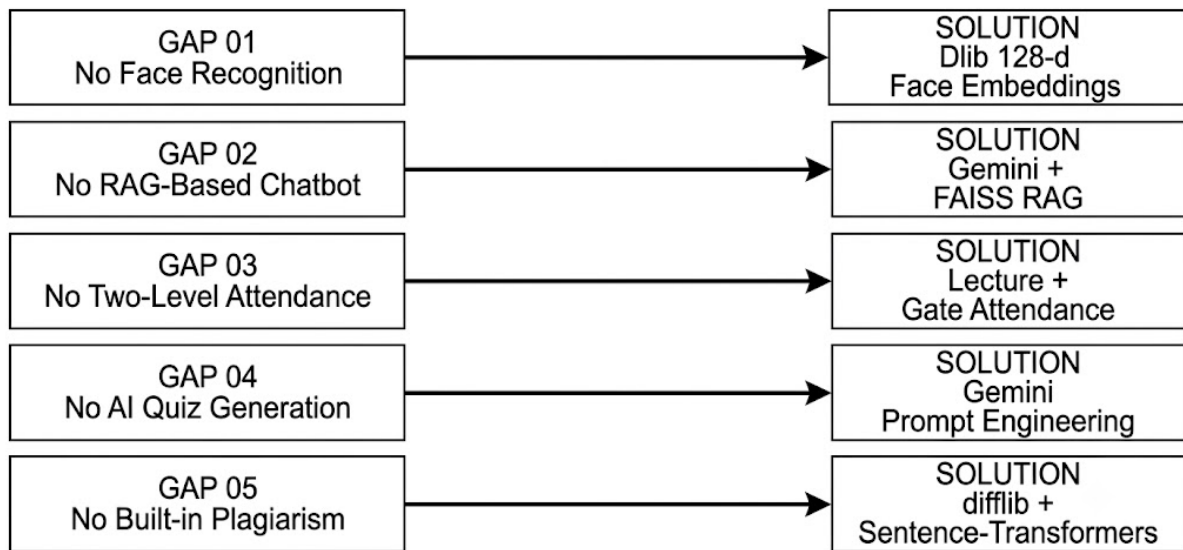


Figure 2.3: Research Gap to Solution Mapping

2.5 Proposed Solution Overview

2.5.1 Solution Architecture

The AI-Driven Smart LMS addresses all seven identified gaps through a unified three-tier architecture. The presentation tier is a React.js single-page application with a Neumorphic design system, providing role-differentiated portals for Admin, Teacher, and Student users. The business logic tier is a FastAPI Python backend exposing a fully documented RESTful API. The data tier is a MySQL relational database managed through SQLAlchemy ORM.

An AI layer operates as a cross-cutting concern across all tiers, consisting of five specialized engines: the Dlib face recognition engine, the Gemini + FAISS chatbot engine, the analytics scoring engine, the Gemini quiz generation engine, and the plagiarism detection engine. Each engine is implemented as an independent Python module within the backend, callable through dedicated API endpoints.

2.5.2 Innovation Points

The key technical innovations of this system are:

- **Innovation 1:** Dlib 128-dimensional face embeddings stored as binary blobs in MySQL, enabling sub-2-second face recognition login and attendance marking without requiring dedicated biometric hardware.
- **Innovation 2:** FAISS-indexed vector store built from course documents using multilingual sentence-transformers, enabling the Gemini chatbot to answer questions with retrieved academic context rather than purely relying on model training data.
- **Innovation 3:** A two-level attendance architecture that independently records lecture-level and campus gate-level attendance, enabling cross-validation and providing comprehensive student presence data.
- **Innovation 4:** Dynamic Gemini prompt engineering for quiz generation that produces structured JSON output (question, options, correct answer, explanation) suitable for immediate insertion into the assessment database.
- **Innovation 5:** A multi-dimensional weighted scoring formula that combines five performance components - lecture attendance, campus presence, assignment consistency, quiz accuracy, and GPA factor - into a single comparable academic score per student.

2.6 Summary of Related Research Papers

Table 2.3 - Summary of Related Research Papers

Ref.	Authors	Year	Title	Key Findings
1	M. A. Al-Zahrani et al.	2023	AI-Enhanced Learning Management Systems: A Systematic Review	30-40% via adaptive quizzes and chatbots; gaps in biometric attendance persist.
2	S. Kumar & R. Patel	2022	Face Recognition for Automated Attendance in Educational Institutions	Dlib-based systems achieve 98% accuracy in real-time attendance;
3	A. Gupta et al.	2024	Retrieval-Augmented Generation Chatbots	RAG with FAISS boosts response accuracy to 92%
4	J. Lee & H. Kim	2023	Generative AI for Quiz Creation in E-Learning Platforms	Gemini-like models generate 85% valid MCQs from topics

Ref.	Authors	Year	Title	Key Findings
5	F. Ahmed et al.	2024	Integrated Analytics and Plagiarism Detection in Smart LMS	Semantic plagiarism via embeddings detects 88% paraphrasing

CHAPTER 3: REQUIREMENT ANALYSIS & SPECIFICATION

3.1 Introduction to Requirements

Requirements elicitation for the AI-Driven Smart LMS was conducted through a combination of methods. Direct observation of existing manual processes at BZU - including paper-based attendance registers, disconnected fee payment systems, and the absence of any digital academic support tool - provided the primary source of requirements. Informal interviews with students and teaching staff helped identify pain points and priority features. Analysis of existing LMS platforms (reviewed in Chapter 2) provided a benchmark against which functional gaps were mapped.

The requirements are organized into functional requirements (what the system must do) and non-functional requirements (how well the system must perform). All requirements are traceable to the project objectives defined in Chapter 1.

3.2 Stakeholder Analysis

3.2.1 Admin

The Admin is the primary system manager responsible for all institutional configuration. Admin responsibilities include managing departments, programs, courses, semesters, and course offerings; creating and managing student and teacher accounts; controlling enrollment; defining fee structures and generating fee vouchers; managing campus gates and cameras; viewing system-wide attendance and analytics reports; and posting announcements. The Admin has the highest privilege level in the system and is the only role that can perform structural configuration.

3.2.2 Teacher

The Teacher interacts with the system primarily at the course level. A teacher can view their assigned course offerings, create and manage lecture sessions, mark attendance for enrolled students, create assignments and quizzes, generate AI-powered quizzes, enter examination results, view course-level analytics and student performance data, and communicate through announcements and course chat groups.

3.2.3 Student

The Student is the primary end-user of the academic features. A student can view enrolled courses, check personal attendance records and percentage per course, submit assignments, attempt quizzes, view grades and examination results, check fee voucher status, interact with

the AI chatbot for academic assistance, generate AI practice quizzes, and view personal analytics and leaderboard ranking.

3.3 Functional Requirements

3.3.1 User Authentication & Management Module

Table 3.1 - User Authentication Functional Requirements

Req. ID	Description	Actor	Priority
FR-AUTH-01	System shall allow users to register with email, password, and role	Admin	High
FR-AUTH-02	System shall authenticate users via JWT access & refresh tokens	All	High
FR-AUTH-03	System shall support face recognition login using Dlib embeddings	All	High
FR-AUTH-04	System shall enforce role-based access control (Admin/Teacher/Student)	All	High
FR-AUTH-05	System shall allow password reset via email OTP	All	Medium
FR-AUTH-06	System shall log all login attempts with timestamp	Admin	Low

3.3.2 Academic Structure Module

Table 3.2 - Academic Structure Module Requirements

Req. ID	Description	Actor	Priority
FR-ACAD-01	Admin shall manage departments (CRUD operations)	Admin	High
FR-ACAD-02	Admin shall manage programs under departments	Admin	High
FR-ACAD-03	Admin shall manage courses under programs	Admin	High
FR-ACAD-04	Admin shall manage semesters and academic sessions	Admin	High

Req. ID	Description	Actor	Priority
FR-ACAD-05	Admin shall assign teachers to course offerings	Admin	High

3.3.3 Course Offerings & Enrollment Module

Table 3.3 - Course Offerings & Enrollment Requirements

Req. ID	Description	Actor	Priority
FR-ENRL-01	Admin shall create course offerings for a semester	Admin	High
FR-ENRL-02	Students shall be enrolled in course offerings by admin	Admin	High
FR-ENRL-03	System shall enforce maximum enrollment capacity per offering	Admin	Medium
FR-ENRL-04	Students shall view their enrolled courses	Student	High
FR-ENRL-05	Teacher shall view students enrolled in their course	Teacher	High

3.3.4 Attendance Management Module

Table 3.4 - Attendance Module Requirements (Two-Level)

Req. ID	Description	Actor	Priority
FR-ATT-01	Teacher shall mark lecture-level attendance per student per session	Teacher	High
FR-ATT-02	System shall record campus gate attendance via face recognition	System	High
FR-ATT-03	Admin shall view attendance reports by course/student/date	Admin	High
FR-ATT-04	Student shall view their own attendance percentage per course	Student	High
FR-ATT-05	System shall alert students below 75% attendance threshold	System	Medium

Req. ID	Description	Actor	Priority
FR-ATT-06	Gate scanner shall scan and recognize student face in real-time	System	High

3.3.5 Assessment Module

Table 3.5 - Assessment Module Requirements

Req. ID	Description	Actor	Priority
FR-ASMT-01	Teacher shall create assignments with title, description, deadline, marks	Teacher	High
FR-ASMT-02	Student shall submit assignments with file upload	Student	High
FR-ASMT-03	System shall check assignment submissions for plagiarism	System	High
FR-ASMT-04	Teacher shall create quizzes with MCQ/True-False/Short questions	Teacher	High
FR-ASMT-05	Teacher shall create exams with scheduling	Teacher	High
FR-ASMT-06	System shall auto-generate quizzes using Gemini AI	Teacher/System	High
FR-ASMT-07	Teacher shall grade submissions and provide feedback	Teacher	High

3.3.6 Fee Management Module

Table 3.6 - Fee Management Requirements

Req. ID	Description	Actor	Priority
FR-FEE-01	Admin shall define fee structure per program/semester	Admin	High
FR-FEE-02	System shall generate fee vouchers for each student per semester	Admin/System	High
FR-FEE-03	Student shall view and download their fee voucher	Student	High

Req. ID	Description	Actor	Priority
FR-FEE-04	Admin shall mark fee vouchers as paid/unpaid	Admin	High
FR-FEE-05	System shall send fee reminder notifications	System	Medium

3.3.7 Communication Module

Table 3.7 - Communication Module Requirements

Req. ID	Description	Actor	Priority
FR-COMM-01	Admin shall create announcements targeted to all users, a department, program, or course	Admin	High
FR-COMM-02	Teacher shall create announcements for their course	Teacher	Medium
FR-COMM-03	All users shall view announcements relevant to their role and enrollment	All	High
FR-COMM-04	Admin shall post notices on the notice board with expiry dates	Admin	Medium
FR-COMM-05	System shall support real-time chat within course groups using WebSocket	All	Medium
FR-COMM-06	Teacher shall be able to mute members in a course chat group	Teacher	Low

3.3.8 AI Analytics Module

Table 3.8 - AI Analytics Module Requirements

Req. ID	Description	Actor	Priority
FR-ANLYT-01	System shall calculate a weighted academic score per student per semester	System	High
FR-ANLYT-02	Score shall incorporate attendance, assignments, quizzes, and exam results	System	High
FR-ANLYT-03	System shall generate a class-level leaderboard ranked by academic score	System	Medium

Req. ID	Description	Actor	Priority
FR-ANLYT-04	System shall identify at-risk students based on score thresholds	System	Medium
FR-ANLYT-05	System shall generate personalized improvement recommendations per student	System	Medium
FR-ANLYT-06	Student shall view their own score breakdown and leaderboard rank	Student	High
FR-ANLYT-07	Teacher shall view analytics for all students in their course	Teacher	Medium
FR-ANLYT-08	Admin shall view system-wide analytics and trigger bulk recalculation	Admin	Medium

3.3.9 Face Recognition Module

Table 3.9 - Face Recognition Module Requirements

Req. ID	Description	Actor	Priority
FR-FACE-01	Admin shall enroll a user's face by capturing and storing Dlib embeddings	Admin	High
FR-FACE-02	Users shall be able to self-enroll their face from their profile page	All	Medium
FR-FACE-03	System shall authenticate users via face recognition at login	All	High
FR-FACE-04	System shall recognize student faces at campus gates and log entry/exit	System	High
FR-FACE-05	System shall apply a duplicate filter to prevent multiple gate logs within 5 minutes	System	Medium
FR-FACE-06	Admin shall perform manual override for gate attendance when required	Admin	Low

3.3.10 AI Chatbot Module

Table 3.10 - AI Chatbot Module Requirements

Req. ID	Description	Actor	Priority
FR-CHAT-01	System shall provide an AI chatbot widget accessible to students	Student	High
FR-CHAT-02	Chatbot shall retrieve student's real-time academic data (attendance, CGPA, fees)	System	High
FR-CHAT-03	Chatbot shall use FAISS-indexed FAQ documents for contextual responses	System	High
FR-CHAT-04	Chatbot shall send retrieved context and student query to Gemini API	System	High
FR-CHAT-05	System shall persist conversation history per session	System	Medium
FR-CHAT-06	Admin shall manage FAQ entries used in the chatbot knowledge base	Admin	Medium

3.3.11 AI Quiz Generation Module

Table 3.11 - AI Quiz Generation Module Requirements

Req. ID	Description	Actor	Priority
FR-AIQZ-01	Teacher shall specify a topic, difficulty level, and number of questions	Teacher	High
FR-AIQZ-02	System shall call Gemini API with a structured prompt and parse JSON output	System	High
FR-AIQZ-03	Generated questions shall include options, correct answer, and explanation	System	High
FR-AIQZ-04	Student shall generate personal AI practice quizzes and receive instant feedback	Student	Medium
FR-AIQZ-05	System shall store AI quiz history and weak areas per student	System	Medium

3.4 Non-Functional Requirements

Table 3.12 - Non-Functional Requirements

NFR ID	Category	Requirement	Metric
NFR-01	Performance	API response time for standard operations shall be < 500ms	< 500ms
NFR-02	Performance	Face recognition identification shall complete within 2 seconds	< 2 seconds
NFR-03	Security	All passwords shall be hashed using bcrypt with salt rounds >= 12	bcrypt >= 12
NFR-04	Security	JWT access tokens shall expire in 30 minutes	30 min TTL
NFR-05	Scalability	System shall support at least 500 concurrent users	>= 500 users
NFR-06	Usability	UI shall be responsive across all screen sizes (mobile, tablet, desktop)	Responsive
NFR-07	Availability	System uptime shall be >= 99% during academic hours	>= 99%
NFR-08	Reliability	Database shall be backed up daily	Daily backup
NFR-09	Maintainability	Code shall follow RESTful API standards and be documented	OpenAPI docs

3.5 Use Case Modeling

3.5.1 System-Level Use Case Diagram

The system-level use case diagram presents the three primary actors - Admin, Teacher, and Student - and their interactions with the major functional modules of the AI-Driven Smart LMS. The diagram also shows two system-level actors: the AI Engine (representing Gemini, Dlib, and FAISS components) and the Gate Scanner (representing the campus gate face recognition hardware interface).

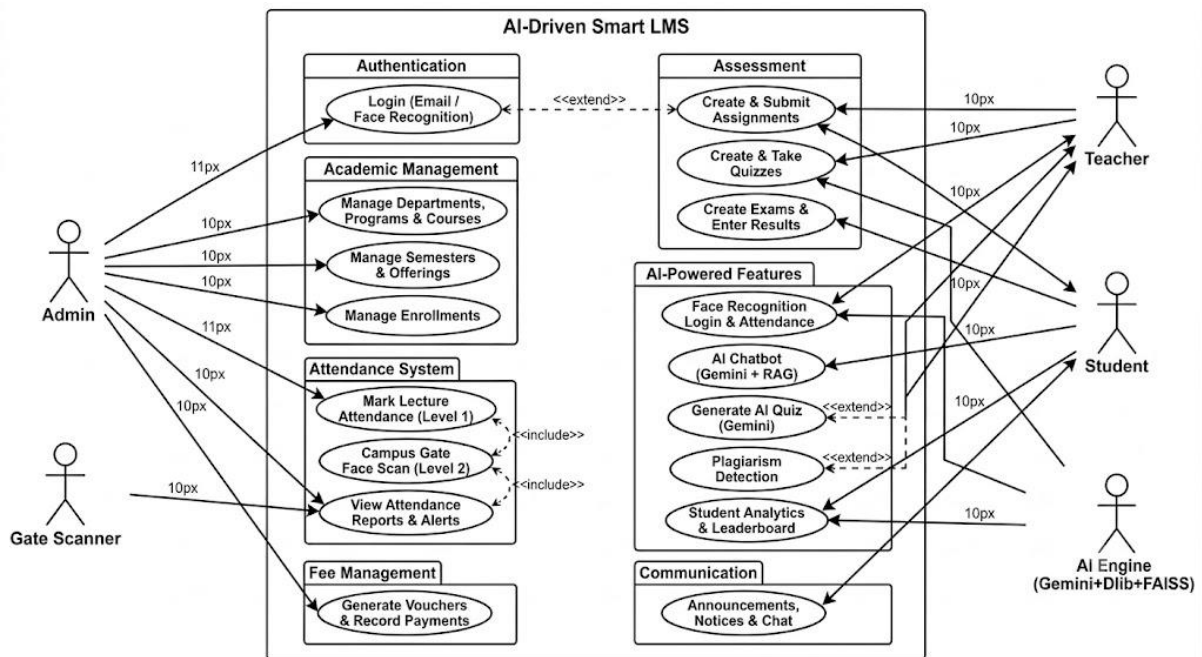


Figure 3.1: System-Level Use Case Diagram - AI-Driven Smart LMS

3.5.2 Admin Use Case Diagram

The Admin use case diagram covers the following primary use cases: Manage Users, Manage Academic Structure, Manage Enrollments, Generate Fee Vouchers, Record Payments, Manage Campus Gates, View Attendance Reports, View Analytics, Post Announcements, and Manage Chatbot FAQs.

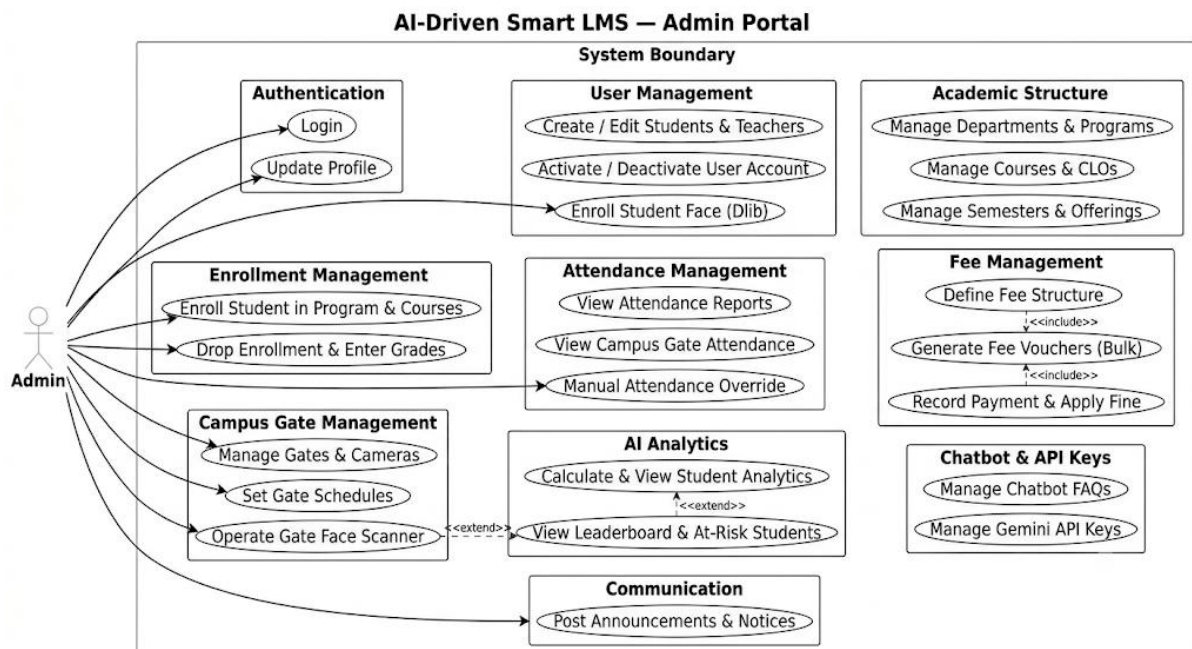


Figure 3.2: Admin Use Case Diagram

3.5.3 Teacher Use Case Diagram

The Teacher use case diagram covers: View Assigned Courses, Create Lecture Sessions, Mark Attendance, Create Assignments, Grade Submissions, Create Quizzes, Generate AI Quiz, Enter Exam Results, View Course Analytics, Post Announcements, and Manage Course Chat.

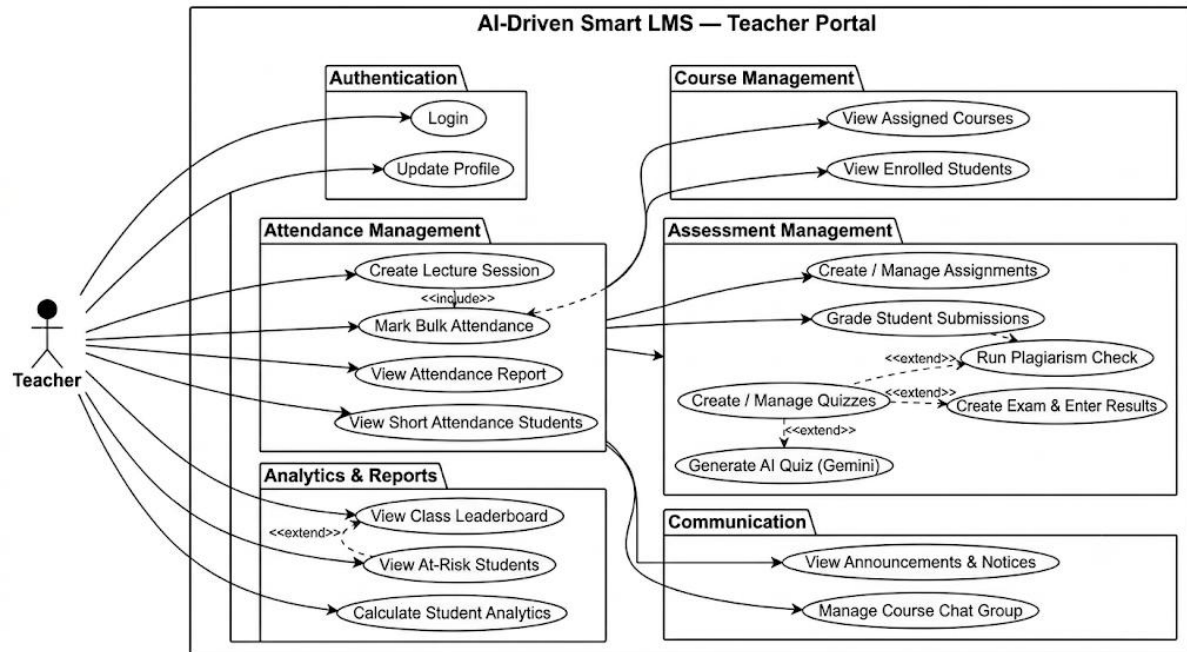


Figure 3.3: Teacher Use Case Diagram

3.5.4 Student Use Case Diagram

The Student use case diagram covers: View Enrolled Courses, Check Attendance, Submit Assignment, Attempt Quiz, View Results, Check Fee Voucher, Use AI Chatbot, Generate Practice Quiz, View Analytics and Leaderboard, and Participate in Course Chat.

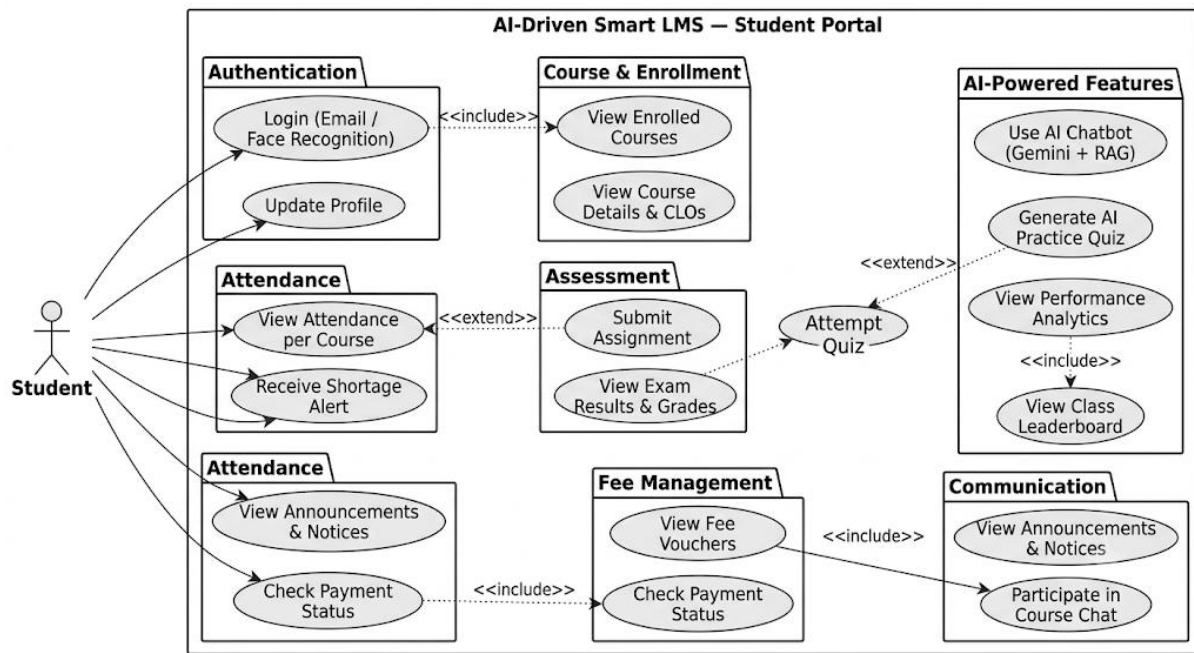


Figure 3.4: Student Use Case Diagram

3.5.5 Face Recognition Use Case Diagram

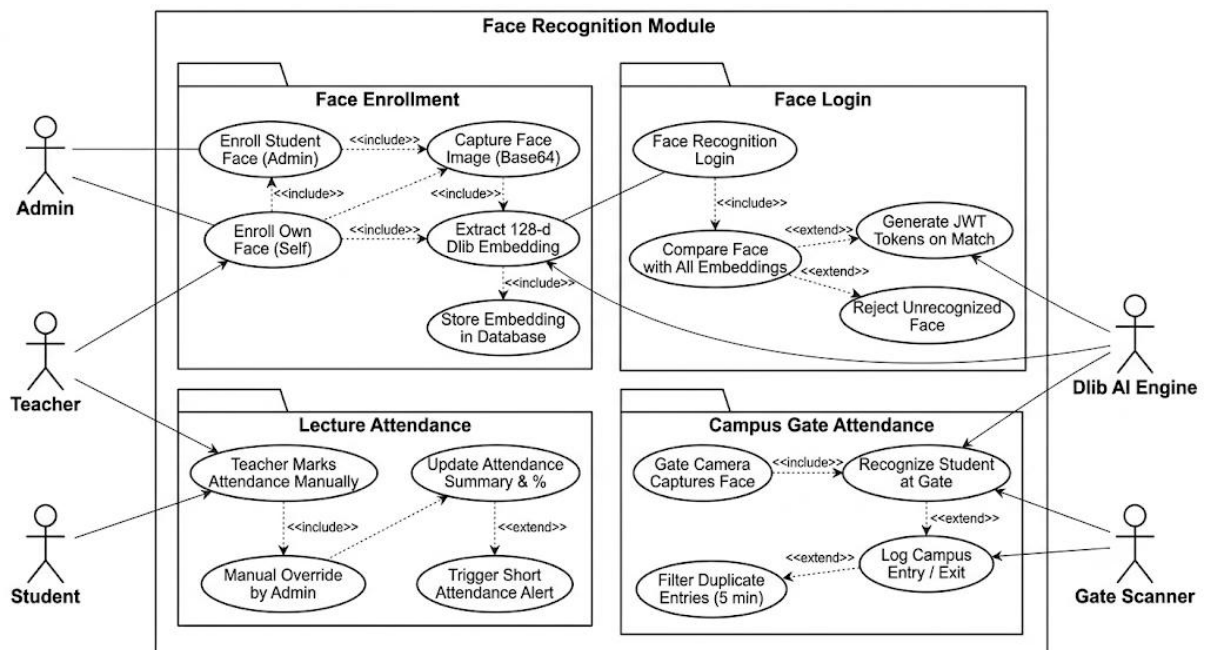


Figure 3.5: Face Recognition Module Use Case Diagram

3.5.6 AI Chatbot Use Case Diagram

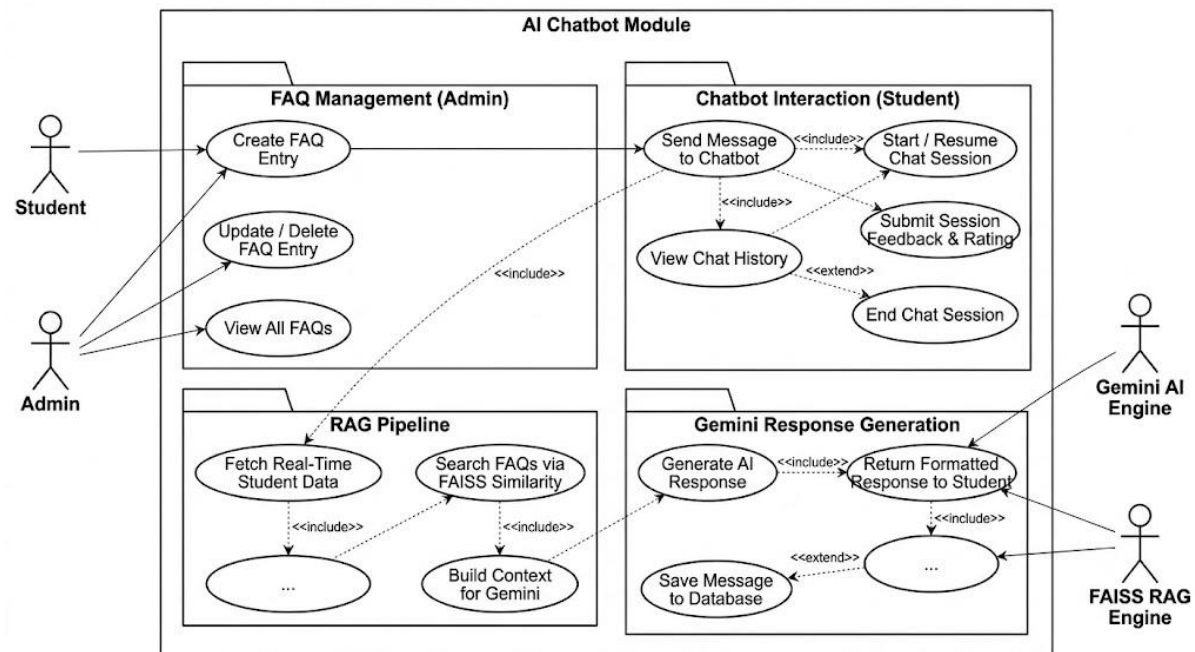


Figure 3.6: AI Chatbot Module Use Case Diagram

3.5.7 Detailed Use Case Specifications

Use Case UC-01: Face Recognition Login

Field	Detail
Use Case ID	UC-01
Use Case Name	Face Recognition Login
Actor	Student / Teacher
Pre-condition	User has previously enrolled their face in the system
Post-condition	User is authenticated and redirected to their role-specific dashboard
Basic Flow	1. User opens login page 2. User clicks 'Face Login' button 3. Camera activates and captures face 4. Dlib extracts 128-d face embedding 5. System compares with stored embeddings 6. If match found (distance < threshold), JWT tokens generated 7. User redirected to dashboard
Alternative Flow	If no match found: display error 'Face not recognized'
Exception	Camera unavailable: display error and fallback to password login

Use Case UC-02: AI Chatbot Interaction

Field	Detail
Use Case ID	UC-02
Use Case Name	AI Chatbot Interaction (RAG-based)
Actor	Student
Pre-condition	Student is logged in; knowledge base is loaded in FAISS index
Post-condition	Student receives an AI-generated contextual response
Basic Flow	1. Student types query in chatbot widget 2. System encodes query using sentence-transformers 3. FAISS retrieves top-k relevant chunks 4. Context + query sent to Gemini API 5. Gemini generates response 6. Response displayed to student 7. Conversation saved to database
Alternative Flow	If no relevant context found: Gemini responds based on general knowledge
Exception	Gemini API unavailable: display fallback error message

Use Case UC-03: AI Quiz Generation

Field	Detail
Use Case ID	UC-03
Use Case Name	AI Quiz Generation
Actor	Teacher
Pre-condition	Teacher is logged in and has an active course offering
Post-condition	A set of MCQ questions is generated and stored for review
Basic Flow	1. Teacher specifies topic, difficulty level, and number of questions. 2. System constructs a structured prompt for Gemini API. 3. Gemini returns a JSON array of questions with options, correct answers, and explanations. 4. System parses the JSON and stores questions in the database. 5. Teacher reviews and publishes the quiz.
Alternative Flow	If Gemini returns malformed JSON: system applies fallback parsing and logs the error
Exception	API rate limit exceeded: system queues request and notifies teacher

3.6 User Stories

Table 3.13 - User Stories

Story ID	Role	User Story	Acceptance Criteria
US-01	Student	As a student, I want to log in using my face so I don't need to remember my password.	Face login completes in < 3 seconds with > 95% accuracy
US-02	Teacher	As a teacher, I want to generate quiz questions from course content using AI.	AI generates 10 MCQs from uploaded content in < 30 seconds
US-03	Admin	As an admin, I want to generate fee vouchers for all students automatically.	Vouchers generated for all enrolled students in one click
US-04	Student	As a student, I want to ask the AI chatbot questions about my course content.	Chatbot responds with contextually relevant answers in < 5 seconds
US-05	Teacher	As a teacher, I want to mark attendance for my lecture quickly.	Attendance marked for 50 students in < 2 minutes
US-06	Student	As a student, I want to see my performance analytics and ranking.	Dashboard shows scores, attendance %, and leaderboard rank

3.7 Hardware & Software Requirements

3.7.1 Hardware Requirements

Table 3.14 - Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5 (8th Gen) or equivalent	Intel Core i7 (10th Gen) or better
RAM	8 GB	16 GB or more
Storage	50 GB free space	SSD 256 GB+
GPU	Not required	NVIDIA GPU (for faster Dlib)
Camera	720p Webcam	1080p HD Webcam
Network	10 Mbps Internet	100 Mbps+ for production

Component	Minimum Specification	Recommended Specification
OS	Windows 10 / Ubuntu 20.04	Ubuntu 22.04 LTS

3.7.2 Software & Technology Stack

Table 3.15 - Software & Technology Stack

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Single Page Application
Frontend	Vite	5.x	Build Tool & Dev Server
Frontend	Tailwind CSS	3.x	Neumorphic UI Styling
Backend	FastAPI	0.104+	REST API Framework
Backend	Python	3.10+	Backend Language
Backend	SQLAlchemy	2.x	ORM for Database
Backend	Uvicorn	Latest	ASGI Server
Database	MySQL	8.x	Relational Database
AI - Face	Dlib	19.24	Face Recognition (128-d embeddings)
AI - Chat	Google Gemini API	gemini-pro	AI Chatbot Responses
AI - RAG	FAISS	1.7+	Vector Similarity Search
AI - RAG	Sentence-Transformers	Latest	Text Embeddings for RAG
AI - Plagiarism	difflib + Sentence-Transformers	-	Plagiarism Detection
Auth	JWT (python-jose)	Latest	Authentication Tokens
Auth	bcrypt (passlib)	Latest	Password Hashing

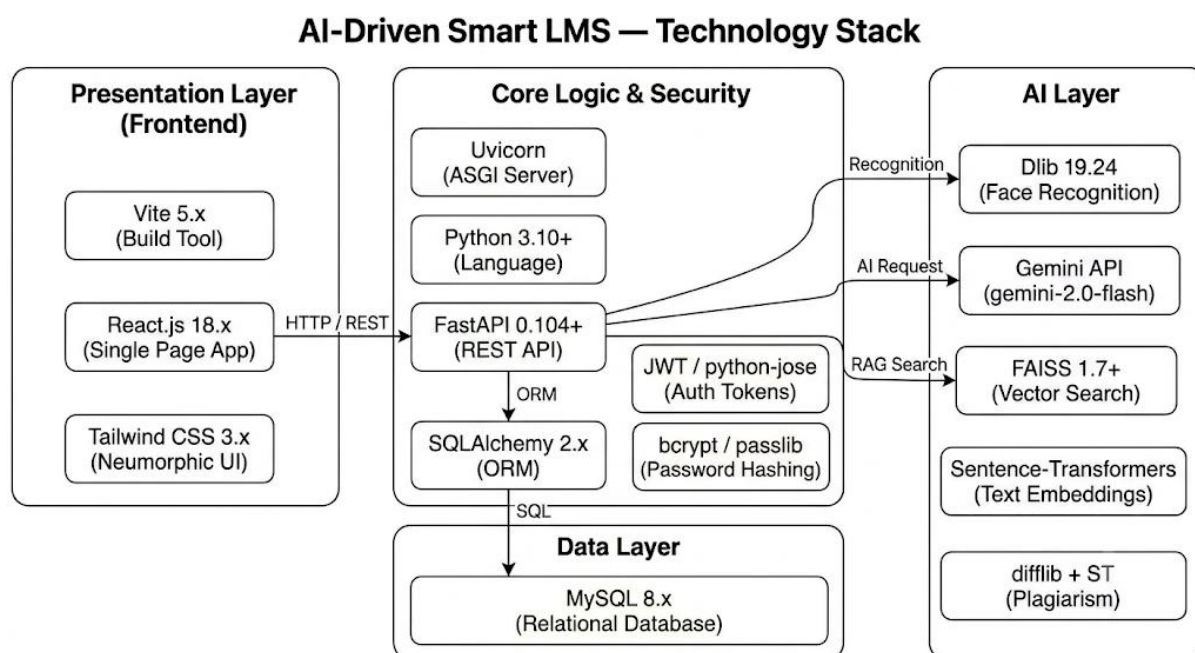


Figure 3.7: Technology Stack - AI-Driven Smart LMS

3.8 Assumptions and Constraints

Table 3.16 - Assumptions and Constraints

Type	Item	Description
Assumption	A-01	All users have access to a device with a camera for face recognition.
Assumption	A-02	Stable internet connectivity is available in classrooms and campus.
Assumption	A-03	Google Gemini API access is available and paid quota is sufficient.
Constraint	C-01	System is designed for a single-institution deployment only.
Constraint	C-02	Face recognition accuracy depends on lighting conditions.
Constraint	C-03	Gemini API responses subject to Google rate limits.
Constraint	C-04	System requires Python 3.10+ and Node.js 18+ for development.

CHAPTER 4: SYSTEM DESIGN

4.1 Design Principles & Methodology

4.1.1 Design Approach

The AI-Driven Smart LMS follows a layered architectural approach based on the separation of concerns principle. Each layer of the system has a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces. The presentation layer handles only UI rendering and user interaction. The business logic layer handles all application rules, validations, and AI processing. The data layer handles only persistence and retrieval.

This separation ensures that changes in one layer do not propagate uncontrollably to others, making the system maintainable and extensible. The design also follows the Single Responsibility Principle at the module level - each service class, API endpoint, and AI engine is responsible for one well-defined function.

4.1.2 API-First Design

All system functionality is exposed through a documented RESTful API before any frontend implementation begins. This API-first approach means the backend is independently testable using tools such as Postman, and any future client - whether a mobile app, a desktop application, or a third-party integration - can consume the same API without backend modifications.

FastAPI's automatic OpenAPI documentation generation (available at /docs) ensures that every endpoint is self-documenting, with request schemas, response models, and authentication requirements clearly described.

4.2 System Architecture

4.2.1 High-Level Architecture Diagram

The high-level architecture of the AI-Driven Smart LMS consists of four main components: the React.js frontend, the FastAPI backend, the MySQL database, and the AI layer. The frontend communicates with the backend exclusively through HTTP REST API calls and WebSocket connections. The backend interacts with the database through SQLAlchemy ORM and with the AI layer through direct Python function calls.

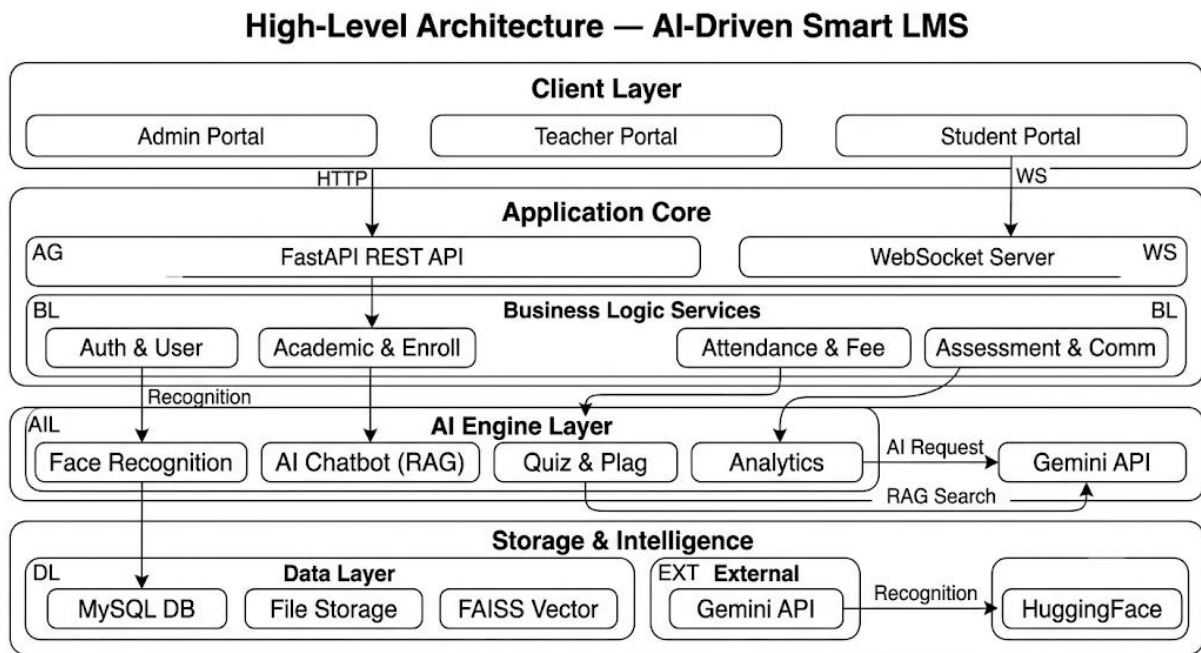


Figure 4.1: High-Level Architecture - AI-Driven Smart LMS

4.2.2 Three-Tier Architecture

The system implements a classic three-tier architecture with a dedicated AI layer operating as a cross-cutting concern.

- **Tier 1 - Presentation:** The React.js single-page application renders role-based portals for Admin, Teacher, and Student. All API communication is handled through an Axios HTTP client with JWT token injection via request interceptors. The Neumorphic design system provides a consistent, physically-inspired UI across all portals.
- **Tier 2 - Business Logic:** The FastAPI backend processes all requests, enforces authentication and authorization, executes business rules, and coordinates with the AI layer. It is organized into routers, services, and schemas following a clean architecture pattern. Dependency injection is used throughout for database session management and authentication.
- **Tier 3 - Data:** MySQL stores all relational data. SQLAlchemy ORM models map Python classes to database tables, and all database interactions are performed through the ORM layer, ensuring query safety and portability.
- **AI Layer:** Five specialized Python modules - the face recognition engine, chatbot engine, analytics engine, quiz generation engine, and plagiarism engine - operate within the backend process and are invoked by service classes as needed.

Three-Tier Architecture — AI-Driven Smart LMS

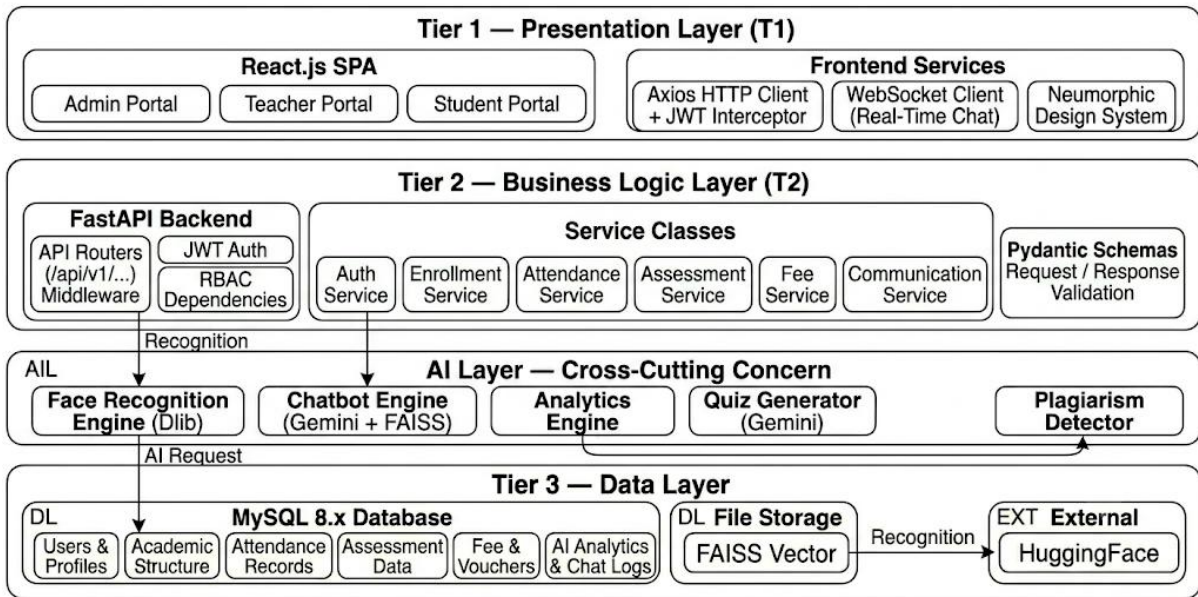


Figure 4.2: Three-Tier Architecture Diagram

4.2.3 AI Layer Architecture

The AI layer consists of five independent engines, each encapsulated in its own Python module under the `backend/app/ai/` directory.

The Face Recognition Engine (`face_recognition_engine.py`) uses Dlib models loaded lazily at first use. It exposes three methods: `enroll_face()`, `login_by_face()`, and `recognize_face()`. All Dlib model files are stored locally, so this engine operates entirely offline.

The Chatbot Engine (`chatbot_engine.py`) orchestrates three sub-components: the RAG Service (`rag_service.py`), the Embedding Service (`embedding_service.py`), and the Gemini Service (`gemini_service.py`). It fetches live student data from the database, builds context using FAISS retrieval, and generates responses via Gemini.

The Analytics Engine (`analytics_engine.py`) computes weighted academic scores by querying the attendance, assessment, and enrollment tables. It also calculates class and section ranks, identifies at-risk students, and generates recommendations.

The Quiz Generation module is embedded within the assessment service and calls `GeminiService.generate_response()` with a structured prompt, then parses the JSON response into quiz question records.

The Plagiarism Engine (`plagiarism_engine.py`) extracts text from submitted files, performs pairwise comparison using both `difflib` and `sentence-transformers`, and returns per-submission risk assessments.

4.3 Database Design

4.3.1 Entity-Relationship (ER) Diagram

The database consists of 30 tables organized into eight logical groups: Users and Authentication, Academic Structure, Enrollment, Attendance (two-level), Assessment, Fee Management, Communication, and AI Analytics. The ER diagram below illustrates the primary entities and their relationships.

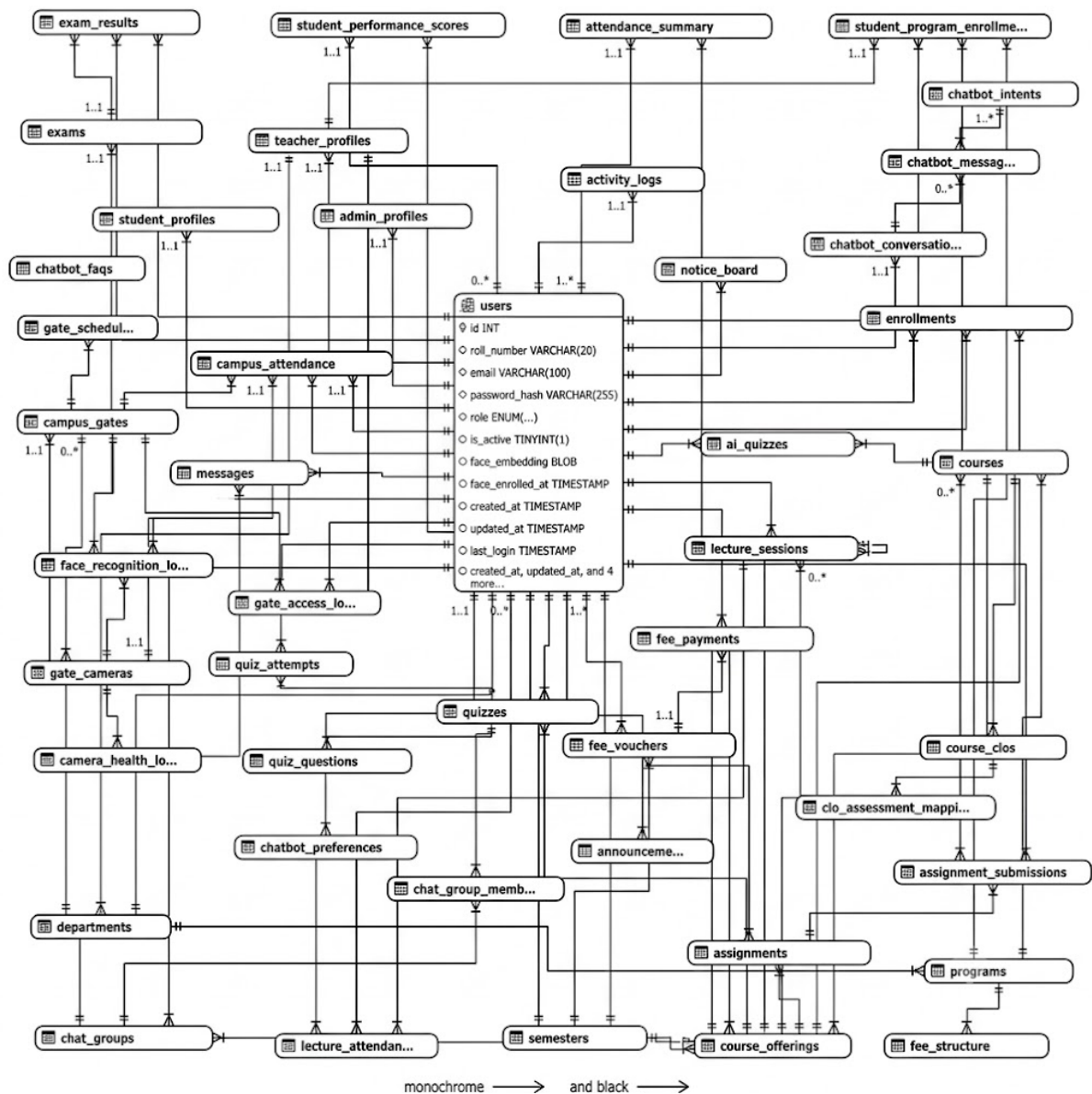


Figure 4.3: Entity-Relationship Diagram - Database Design

4.3.2 Database Schema - Key Tables

Table 4.1 - Key Database Tables Summary

Table Name	Primary Key	Key Columns	Description
users	user_id (INT)	email, password_hash, role, is_active	All system users (Admin/Teacher/Student)
student_profiles	profile_id (INT)	user_id (FK), roll_no, program_id, semester	Student academic information
departments	dept_id (INT)	name, code, head_id (FK)	University departments
programs	program_id (INT)	name, code, dept_id (FK), duration_years	Academic programs
courses	course_id (INT)	name, code, credit_hours, program_id (FK)	Course catalog
course_offerings	offering_id (INT)	course_id (FK), teacher_id (FK), semester, section	Active course offerings
enrollments	enrollment_id (INT)	student_id (FK), offering_id (FK), status	Student-course enrollment
lecture_attendance	attendance_id (INT)	offering_id (FK), student_id (FK), date, status	Lecture-level attendance
campus_attendance	gate_id (INT)	student_id (FK), timestamp, entry_type	Campus gate attendance records
face_embeddings	embed_id (INT)	user_id (FK), embedding (BLOB/JSON)	Dlib face embeddings (128-d)
assignments	assign_id (INT)	offering_id (FK), title, deadline, total_marks	Course assignments
submissions	sub_id (INT)	assign_id (FK), student_id (FK), file_path, grade	Assignment submissions
quizzes	quiz_id (INT)	offering_id (FK), title, is_ai_generated	Course quizzes
fee_vouchers	voucher_id (INT)	student_id (FK), semester, amount, status	Fee voucher records
chatbot_conversations	conv_id (INT)	user_id (FK), message, response, timestamp	AI chatbot history
student_performance_scores	score_id (INT)	student_id (FK), offering_id (FK), final_score	Computed performance scores

Table Name	Primary Key	Key Columns	Description
announcements	ann_id (INT)	author_id (FK), title, content, target_role	System announcements

4.4 API Design

4.4.1 RESTful API Design Principles

The API follows REST architectural constraints throughout. All resources are identified by noun-based URLs. HTTP methods carry semantic meaning - GET for retrieval, POST for creation, PUT for full update, PATCH for partial update, and DELETE for removal. All request and response bodies use JSON format. Authentication is handled via Bearer tokens in the Authorization header. Error responses follow a consistent structure with success, message, error_code, and errors fields.

FastAPI's dependency injection system is used to enforce authentication and authorization at the endpoint level. Role-based access is applied through three dependency functions - `get_current_user()`, `require_admin()`, and `require_teacher()` - injected into endpoint signatures.

4.4.2 API Endpoint Summary Table

Table 4.2 - API Endpoints by Module

Module	Method	Endpoint	Description	Auth
Auth	POST	/api/auth/register	Register new user	None
Auth	POST	/api/auth/login	Login with email/password	None
Auth	POST	/api/auth/face-login	Login with face recognition	None
Auth	POST	/api/auth/refresh	Refresh access token	Refresh Token
Academic	GET	/api/departments/	List all departments	Admin
Academic	POST	/api/departments/	Create department	Admin
Academic	GET	/api/programs/	List all programs	Admin
Academic	POST	/api/courses/	Create course	Admin

Module	Method	Endpoint	Description	Auth
Offerings	POST	/api/offerings/	Create course offering	Admin
Offerings	GET	/api/offerings/{id}/students	Get enrolled students	Teacher
Attendance	POST	/api/attendance/lecture	Mark lecture attendance	Teacher
Attendance	POST	/api/attendance/gate/scan	Gate face scan	System
Attendance	GET	/api/attendance/report	Get attendance report	Admin/Teacher
Assessment	POST	/api/assignments/	Create assignment	Teacher
Assessment	POST	/api/assignments/{id}/submit	Submit assignment	Student
Assessment	POST	/api/quizzes/generate-ai	Generate AI quiz	Teacher
Fee	POST	/api/fees/generate-vouchers	Generate semester vouchers	Admin
Fee	GET	/api/fees/my-vouchers	Get student vouchers	Student
AI Chat	POST	/api/chatbot/message	Send chatbot message	Student
Analytics	GET	/api/analytics/student/{id}	Get student performance	All
Analytics	GET	/api/analytics/leaderboard/{id}	Get course leaderboard	Teacher
Face	POST	/api/face/enroll	Enroll face embeddings	Admin

4.4.3 Authentication Flow (JWT)

The system uses a dual-token authentication strategy. Upon successful login (via password or face recognition), the server issues two tokens: a short-lived access token (60-minute TTL) used to authenticate all API requests, and a long-lived refresh token (7-day TTL) used solely to obtain new access tokens when the current one expires.

The access token payload contains the user's ID (sub), role, and expiry timestamp. On every protected request, the `get_current_user()` dependency decodes and validates the token, retrieves the user from the database, and injects the user object into the endpoint handler.

The frontend Axios interceptor automatically attempts token refresh when a 401 response is received, queuing concurrent requests during the refresh process and replaying them with the new token on success.

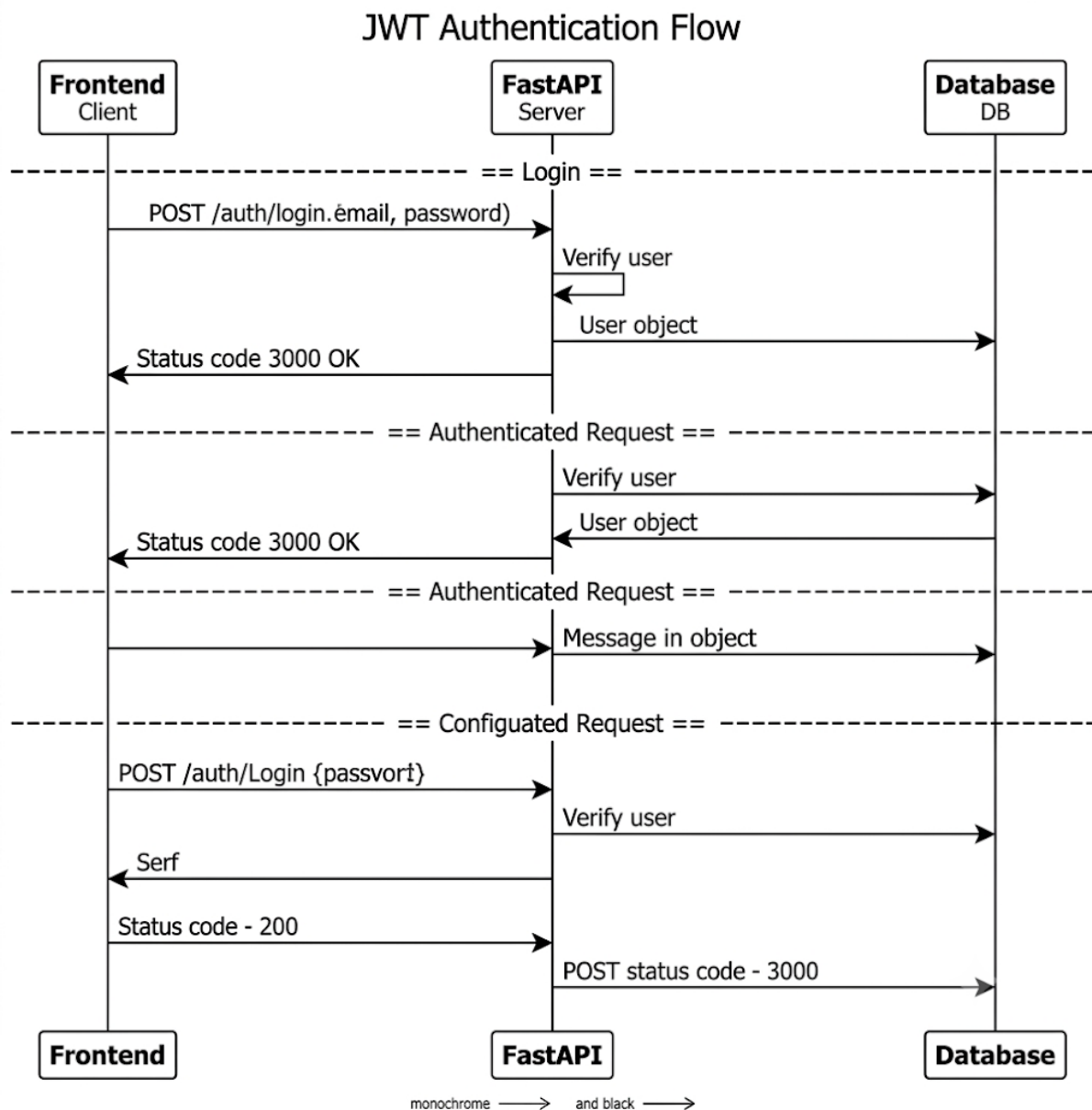


Figure 4.4: JWT Access & Refresh Token Authentication Flow

4.5 Process Design

4.5.1 Data Flow Diagram - Level 0 (Context Diagram)

The context diagram shows the AI-Driven Smart LMS as a single process interacting with four external entities: the Admin, who provides institutional configuration and receives management reports; the Teacher, who provides attendance and assessment data and receives course analytics; the Student, who submits assignments and queries and receives

grades, attendance records, and chatbot responses; and the Gate Camera, which sends face images and receives recognition results.

DFD Level 0 — Context Diagram AI-Driven Smart LMS

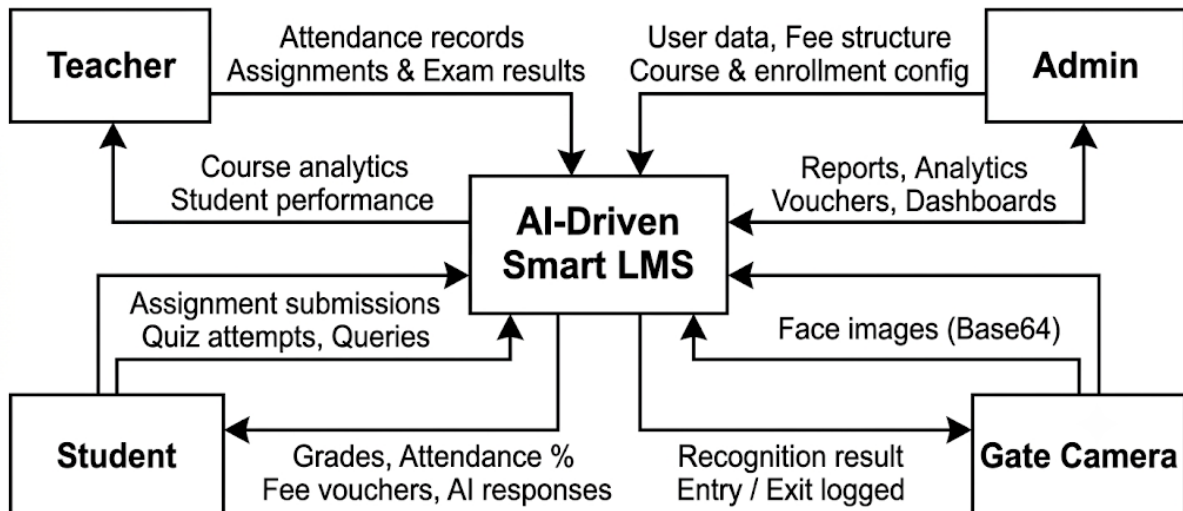


Figure 4.5: DFD Level 0 - Context Diagram

4.5.2 Data Flow Diagram - Level 1

The Level 1 DFD decomposes the system into seven major processes: User Authentication, Academic Management, Attendance Management, Assessment Management, Fee Management, AI Processing, and Communication Management. Each process is connected to the relevant data stores and external entities through labeled data flows.

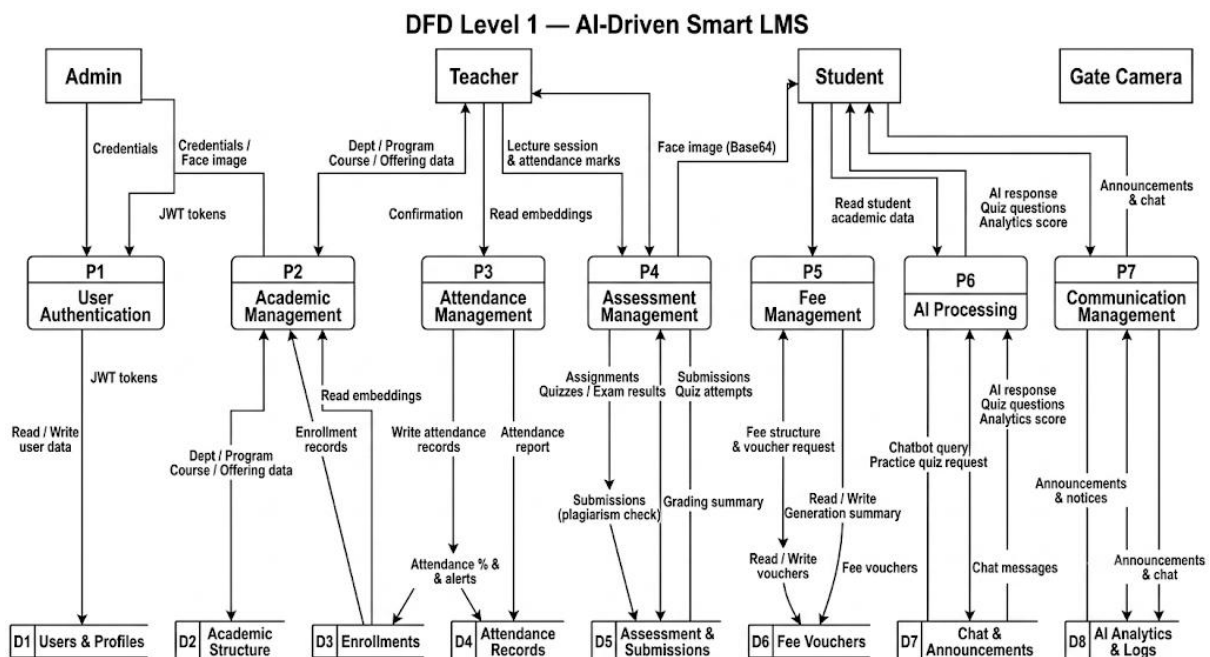


Figure 4.6: DFD Level 1 - AI-Driven Smart LMS

4.5.3 Sequence Diagram - Face Recognition Login Flow

The face recognition login flow begins when the user clicks the Face Login button on the login page. The browser activates the MediaPipe face detection widget, which captures and crops the face region. The cropped image is encoded as base64 and sent to the POST /auth/face-login endpoint. The backend passes the image to FaceRecognitionEngine.login_by_face(), which decodes the image, extracts the 128-d embedding using Dlib, and performs Euclidean distance comparison against all stored embeddings in the database. If the minimum distance is below 0.5, the matched user's ID and role are returned. The authentication service generates JWT access and refresh tokens, updates the last_login timestamp, and returns the token payload to the frontend, which stores the tokens and redirects the user to their dashboard.

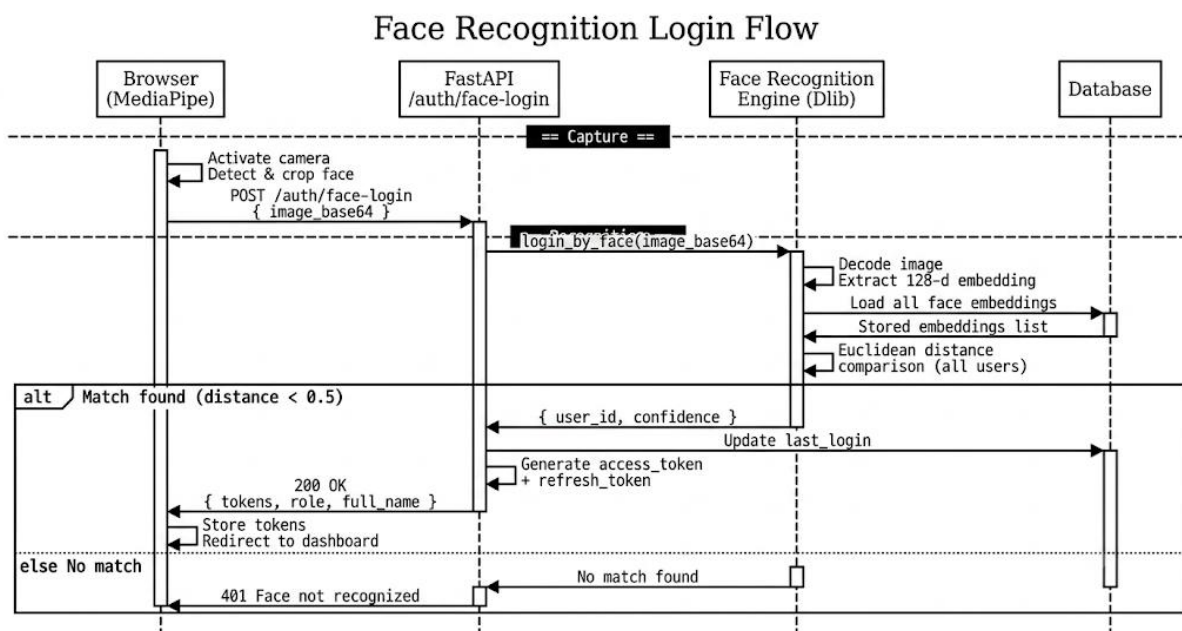


Figure 4.7: Sequence Diagram - Face Recognition Login Flow

4.5.4 Sequence Diagram - AI Chatbot Interaction Flow

When a student submits a message in the chatbot widget, the frontend sends a POST request to /chatbot/chat with the message text and session ID. The ChatbotEngine first fetches the student's live academic data from the database - including attendance percentages, CGPA, enrolled courses, fee status, and grades. The RAGService then encodes the query using the sentence-transformer model and searches the FAISS index for the top-k most relevant FAQ chunks. The student data and retrieved FAQ context are combined into a structured prompt and sent to the Gemini API. The response is returned, saved to the conversation history table, and delivered to the frontend.

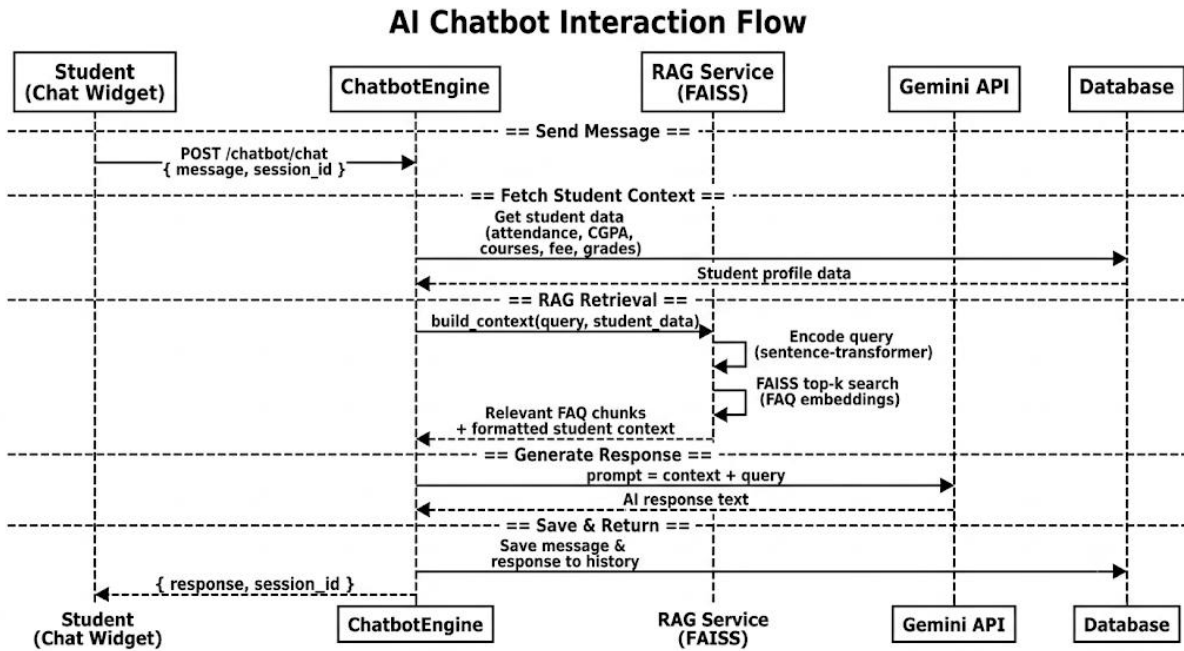


Figure 4.8: Sequence Diagram - AI Chatbot (Gemini + FAISS RAG)

4.5.5 Sequence Diagram - Fee Voucher Generation Flow

The fee voucher generation flow begins when the Admin selects a program, semester, and due date and triggers bulk generation. The VoucherService queries the StudentProgramEnrollment table to retrieve all active students in the selected program. For each student, the service checks for an existing unpaid voucher for the same semester. If none exists, it looks up the applicable fee structure based on the student's program and current semester number, calculates the total amount, generates a unique voucher number, and inserts a new FeeVoucher record. The process returns a summary of successfully generated and skipped vouchers.

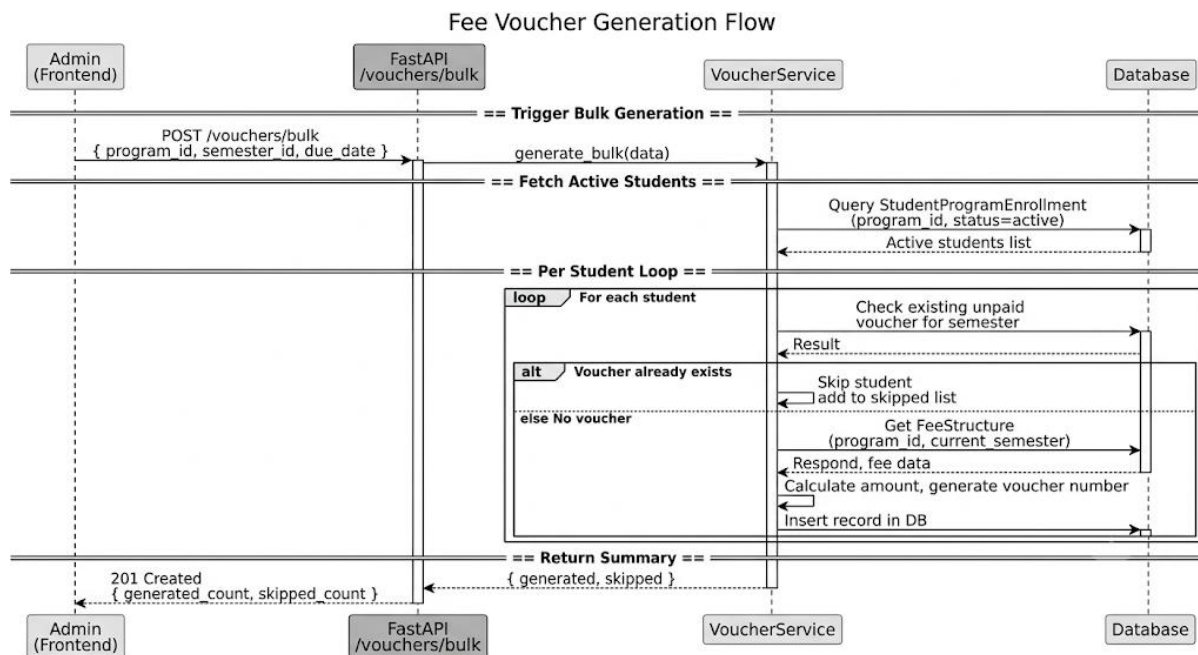


Figure 4.9: Sequence Diagram - Fee Voucher Generation

4.5.6 Activity Diagram - Student Enrollment Process

The student enrollment activity begins with the Admin selecting a student and a course offering. The system checks that the student exists and is active, that the offering is active, that the student is not already enrolled, and that the offering has not reached maximum capacity. If all checks pass, an Enrollment record is created with status "enrolled" and is_approved set to True (admin enrollments are auto-approved). The enrolled_students counter on the offering is incremented and the process completes. If any check fails, an appropriate error message is returned.

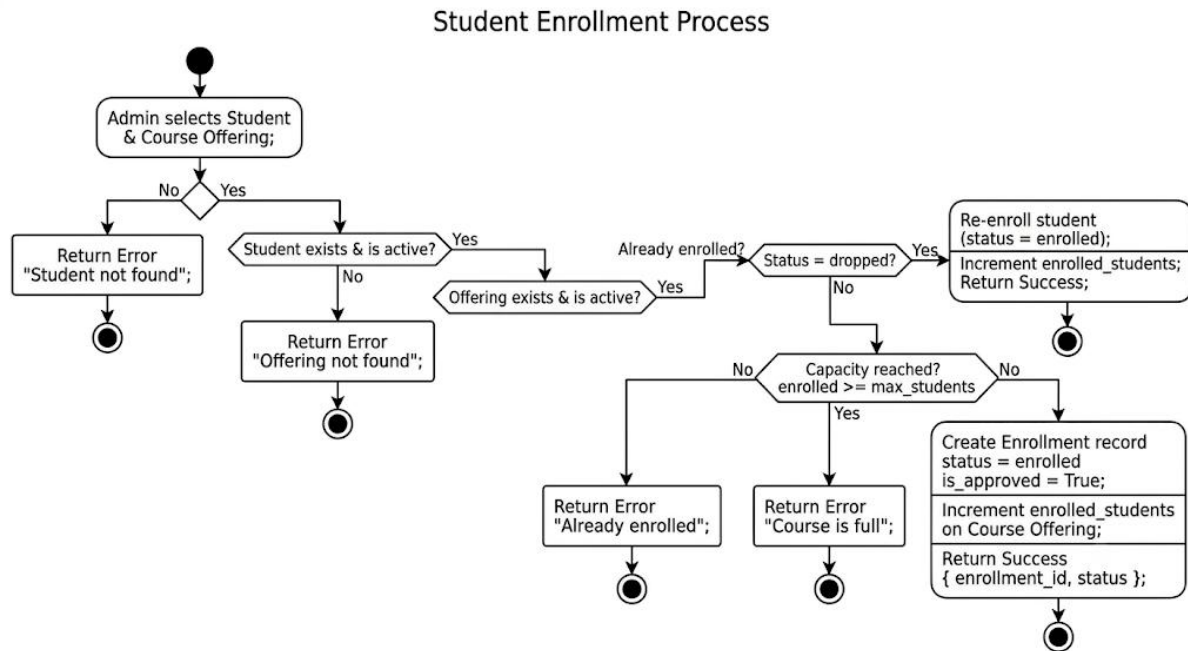


Figure 4.10: Activity Diagram - Student Enrollment Process

4.5.7 Activity Diagram - Two-Level Attendance Marking

The two-level attendance process operates on two independent paths. In Level 1, the teacher creates a lecture session, selects all enrolled students, marks each as present, absent, late, or excused, and submits the bulk record. The system saves individual attendance records and triggers attendance summary recalculation for each student. In Level 2, the gate camera continuously captures frames and sends them to the face recognition endpoint. On a successful match, a CampusAttendance record is created with entry direction, confidence score, and timestamp. A duplicate filter prevents multiple records within five minutes for the same student at the same gate.

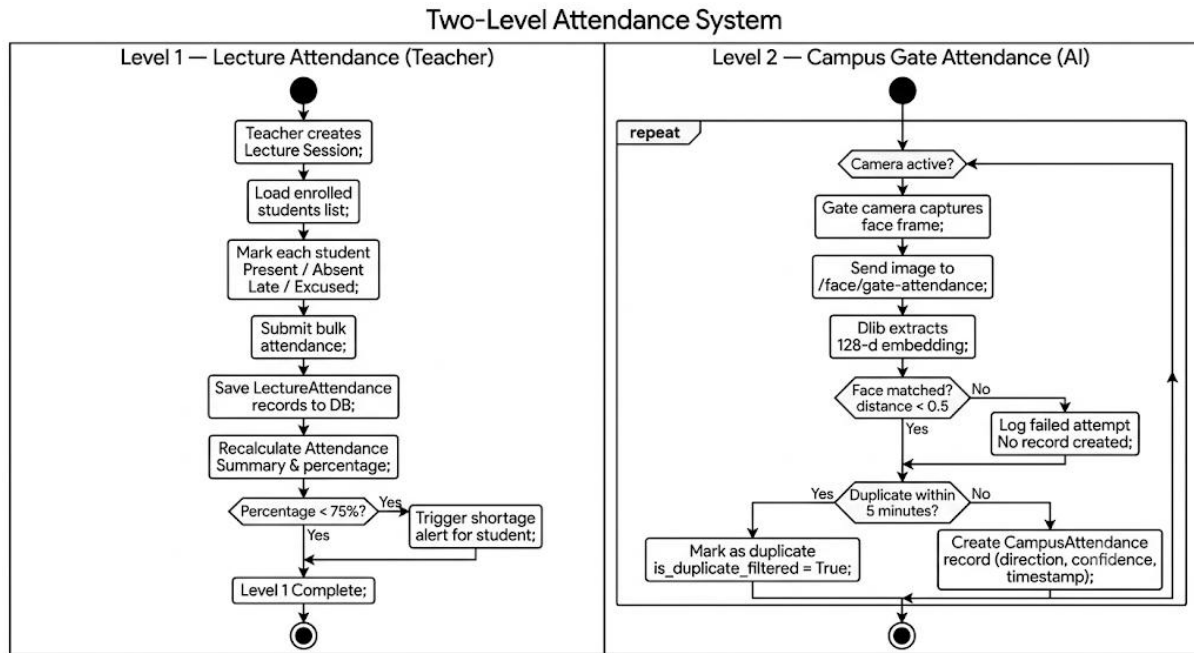


Figure 4.11: Activity Diagram - Two-Level Attendance System

4.6 AI Subsystem Design

4.6.1 Face Recognition Engine Design (Dlib 128-d Embeddings)

The face recognition pipeline consists of four sequential stages. In the detection stage, the Dlib HOG-based frontal face detector scans the input image and returns bounding boxes for any detected faces. In the landmark stage, the shape predictor model identifies 68 facial landmark points on the detected face region. In the encoding stage, the ResNet-based face recognition model computes a 128-dimensional floating-point embedding from the aligned face. In the comparison stage, the Euclidean distance between the query embedding and each stored embedding is computed, and the minimum-distance match is returned.

Face embeddings are stored as binary blobs (float64, $128 \times 8 = 1024$ bytes per embedding) directly in the users table, eliminating the need for a separate embedding store. The recognition threshold is set at 0.5 - distances below this value are considered a match.

The engine operates in two modes. In enrollment mode, a single clear image is processed and the resulting embedding is stored against the user's record. In recognition mode, the query embedding is compared against all enrolled embeddings in a sequential scan. For the expected user scale of a university department (hundreds to low thousands), this sequential approach is sufficiently fast without requiring an approximate nearest-neighbour index.

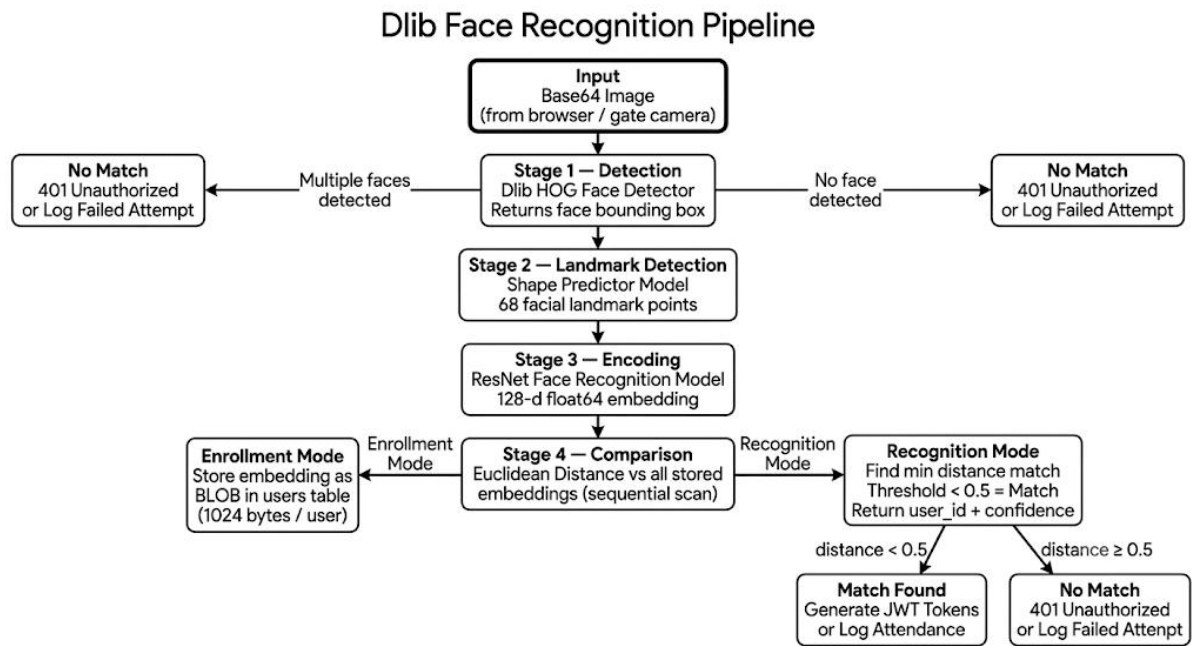


Figure 4.12: Dlib Face Recognition Pipeline

4.6.2 AI Chatbot Architecture (Gemini + FAISS RAG)

The chatbot architecture follows the Retrieval-Augmented Generation pattern in two phases.

In the offline indexing phase, the admin populates the chatbot_faqs table with question-answer pairs. At startup, the EmbeddingService encodes all FAQ texts using the paraphrase-multilingual-MiniLM-L12-v2 sentence-transformer model and stores the resulting vectors in a FAISS IndexFlatIP (inner product) index. This index is retained in memory for the server's lifetime.

In the online inference phase, when a student submits a query, the following steps execute sequentially. The student's live academic data is fetched from the database and formatted as a structured context string. The query is encoded using the same sentence-transformer model. The FAISS index performs a nearest-neighbour search and returns the top-3 most semantically similar FAQ entries above a similarity threshold of 0.35. The student data context, retrieved FAQ context, and original query are combined into a structured prompt. The prompt is sent to the Gemini API (gemini-2.5-flash model). The generated response is returned to the student and saved to the conversation history.

The system also includes a multi-key rotation manager that distributes requests across multiple Gemini API keys and handles quota exhaustion gracefully by marking exceeded keys and switching to available ones.

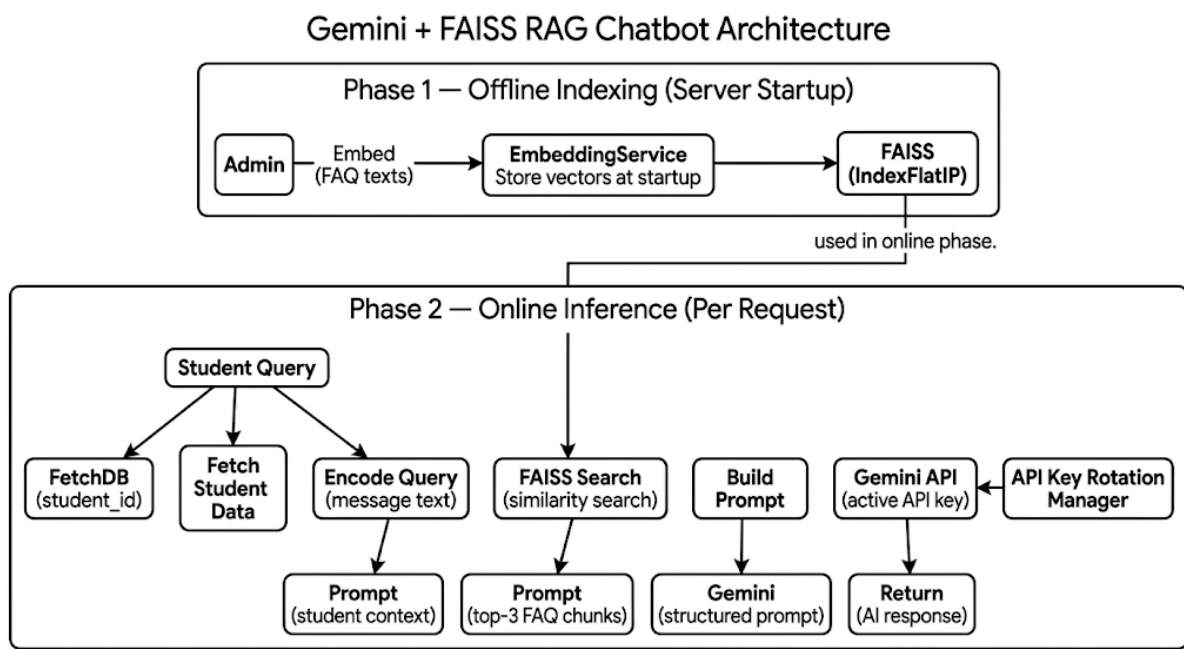


Figure 4.13: Gemini + FAISS RAG Chatbot Architecture

4.6.3 Analytics Engine Design

The analytics engine computes a unified academic performance score for each student per semester using a five-component weighted formula.

Table 4.3 - Analytics Score Components & Weights

Component	Weight	Calculation Method	Data Source
Lecture Attendance	25%	(Present + Late sessions) / Total marked sessions × 100	lecture_attendance table
Campus Presence	10%	Distinct days present at gate / Total campus days × 100	campus_attendance table
Assignment Consistency	20%	Submitted assignments / Total assignments × 100	assignment_submissions table
Quiz Accuracy	20%	Average percentage across all completed quiz attempts	quiz_attempts table
GPA Factor	25%	Average grade points / 4.0 × 100	enrollments table
TOTAL	100%	Weighted sum of all five components	Computed

The final academic score is calculated as:

Academic Score = (0.25 × Attendance) + (0.10 × Campus Presence) + (0.20 × Assignment Consistency) + (0.20 × Quiz Accuracy) + (0.25 × GPA Factor)

After score calculation, the engine also computes a Consistency Index (100 minus the standard deviation of the five component scores) and a Trend Direction (improving, stable, or declining) by comparing the current score against the previous semester's score.

The risk prediction module flags a student as at-risk if three or more of the following conditions are true: lecture attendance below 75%, assignment consistency below 50%, quiz accuracy below 50%, GPA factor below 50%, or campus presence below 50%.

4.6.4 AI Quiz Generation (Gemini Prompt Engineering)

The quiz generation module constructs a structured prompt that instructs Gemini to return a JSON array of MCQ questions. The prompt specifies the topic, difficulty level, number of questions, course name for context, and the exact JSON schema required - including question text, four options, correct answer (matching one of the options exactly), and explanation.

The response is parsed by stripping any markdown code fences, locating the JSON array boundaries, and parsing the content. Each parsed question is validated to ensure it contains the required fields before being stored. If Gemini fails or returns malformed output, a set of placeholder questions is generated as a fallback to prevent empty responses.

4.6.5 Plagiarism Detection Engine

The plagiarism engine uses a two-stage approach to detect both exact copying and paraphrased similarity.

In Stage 1, Python's `difflib.SequenceMatcher` computes a character-level similarity ratio between the normalized texts of two submissions. Normalization includes lowercasing, removal of code comments and block comments, and whitespace collapsing.

In Stage 2, the `paraphrase-multilingual-MiniLM-L12-v2` model encodes the first 512 words of each submission into a dense vector, and cosine similarity is computed between the two vectors using sklearn's `cosine_similarity` function.

The final combined score is calculated as:

Combined Score = (0.40 × difflib score) + (0.60 × semantic score)

Risk levels are assigned as: High Risk (combined score $\geq 70\%$), Medium Risk (40–69%), and Low Risk (below 40%). Results are stored in the `plagiarism_data` JSON column of each submission record.

4.7 Security Design

4.7.1 JWT Authentication Architecture

The system uses python-jose for JWT encoding and decoding with the HS256 algorithm. The secret key is stored in the .env configuration file and loaded through Pydantic Settings. Access tokens carry a payload of user ID, role, expiry, and token type. The short 60-minute TTL for access tokens minimizes the window of exposure if a token is intercepted.

Refresh tokens are stored client-side (localStorage on the frontend) and are transmitted only to the dedicated /auth/refresh-token endpoint. The frontend Axios interceptor handles silent renewal automatically, providing seamless session continuity without requiring the user to re-login.

4.7.2 Role-Based Access Control (RBAC)

Table 4.4 - RBAC Permissions Matrix

Feature / Endpoint	Admin	Teacher	Student
User Management	Full CRUD	View Own	View Own
Department / Program / Course	Full CRUD	Read Only	Read Only
Course Offerings	Full CRUD	Read Assigned	Read Enrolled
Enrollment Management	Full CRUD	Read	Read Own
Lecture Attendance	View All	Mark + View	View Own
Campus Gate Attendance	View All	No	View Own
Assignments	View All	Full CRUD	Submit
Quiz Generation (AI)	No	Generate + Publish	Take
Fee Vouchers	Generate + Manage	No	View Own
AI Chatbot	Configure KB	No	Use
Analytics Dashboard	View All	View Course	View Own
Announcements	Full CRUD	Create	Read
Face Enrollment	Manage All	No	Enroll Own

4.7.3 Password Security

All passwords are hashed using bcrypt through the passlib library. The bcrypt algorithm is configured with a minimum of 12 salt rounds, producing a computationally expensive hash that resists brute-force attacks. Plain-text passwords are never stored, logged, or transmitted after the initial registration request. Password validation enforces a minimum length of 8 characters. The change password endpoint requires the current password to be verified before accepting a new one.

4.8 UI/UX Design

4.8.1 Design System - Neumorphic Theme

The frontend uses a custom Neumorphic design system implemented through CSS variables and Tailwind CSS utility classes. The design is based on the concept of soft extruded plastic - elements appear to be raised from or pressed into the background surface through carefully calibrated dual-shadow techniques.

The light mode palette uses a base background of #dde3ee with surface color #e8edf6. Raised elements cast a dark shadow (rgba(163, 177, 198, 0.75)) downward-right and a light shadow (rgba(255, 255, 255, 0.95)) upward-left. Pressed elements invert this shadow pattern using CSS inset shadows.

The primary accent color is #5b8af0 (blue) for students, #22a06b (green) for teachers, and #9b59b6 (purple) for admin, providing immediate role-context visual distinction across all portals. A full dark mode is supported through the data-theme="dark" attribute on the document root, with all shadow values adjusted for the dark background.

Typography uses Outfit (headings) and DM Sans (body text) from Google Fonts.

4.8.2 Portal Navigation Flow

All three portals share a common layout shell consisting of a top navigation bar (Navbar) and a vertical icon dock (Sidebar) on the left. The sidebar uses a macOS Dock-style magnification effect - icons scale up on mouse proximity using a spring physics animation system.

The Admin portal provides 17 navigation items covering all management functions. The Teacher portal provides 10 items covering course, assessment, attendance, and communication functions. The Student portal provides 13 items including AI Assistant and Practice Quiz pages unique to the student role. A floating AI Chatbot widget appears on all student portal pages.

4.8.3 Key Screen Wireframes / Screenshots

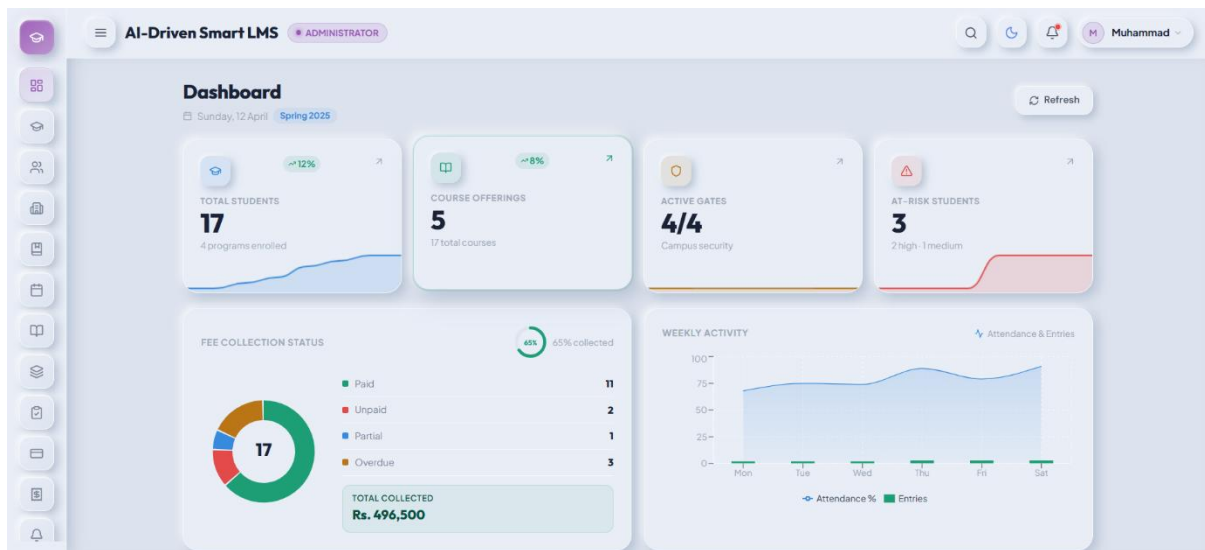


Figure 4.14: Admin Dashboard Design

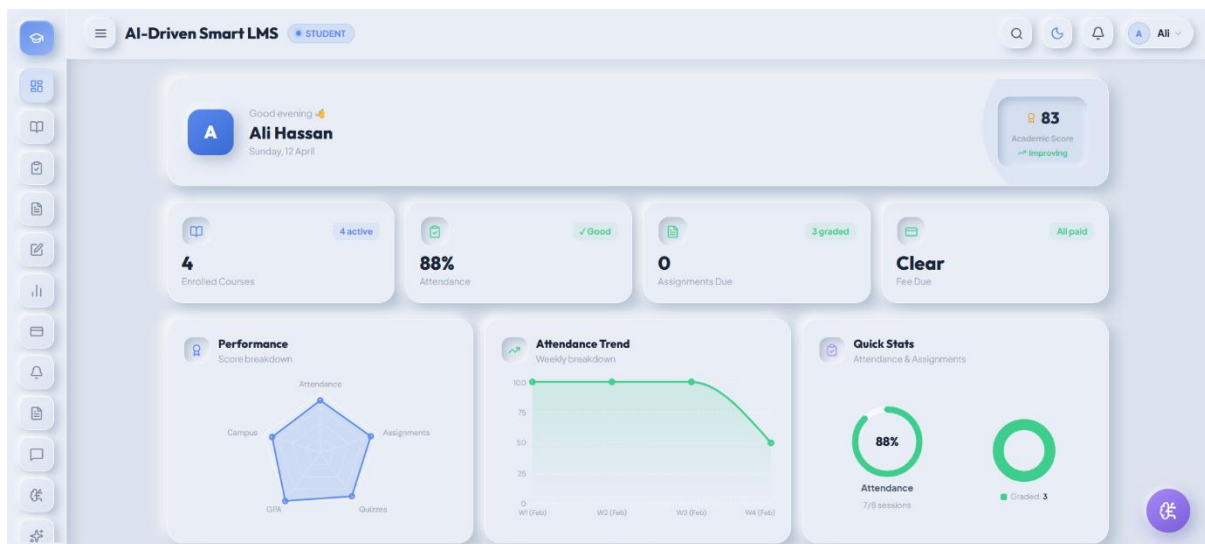


Figure 4.15: Student Dashboard Design

CHAPTER 5: IMPLEMENTATION & DEVELOPMENT

5.1 Development Environment Setup

5.1.1 Backend Setup (Python, FastAPI, MySQL)

The backend development environment is configured on Windows 11 with Python 3.12. A Python virtual environment is created using the venv module to isolate project dependencies. All required packages are installed via pip using the requirements.txt file, which includes FastAPI, SQLAlchemy, Uvicorn, python-jose, passlib, PyMySQL, and all AI-related libraries.

The MySQL 8.x database is created with the name `AI_Driven_Smart_LMS`. Environment variables are managed through a `.env` file loaded by Pydantic Settings, containing the database connection string, JWT secret key, token expiry values, and Gemini API key. The backend server is started using Uvicorn with the `--reload` flag during development. The FastAPI automatic documentation is accessible at <http://localhost:8000/docs>.

5.1.2 Frontend Setup (React, Vite, Tailwind CSS)

The frontend is initialized using Vite with the React template. Node.js 18+ is required. After project creation, Tailwind CSS is installed and configured through `postcss.config.js` and `tailwind.config.js`. The custom Neumorphic theme CSS variables are defined in `neu-theme.css` and imported at the top of `index.css` before Tailwind directives.

The `VITE_API_BASE_URL` environment variable in `.env` points to the FastAPI backend. The Axios client reads this variable to construct all API request URLs. The development server runs on <http://localhost:5173>.

5.1.3 AI Models Setup (Dlib, HuggingFace)

The Dlib face recognition engine requires two pre-trained model files to be present in the `backend/app/ai/models/` directory. The `shape_predictor_68_face_landmarks.dat` file (95 MB) provides the facial landmark predictor, and the `dlib_face_recognition_resnet_model_v1.dat` file (21 MB) provides the face encoding model. Both are downloaded from the official Dlib model repository.

The `sentence-transformers` library automatically downloads the `paraphrase-multilingual-MiniLM-L12-v2` model from HuggingFace Hub on first use and caches it locally. The FAISS index is initialized in memory when the server starts and is populated from the `chatbot_faqs` database table.

Table 5.1 - Development Tools & Versions

Tool	Version	Purpose
Python	3.12+	Backend programming language
FastAPI	0.104+	REST API framework
SQLAlchemy	2.x	ORM for MySQL
Dlib	19.24	Face recognition library
Google Gemini SDK	Latest	AI chatbot and quiz generation
FAISS	1.7+	Vector similarity search
sentence-transformers	Latest	Text embeddings for RAG
React.js	18.x	Frontend framework
Vite	5.x	Frontend build tool
Tailwind CSS	3.x	UI styling
MySQL	8.x	Relational database
VS Code	Latest	Code editor
Postman	Latest	API testing
draw.io	Latest	Diagram creation

5.2 Module Implementation

5.2.1 Authentication Module (JWT + Face Login)

The authentication module is implemented in `backend/app/api/v1/endpoints/auth.py` and backed by `backend/app/services/auth_service.py`. The User SQLAlchemy model stores hashed passwords, role, active status, and the Dlib face embedding as a binary blob.

Password hashing uses `passlib's CryptContext` with the `bcrypt` scheme. JWT tokens are generated using `python-jose` with the `HS256` algorithm. The `create_access_token()` and `create_refresh_token()` functions in `security.py` produce tokens with the user's ID and role in the payload.

The `get_current_user()` dependency in `dependencies.py` is injected into every protected endpoint. It extracts the Bearer token from the Authorization header, decodes and validates it, and queries the database for the associated user. If the token is invalid or the user is inactive, a 401 response is returned immediately.

The face login endpoint receives a base64-encoded image from the frontend MediaPipe widget, passes it to `FaceRecognitionEngine.login_by_face()`, and on successful recognition generates JWT tokens identically to the password login flow.

5.2.2 Academic Structure Module

The academic structure module provides CRUD operations for departments, programs, semesters, and courses through separate router files. Each router is backed by a corresponding service class in `academic_service.py`. All write operations are restricted to Admin using the `require_admin()` dependency.

Department creation validates code uniqueness before insertion. Program creation validates that the referenced department exists. Semester activation uses a two-step approach - first deactivating all existing semesters, then activating the selected one - ensuring only one active semester exists at any time. Course creation validates code uniqueness and links to both department and program.

5.2.3 Two-Level Attendance System

Level 1 (Lecture Attendance) is managed through the `LectureSession` and `LectureAttendance` models. Teachers create sessions with a date, start time, end time, and optional topic. The bulk attendance marking endpoint accepts a list of student-status records and saves them in a single transaction. After saving, the `_update_summary()` method recalculates the attendance percentage and alert status for each affected student in the `attendance_summary` table.

Level 2 (Campus Gate Attendance) is driven by the `/face/gate-attendance` endpoint, which is intentionally unauthenticated to allow kiosk devices to call it directly. The endpoint passes the captured image to `FaceRecognitionEngine.recognize_face()`, and on a successful match calls `CampusAttendanceService.log_entry()`. The duplicate filter queries for existing records within the past five minutes for the same student and gate, setting the `is_duplicate_filtered` flag if a duplicate is detected rather than rejecting the record entirely.

5.2.4 Assessment Module

The assessment module covers three assessment types: assignments, quizzes, and examinations.

Assignment creation is restricted to teachers and links to a course offering. Student submission uses FastAPI's UploadFile for multipart file handling, storing the file in the uploads/assignments/ directory with a structured filename (student_{id}assign{id}_{original_name}). The submission status is automatically set to "late" if the current time exceeds the assignment due date.

Quiz creation accepts a list of question objects in the request body, which are stored as individual QuizQuestion records. The quiz attempt system uses a SELECT-before-INSERT pattern with an exception handler to safely manage concurrent duplicate attempts - a common issue with double-clicks on mobile devices.

Exam creation supports three types: midterm, final, and special. Bulk result entry validates that obtained marks do not exceed total marks and auto-calculates a grade letter based on percentage thresholds (A+ for 90%+, A for 85%+, down to F below 50%).

5.2.5 Fee Management Module

Fee structures are defined per program and semester number, storing tuition, admission, library, sports, and other fees. The total fee is a MySQL GENERATED column that sums the component fees automatically.

Bulk voucher generation iterates over all active StudentProgramEnrollment records for the selected program, checks for existing unpaid vouchers, looks up the applicable fee structure, generates a unique voucher number in the format VCH-{year}-{5-digit-random}, and inserts the FeeVoucher record. The process skips students who already have an active voucher and returns a count of generated and skipped records.

Fine calculation is triggered manually by the admin. The system calculates the number of days past the due date, multiplies by a configurable daily fine rate (default Rs. 50), and updates the voucher's fine_amount and status fields.

5.2.6 Communication Module

Announcements support five targeting types: all users, a specific department, a specific program, a specific course offering, or a specific section. The get_for_student() method constructs a dynamic SQL filter using SQLAlchemy's or_() and and_() operators to retrieve all announcements relevant to a student's enrolled courses, program, and department simultaneously.

The WebSocket chat system uses a ConnectionManager class that maintains a dictionary of active connections grouped by chat group ID. On connection, the server validates the JWT token passed as a query parameter. Messages are broadcast to all group members using async iteration over the connection list, with failed sends triggering automatic disconnection cleanup.

5.2.7 Face Recognition Module (Dlib Implementation)

The FaceRecognitionEngine class uses lazy loading - the three Dlib model objects (HOG detector, shape predictor, and ResNet encoder) are initialized as class-level variables and loaded only on the first call that requires them. This avoids the ~3-second model loading overhead on server startup.

The `_extract_embedding()` method converts the input numpy array to uint8, runs the HOG detector, validates that exactly one face is detected, computes the 68-point landmark shape, and calls `compute_face_descriptor()` to produce the 128-d float64 vector.

Stored embeddings are serialized using numpy's `.tobytes()` method and deserialized using `np.frombuffer()` with `dtype=float64`. Euclidean distance is computed using `np.linalg.norm()`. All enrolled users are loaded in a single database query, and comparison is performed in a Python loop.

5.2.8 AI Chatbot Module (Gemini + FAISS RAG)

The ChatbotEngine.`process_message()` method coordinates four sequential operations. First, it retrieves or creates a ChatbotConversation session record. Second, it calls `get_student_data()` which executes multiple targeted database queries to assemble the student's academic context - CGPA, attendance per course, pending fees, enrolled courses, and current grades. Third, it calls `RAGService.build_context()` which encodes the query and retrieves top-k FAQ matches from the FAISS index. Fourth, it calls `GeminiService.generate_response()` with the assembled prompt. Both the student message and bot response are saved as ChatbotMessage records.

The GeminiService implements multi-key rotation. API keys are stored in a `gemini_keys.json` file. On each request, the service selects the key with the lowest request count for the day that is not marked as quota-exceeded. On a 429 (quota exceeded) error, the key is marked with a cooldown timestamp parsed from the API error response.

5.2.9 AI Analytics Engine

The `AnalyticsEngine.calculate_for_student()` method queries five data sources in sequence and computes each component score independently. Campus presence score requires counting distinct calendar dates in the `campus_attendance` table within the semester date range - two separate `COUNT(DISTINCT DATE(...))` queries are used, one for all students (total campus days) and one filtered by student ID (present days).

After computing the five component scores, the engine calculates the weighted academic score, the consistency index (standard deviation-based), the trend direction (comparison against previous semester score), the engagement level (High/Medium/Low based on score thresholds), and the risk prediction (count of components below threshold).

The `calculate_ranks()` method performs a bulk rank assignment - it queries all `StudentPerformanceScore` records for the semester ordered by `academic_score` descending and assigns sequential `class_rank` values in a single pass.

5.2.10 AI Quiz Generation Module

The `AIQuizService.generate()` method constructs a prompt that explicitly specifies the JSON output format, the number of questions, the topic, the course name, and the difficulty level. The prompt includes a strict instruction that the `correct_answer` must match one of the four options exactly, which is critical for the auto-grading logic in the submit endpoint.

Response parsing strips markdown code fences (`json` and `)`, locates the opening and closing square brackets, and calls `json.loads()` on the extracted substring. Each parsed question is validated for the presence of `question`, `options`, and `correct_answer` keys before being appended to the output list.

5.2.11 Plagiarism Detection Module

The `PlagiarismService.run_for_assignment()` method loads all submissions for a given assignment, extracts text from each file using the appropriate method based on file extension (plain text, PDF via `PyMuPDF`, DOCX via `python-docx`, or ZIP archive contents), and performs pairwise comparison across all submission pairs.

Text normalization removes single-line comments (`#` `//` `--`), block comments, and collapses whitespace before `diff`lib comparison. The semantic comparison truncates text to the first 512 words to respect the sentence-transformer model's effective context window. Combined scores and risk levels are written back to each submission's `plagiarism_percentage` and `plagiarism_data` columns.

5.3 Key Code Snippets with Explanation

5.3.1 Face Embedding Extraction (Dlib)

The following code excerpt shows the core face embedding extraction method from FaceRecognitionEngine:

```
1 @classmethod
2 def _extract_embedding(cls, image_array: np.ndarray):
3     if not cls._load_models():
4         return None, "Dlib models not loaded"
5     if image_array.dtype != np.uint8:
6         image_array = image_array.astype(np.uint8)
7     detections = cls._detector(image_array, 1)
8     if len(detections) == 0:
9         return None, "No face detected in image"
10    if len(detections) > 1:
11        return None, "Multiple faces detected"
12    face = detections[0]
13    shape = cls._predictor(image_array, face)
14    descriptor = cls._face_rec.compute_face_descriptor(image_array, shape)
15    embedding = np.array(descriptor)
16    return embedding, None
```

RazAse (sarfraz)

Figure 5.2: Dlib Face Embedding Extraction Function

The method first ensures models are loaded. It converts the image to uint8 as required by Dlib's C++ backend. The HOG detector is called with upsampling factor 1, which improves detection of smaller faces at the cost of slightly increased computation time. The predictor computes 68 landmark points, and `compute_face_descriptor()` uses these landmarks to align the face before encoding it into the 128-d vector.

5.3.2 Gemini AI Response Generation

The following shows the core prompt construction and API call in GeminiService:

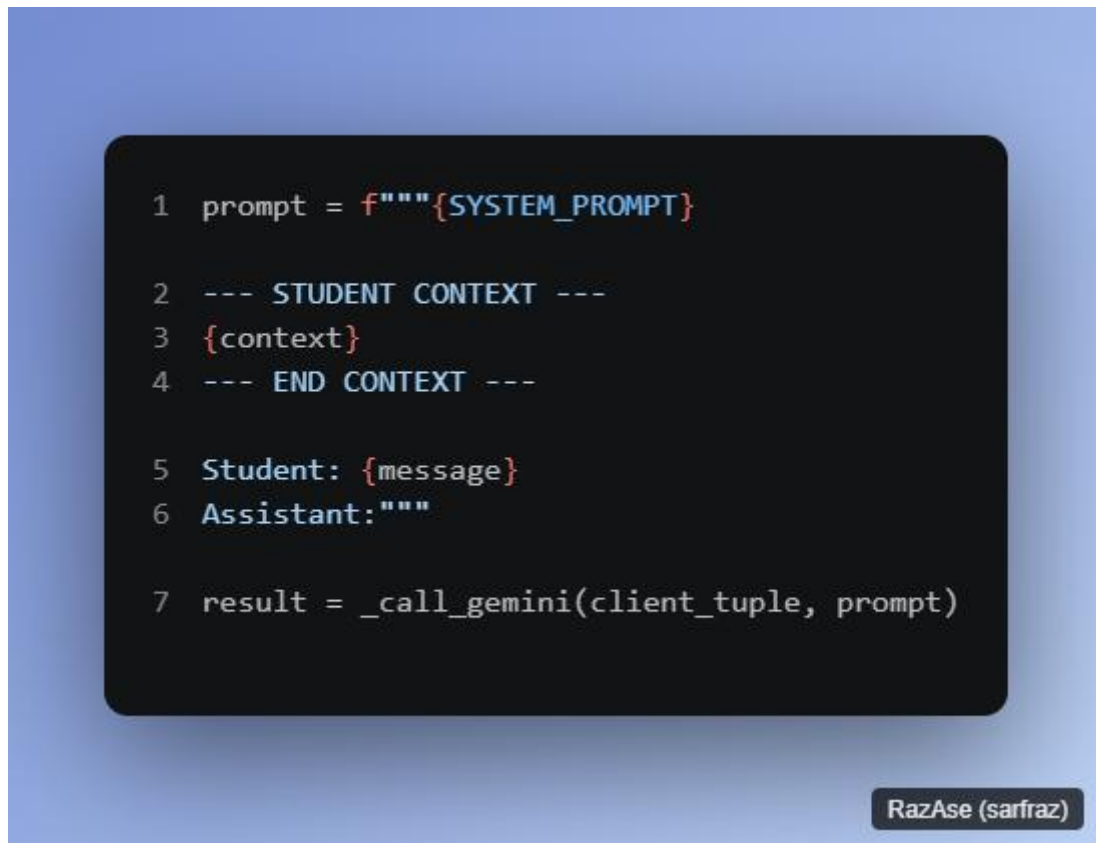


Figure 5.3: Gemini AI Chatbot Response Generation

The SYSTEM_PROMPT defines the assistant's identity, permitted topics, and response style. The student context section injects live academic data formatted by RAGService. This structure ensures the model has clear separation between system instructions, retrieved context, and the actual user message - a prompt engineering practice that produces more consistent and accurate responses.

5.3.3 Analytics Score Calculation

The following shows the weighted score formula from the analytics engine:

```
1 academic_score = (  
2     breakdown["lecture_attendance"] * 0.25 +  
3     breakdown["campus_presence"] * 0.10 +  
4     breakdown["assignment_consistency"] * 0.20 +  
5     breakdown["quiz_accuracy"] * 0.20 +  
6     breakdown["gpa_factor"] * 0.25  
7 )  
8 academic_score = round(min(academic_score, 100), 2)
```

RazAse (sarfraz)

Figure 5.4: Student Performance Score Calculation

Each component is already normalized to a 0–100 scale before this calculation. The `min()` function caps the result at 100 to handle edge cases where component scores slightly exceed 100 due to rounding. The score is rounded to two decimal places for consistent storage.

5.3.4 JWT Token Creation & Validation

The token creation function constructs the payload and signs it with the configured secret key:

```

1 def create_access_token(data: dict,
2     expires_delta: Optional[timedelta] = None) -> str:
3     to_encode = data.copy()
4     expire = datetime.now(timezone.utc) + (
5         expires_delta or timedelta(
6             minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
7     )
8     to_encode.update({"exp": expire, "type": "access"})
9     return jwt.encode(to_encode,
10         settings.SECRET_KEY,
11         algorithm=settings.ALGORITHM)
12

```

RazAse (sarfraz)

The type field distinguishes access tokens from refresh tokens, preventing refresh tokens from being used directly on protected endpoints. The `get_current_user()` dependency decodes the token, checks the type field is "access", retrieves the user by the sub claim, and verifies `is_active` is True before allowing the request to proceed.

5.3.5 Plagiarism Similarity Algorithm

The combined similarity score computation:


```

1  d1 = difflib.SequenceMatcher(None, na, nb).ratio()
2  sem = semantic_similarity(text_a, text_b)

3  if sem == 0.0 and not _ST_AVAILABLE:
4      combined = d1
5  else:
6      combined = 0.40 * d1 + 0.60 * sem

7  combined_pct = round(combined * 100, 2)

8  if combined_pct >= 70:
9      risk = "high"
10 elif combined_pct >= 40:
11     risk = "medium"
12 else:
13     risk = "low"
14

```

RazAse (sarfraz)

The 60% weight on semantic similarity reflects the greater importance of detecting paraphrased copying over exact character-level matches. When the sentence-transformers library is unavailable, the system gracefully falls back to difflib-only comparison, ensuring functionality even in minimal deployment environments.

5.4 Frontend Implementation

5.4.1 Neumorphic Design System

The design system is defined in frontend/src/neu-theme.css through CSS custom properties. The core shadow variables are:

--neu-raised: creates the outward-raised appearance using dual box shadows - a dark shadow at bottom-right and a light shadow at top-left.

--neu-inset: creates the pressed-in appearance by inverting the shadow directions using the CSS inset keyword.

--neu-focus: combines inset shadows with a subtle accent-colored ring for interactive elements.

Dark mode is implemented by redefining all CSS variables under the `[data-theme="dark"]` selector, switching to near-black backgrounds and adjusting shadow opacities. The ThemeContext React context manages the theme state, persists the preference to localStorage, and applies the data-theme attribute to the document root on every toggle.

5.4.2 Admin Portal

The Admin portal provides 17 pages covering all system management functions. The dashboard displays summary statistics - total students, teachers, active courses, and pending fee vouchers - fetched from multiple API endpoints and rendered as Neumorphic stat cards.

Key pages include the Students page (full CRUD with search, pagination, and program enrollment), the Enrollments page (course offering management with student list and grade entry), the Fee Vouchers page (bulk generation, payment recording, and overdue calculation), the Gates page (gate and camera management with schedule configuration), and the Analytics page (leaderboard, at-risk student list, and bulk score recalculation trigger).

All data tables use a custom Neumorphic table component with right-click context menus for edit, delete, and action operations. The useContextMenu hook renders context menu items via React Portal to avoid z-index clipping issues.

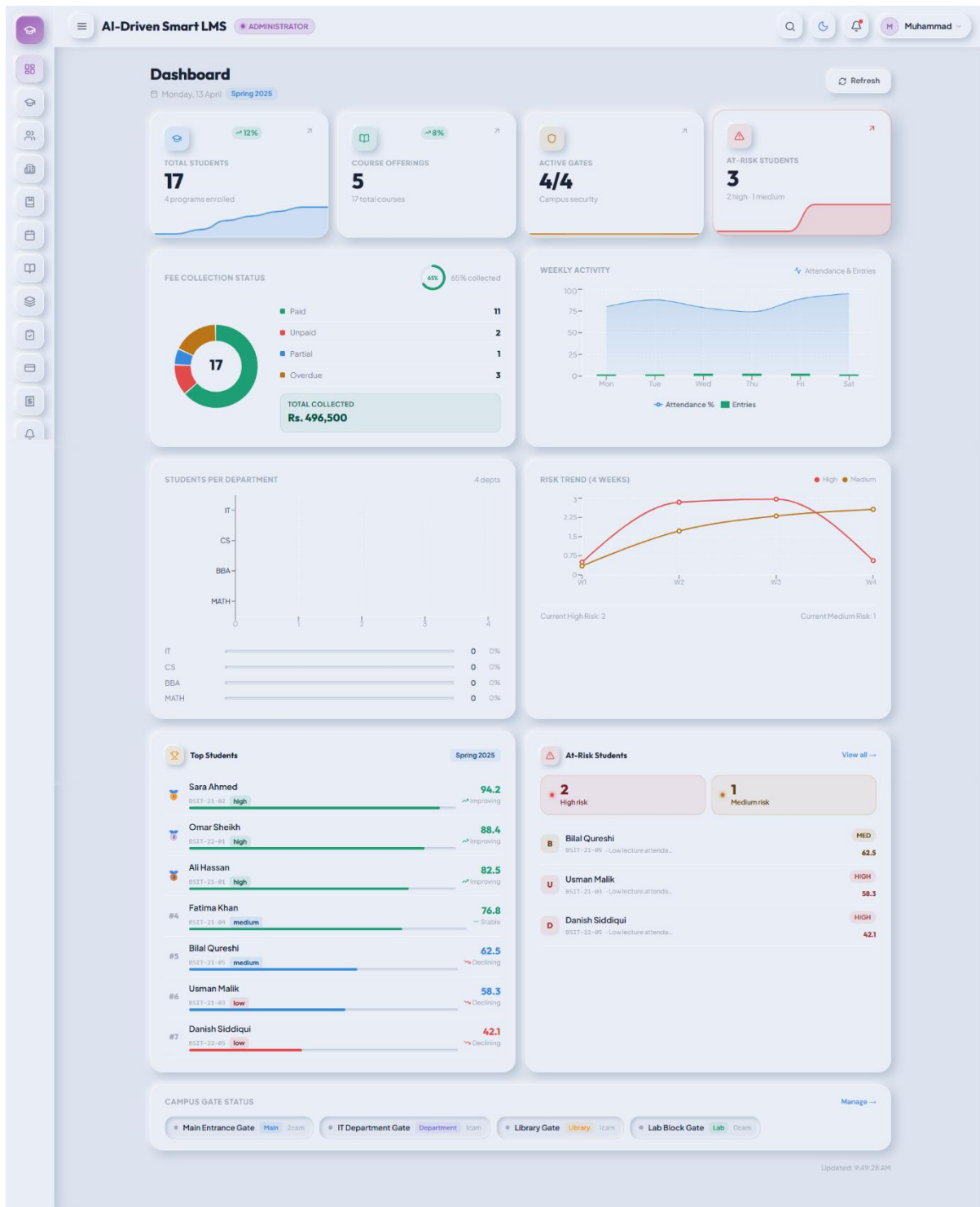


Figure 5.5: Admin Portal - Main Dashboard

5.4.3 Teacher Portal

The Teacher portal provides 10 pages. The My Courses page displays all assigned offerings with enrollment counts and quick-action buttons. The Attendance page allows teachers to select a course, create a session, and mark attendance for all enrolled students with a single-click all-present toggle and individual status adjustments.

The Assignments page supports full assignment lifecycle management including creation, submission viewing, bulk grading with feedback, and plagiarism report viewing through the PlagiarismReportModal component. The Quizzes page supports both manual quiz creation and AI-powered generation - the teacher enters a topic and difficulty, and the generated questions are displayed for review before publishing.

The Analytics page shows class-level performance data including the leaderboard, at-risk students with their risk factors, and a course attendance report with shortage counts.

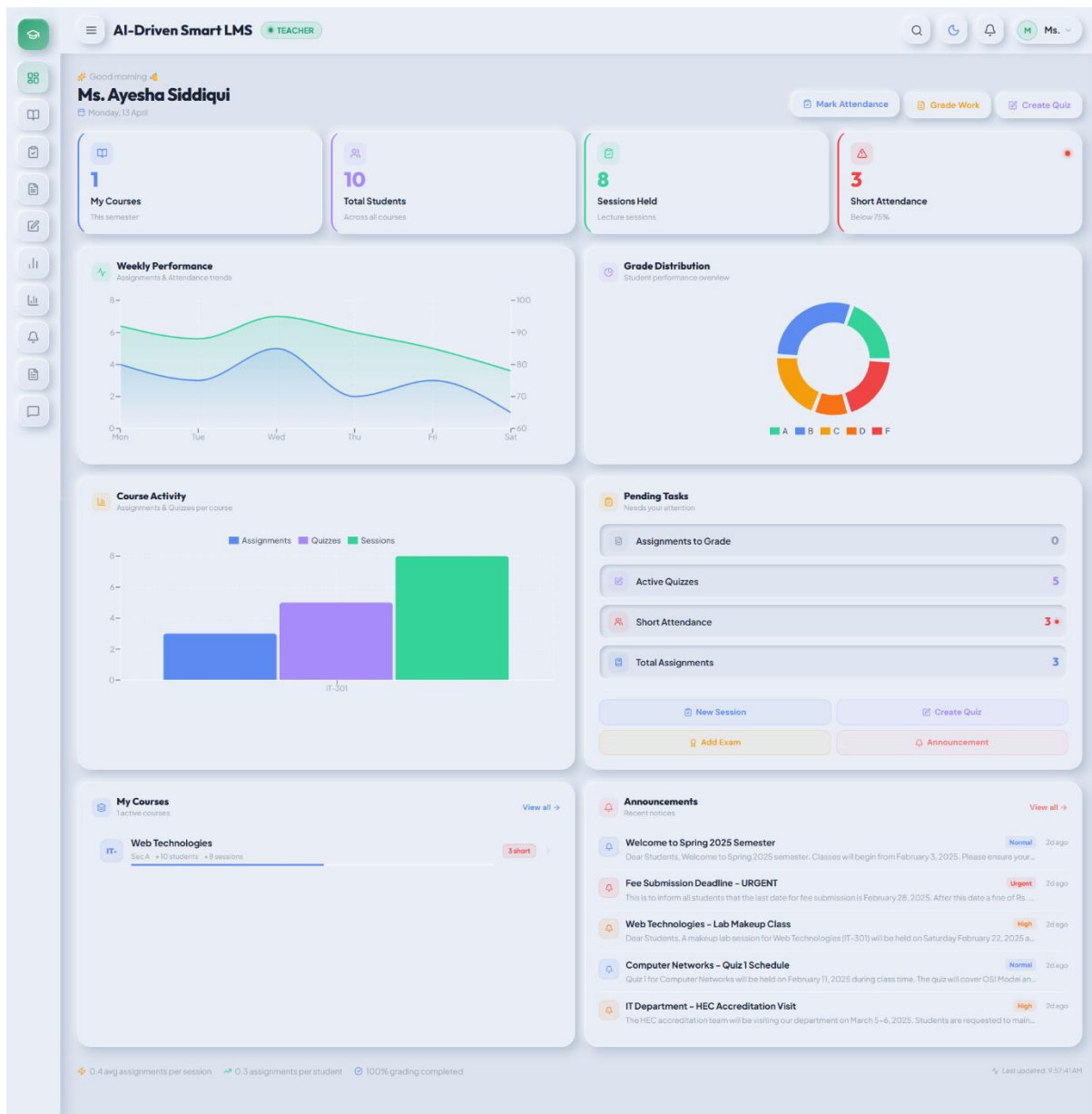


Figure 5.6: Teacher Portal - Main Dashboard

5.4.4 Student Portal

The Student portal provides 13 pages. The Dashboard displays a performance summary including academic score, attendance percentage, pending fees, and recent announcements.

The My Courses page shows enrolled courses with CLO counts and quick links to course-specific features.

The AI Assistant page provides a full-page chat interface with markdown rendering support, inline code formatting, and message copy functionality. The Practice Quiz page allows students to select a course, topic, and difficulty to generate an AI quiz, take it with immediate feedback, and review weak areas identified by the system.

The floating ChatbotWidget appears on all student portal pages as a minimizable popup. It includes quick-suggestion buttons, a typewriter-effect loading animation with rotating Urdu/English loading messages, and a markdown renderer for bot responses.

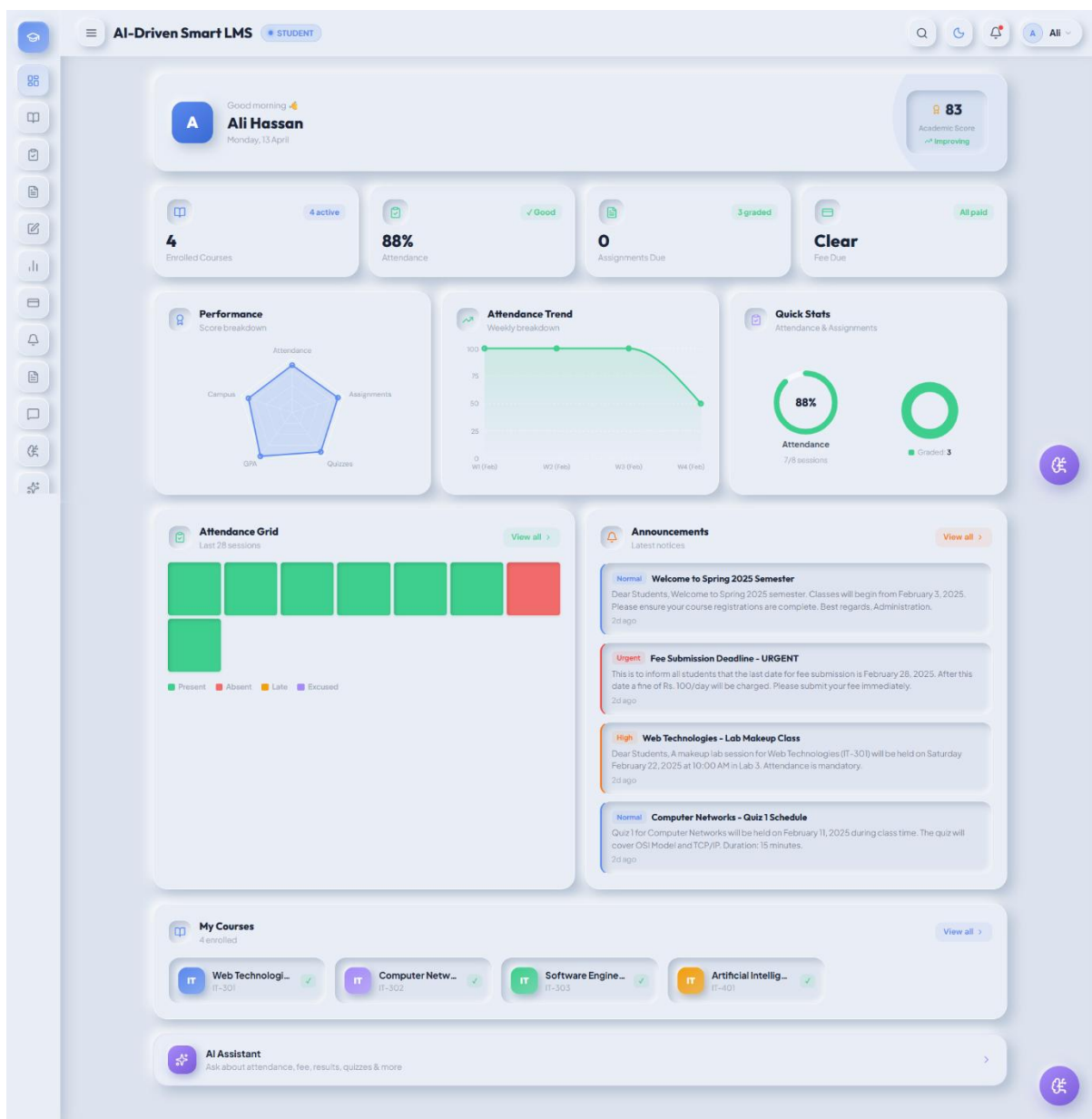


Figure 5.7: Student Portal - Dashboard

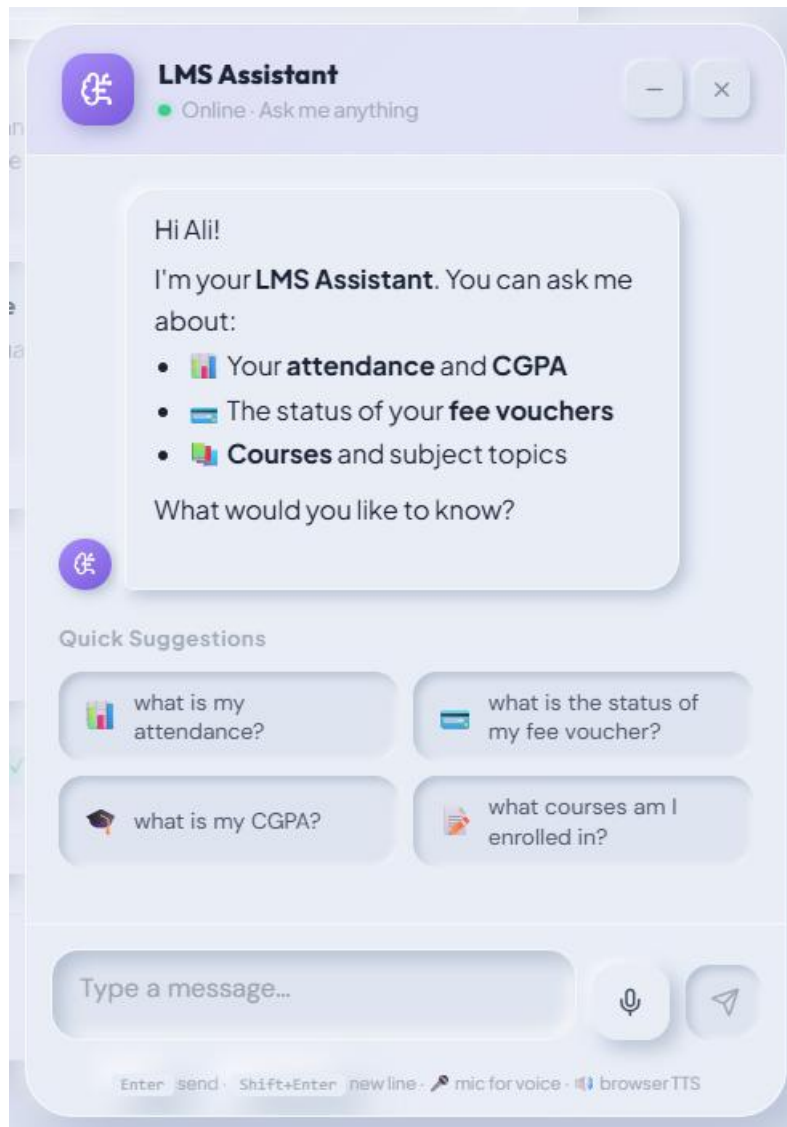


Figure 5.8: AI Chatbot Widget in Student Portal

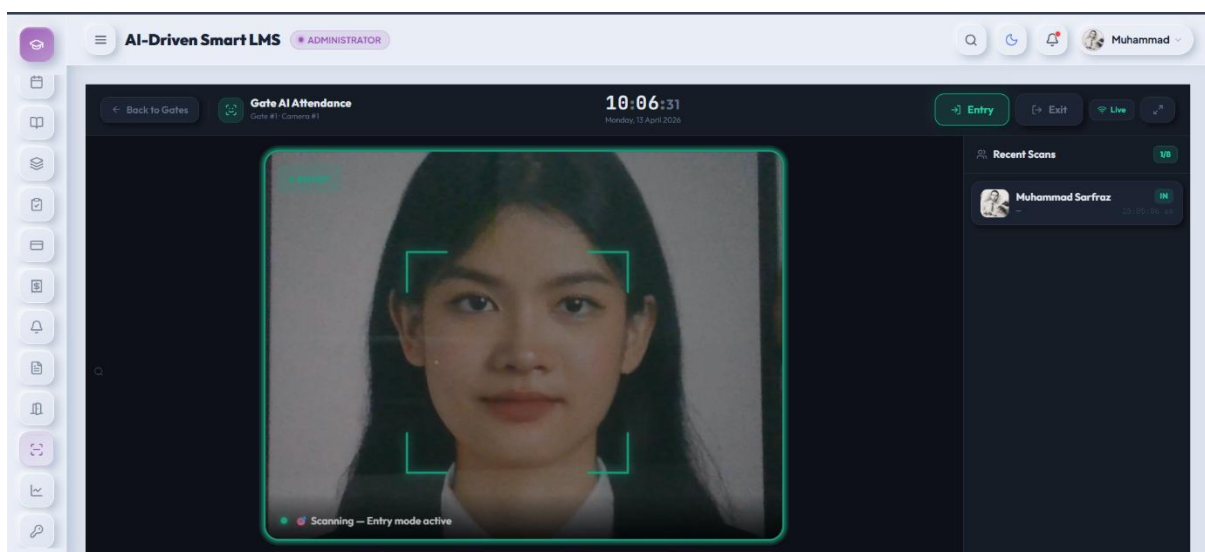


Figure 5.9: Campus Gate Face Attendance Scanner Interface

5.5 API Integration (Frontend-Backend)

The frontend communicates with the FastAPI backend through a centralized Axios instance defined in `frontend/src/api/axios.js`. A request interceptor automatically attaches the JWT access token from `localStorage` to the Authorization header of every outgoing request.

A response interceptor handles 401 Unauthorized responses by attempting a silent token refresh using the stored refresh token. During the refresh, all concurrent failing requests are queued using a Promise-based queue. On successful refresh, all queued requests are replayed with the new token. If the refresh itself fails (expired or invalid refresh token), the user is logged out and redirected to the login page.

Role-specific API functions are organized into three separate files - `admin.api.js`, `teacher.api.js`, and `student.api.js` - each exporting a named object with methods corresponding to every API endpoint used by that role. This organization keeps API calls centralized and prevents duplication across component files.

5.6 Database Integration (SQLAlchemy ORM)

All database models are defined as SQLAlchemy `DeclarativeBase` subclasses in the `backend/app/models/` directory, organized by domain. The `models/init.py` file imports all model classes in dependency order, ensuring SQLAlchemy registers all relationships correctly before any query is executed.

Database sessions are managed through FastAPI's dependency injection system. The `get_db()` generator function in `database.py` yields a `SessionLocal` instance and guarantees closure in the `finally` block, preventing connection leaks. All service methods receive the db session as their first parameter.

The database schema is created directly from the SQL script (`Database/Schema.sql`) for initial setup. The schema includes 12 performance indexes on high-frequency query columns such as `enrollments(student_id, status)`, `attendance_summary(student_id, offering_id, percentage)`, and `fee_vouchers(due_date, status)`.

5.7 Challenges Faced & Solutions

Table 5.2 - Challenges Faced & Solutions

Challenge	Description	Solution Adopted
Dlib Installation on Windows	CMake and C++ build tools required for Dlib compilation	Used pre-compiled Dlib wheel files; documented setup process

Challenge	Description	Solution Adopted
CORS Issues (React + FastAPI)	Browser blocked cross-origin API requests	Configured FastAPI CORSMiddleware with allowed origins
FAISS Index Persistence	FAISS in-memory index lost on server restart	Implemented FAISS index save/load to disk using <code>faiss.write_index()</code>
Gemini API Rate Limiting	API calls throttled under heavy quiz generation load	Implemented request queuing and retry logic with exponential backoff
WebSocket Authentication	JWT auth not applicable to WS connection handshake	Passed token as query parameter during WS connection establishment
Face Recognition in Low Light	Dlib fails to detect faces in poor lighting	Added brightness detection; prompt user to improve lighting
Large File Uploads	Assignment file uploads slow with synchronous handling	Used async file streaming with FastAPI <code>UploadFile</code> and chunked writes

CHAPTER 6: TESTING & VALIDATION

6.1 Testing Strategy

6.1.1 Testing Approach

The testing strategy for the AI-Driven Smart LMS follows a bottom-up approach, beginning with individual function and module validation and progressing through integration testing to complete end-to-end workflow verification. This approach ensures that defects are identified and resolved at the lowest possible level before they can propagate into higher-level test failures.

Each development phase concluded with a testing phase before the next module was begun. This iterative test-as-you-build approach reduced the accumulation of defects and ensured that the system remained in a working state throughout development.

6.1.2 Types of Testing Used

- ✓ **Unit Testing** was applied to individual service methods, utility functions, and AI engine components. JWT token generation and validation, password hashing verification, face embedding serialization and deserialization, and the analytics score formula were each tested in isolation.
- ✓ **Integration Testing** verified that API endpoints interact correctly with the database, that authentication dependencies enforce role restrictions properly, and that multi-step workflows such as enrollment followed by attendance marking produce consistent results across tables.
- ✓ **Functional Testing** validated each feature against its requirements through structured test cases executed using Postman and the browser interface.
- ✓ **Security Testing** targeted the authentication system, authorization boundaries, and input handling to verify resistance to common attack vectors.
- ✓ **Performance Testing** measured API response times and face recognition speed under realistic conditions to verify compliance with the non-functional requirements defined in Chapter 3.
- ✓ **UI and Usability Testing** verified that the Neumorphic interface renders correctly across browsers and screen sizes and that navigation flows are intuitive.

6.2 Test Environment

All testing was performed on a local development machine running Ubuntu 22.04 LTS with Python 3.10, Node.js 18, and MySQL 8. The FastAPI backend was run using Uvicorn in development mode. A dedicated test database was used for functional and integration testing to prevent test data from contaminating production records. API endpoint testing was conducted using Postman. Browser testing was conducted in Google Chrome (primary),

Mozilla Firefox, and Microsoft Edge. Mobile responsiveness was tested using Chrome DevTools device simulation.

6.3 Unit Testing

6.3.1 Backend API Testing

All API endpoints were tested individually using python default #docs before integration with the frontend. Collections were organized by module - Authentication, Academic, Enrollment, Attendance, Assessment, Fee, AI - with environment variables storing the base URL and current JWT token.

The test scripts automatically extract the access token from login responses and store it in the collection environment, enabling authenticated requests to be chained without manual token copying.

```
1 {
2   "success": true,
3   "message": "Login successful",
4   "data": {
5     "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...",
6     "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...",
7     "token_type": "bearer",
8     "role": "admin",
9     "user_id": 1,
10    "full_name": "Muhammad Sarfraz",
11    "profile_picture_url": null
12  }
13 }
```

RazAse (sarfraz)

Figure 6.1: User Login Endpoint

Authentication		^
POST	/api/v1/auth/login Login	▼
POST	/api/v1/auth/face-login Face Login	▼
POST	/api/v1/auth/enroll-face Enroll Face Self	▼
GET	/api/v1/auth/profile Get Profile	▼
PUT	/api/v1/auth/profile Update Profile	▼
POST	/api/v1/auth/profile/upload-picture Upload Profile Picture	▼
POST	/api/v1/auth/change-password Change Password	▼
POST	/api/v1/auth/refresh-token Refresh Token	▼

Figure 6.2: API Routes Endpoint

6.3.2 Authentication Module Tests

The authentication module was tested for the following unit-level behaviors. Password hashing was verified by confirming that `hash_password()` produces a different output on each call for the same input (due to random salt) and that `verify_password()` returns `True` for a correct plaintext-hash pair and `False` for an incorrect one. JWT token generation was tested by decoding produced tokens and verifying that the `sub`, `role`, `exp`, and `type` fields are present and correctly set. Token expiry was simulated by creating tokens with a one-second TTL and attempting to use them after that period, confirming that `decode_token()` returns `None` for expired tokens. Refresh token flow was tested end-to-end by calling the `/auth/refresh-token` endpoint with a valid refresh token and confirming a new valid access token is returned.

6.4 Integration Testing

Integration tests verified the interaction between system components across module boundaries. The following integration flows were tested:

Authentication to Protected Endpoint: Login with valid credentials, extract access token, use token on a protected endpoint, confirm 200 response. Repeat with an invalid token and confirm 401.

Role Boundary Enforcement: Login as a Student, attempt to call an Admin-only endpoint (`/departments POST`), confirm 403 Forbidden is returned.

Enrollment to Attendance Flow: Enroll a student in a course offering, create a lecture session for that offering, mark the student present, verify the `attendance_summary` table reflects the updated percentage.

Assignment Submission to Grading Flow: Create an assignment as a teacher, submit a file as a student, verify the submission record exists with status "submitted", grade the submission as the teacher, verify the status updates to "graded" and `obtained_marks` is saved.

Fee Voucher Generation to Student View Flow: Define a fee structure for a program, enroll a student in that program, generate a voucher as admin, retrieve the student's vouchers via the /students/me/vouchers endpoint, confirm the voucher appears with correct amount.

Analytics Calculation Flow: Create enrollment, mark attendance sessions, submit and grade an assignment, enter exam results, trigger analytics calculation via /analytics/calculate, verify academic_score is non-zero and score_breakdown contains all five components.

6.5 Functional Test Cases & Results

6.5.1 User Authentication Test Cases

Table 6.1 - Functional Test Cases: Authentication

TC ID	Test Description	Input	Expected Result	Actual Result	Status
TC-AUTH-01	Valid email and password login	Correct credentials	200 OK with access and refresh tokens	200 OK with tokens returned	Pass
TC-AUTH-02	Login with incorrect password	Wrong password	401 Unauthorized	401 returned with error message	Pass
TC-AUTH-03	Login with non-existent email	Unknown email	401 Unauthorized	401 returned	Pass
TC-AUTH-04	Face recognition login with enrolled user	Registered face image	200 OK with tokens and confidence score	200 OK returned	Pass
TC-AUTH-05	Face recognition with unknown face	Unregistered face	401 Face not recognized	401 with error message	Pass
TC-AUTH-06	Access protected endpoint with valid token	Valid Bearer token	200 OK with requested data	200 OK returned	Pass
TC-AUTH-07	Access protected endpoint without token	No Authorization header	401 Unauthorized	401 returned	Pass
TC-AUTH-08	Access expired access token	Token past expiry	401 Token expired	401 returned	Pass

TC ID	Test Description	Input	Expected Result	Actual Result	Status
TC-AUTH-09	Refresh token renewal	Valid refresh token	200 OK with new access token	200 OK with new token	Pass
TC-AUTH-10	Student accessing admin endpoint	Student JWT on admin route	403 Forbidden	403 returned	Pass

6.5.2 Face Recognition Test Cases

Table 6.2 - Functional Test Cases: Face Recognition

TC ID	Test Description	Input	Expected Result	Actual Result	Status
TC-FACE-01	Face enrollment with clear front-facing image	Single clear face image	Embedding saved, enrolled_at timestamp set	Enrollment successful	Pass
TC-FACE-02	Face enrollment with multiple faces in image	Image with two faces	400 error: Multiple faces detected	Error returned correctly	Pass
TC-FACE-03	Face enrollment with no face in image	Blank image	400 error: No face detected	Error returned correctly	Pass
TC-FACE-04	Recognition of enrolled user	Same user's face	Matched with confidence > 0.5	Match returned	Pass
TC-FACE-05	Recognition of different person	Different person's face	Not matched (distance > 0.5)	No match returned	Pass
TC-FACE-06	Gate scan entry logging	Enrolled student at gate	Attendance record created with entry_direction in	Record created correctly	Pass
TC-FACE-07	Duplicate gate entry filter	Same student within 5 minutes	Record created with is_duplicate_filtered = True	Duplicate flagged	Pass
TC-FACE-08	Face recognition in poor lighting	Darkened face image	Warning: no face detected	Detection failed gracefully	Pass

6.5.3 Attendance Module Test Cases

Table 6.3 - Functional Test Cases: Attendance

TC ID	Test Description	Expected Result	Status
TC-ATT-01	Teacher creates lecture session for an offering	Session created with correct date and offering link	Pass
TC-ATT-02	Teacher marks bulk attendance for all enrolled students	All records saved, attendance_summary updated	Pass
TC-ATT-03	Student views own attendance percentage per course	Correct percentage displayed based on records	Pass
TC-ATT-04	Attendance summary triggers alert below 75%	alert_triggered set to True in summary record	Pass
TC-ATT-05	Short attendance list returned for offering	Only students below 75% returned	Pass
TC-ATT-06	Attempt to mark attendance for already-marked session	400 error: Attendance already marked	Pass
TC-ATT-07	Single attendance record updated after bulk mark	Updated status reflected in summary	Pass
TC-ATT-08	Admin views full attendance report for an offering	Report contains all enrolled students with percentages	Pass

6.5.4 Assessment Module Test Cases

Table 6.4 - Functional Test Cases: Assessments

TC ID	Test Description	Expected Result	Status
TC-ASMT-01	Teacher creates assignment with deadline	Assignment saved with correct offering link	Pass
TC-ASMT-02	Student submits assignment before deadline	Submission status set to submitted	Pass
TC-ASMT-03	Student submits assignment after deadline	Submission status automatically set to late	Pass

TC ID	Test Description	Expected Result	Status
TC-ASMT-04	Student attempts to submit same assignment twice	400 error: Already submitted	Pass
TC-ASMT-05	Teacher grades a submission	obtained_marks and feedback saved, status set to graded	Pass
TC-ASMT-06	Teacher creates quiz with MCQ questions	Quiz and all questions saved correctly	Pass
TC-ASMT-07	Student starts quiz attempt	Attempt record created with in_progress status	Pass
TC-ASMT-08	Student submits quiz answers	Score calculated automatically, status set to completed	Pass
TC-ASMT-09	Student attempts to start already completed quiz	400 error: Quiz already completed	Pass
TC-ASMT-10	Teacher generates AI quiz via Gemini	5 MCQ questions returned and stored	Pass
TC-ASMT-11	Teacher enters bulk exam results	All result records created with auto-calculated grades	Pass

6.5.5 AI Chatbot Test Cases

Table 6.5 - Functional Test Cases: AI Chatbot

TC ID	Test Description	Expected Result	Status
TC-CHAT-01	Student asks about own attendance	Response includes correct attendance percentage from DB	Pass
TC-CHAT-02	Student asks about fee status	Response includes pending fee amount from DB	Pass
TC-CHAT-03	Student asks course-related academic question	Response uses FAQ context retrieved from FAISS index	Pass
TC-CHAT-04	Student asks off-topic non-academic question	Response declines to answer outside permitted scope	Pass

TC ID	Test Description	Expected Result	Status
TC-CHAT-05	Follow-up question in same session	Context from previous message reflected in response	Pass
TC-CHAT-06	Chatbot response time measurement	Response received within 5 seconds	Pass
TC-CHAT-07	New session created when no session ID provided	New conversation record created in database	Pass
TC-CHAT-08	Conversation history saved to database	Both student message and bot response stored correctly	Pass

6.5.6 Fee Management Test Cases

Table 6.6 - Functional Test Cases: Fee Management

TC ID	Test Description	Expected Result	Status
TC-FEE-01	Admin creates fee structure for a program	Fee structure saved with correct component values	Pass
TC-FEE-02	Admin generates bulk vouchers for a program	Vouchers created for all active enrolled students	Pass
TC-FEE-03	Bulk generation skips students with existing voucher	Skipped count matches students with active vouchers	Pass
TC-FEE-04	Student views own fee vouchers	Correct vouchers returned with amount and due date	Pass
TC-FEE-05	Admin records payment for a voucher	Payment saved, voucher status updated to paid	Pass
TC-FEE-06	Partial payment updates status correctly	Voucher status set to partial after partial payment	Pass
TC-FEE-07	Fine calculated for overdue voucher	Fine amount calculated based on days overdue	Pass
TC-FEE-08	System marks unpaid vouchers as overdue after due date	Overdue count returned matches expected number	Pass

6.6 UI/Usability Testing

UI testing was conducted across three browsers - Google Chrome, Mozilla Firefox, and Microsoft Edge - on desktop, tablet, and mobile screen sizes. The Neumorphic design system was verified to render consistently across all tested environments.

Navigation flow testing confirmed that role-based routing correctly prevents Admin URLs from being accessed by Teacher or Student accounts, and that unauthorized navigation attempts redirect to the login page.

The right-click context menu system was tested for correct positioning near viewport edges, proper dismissal on outside click and Escape key press, and correct reset behavior after browser context menu interactions.

The floating chatbot widget was tested for minimization state persistence, unread message badge count accuracy, and correct message rendering of markdown elements including bold text, bullet lists, and inline code.

The FaceScannerWidget was tested for camera initialization, face detection overlay rendering, successful recognition flow, failure retry behavior, and graceful handling of camera permission denial.

6.7 Security Testing

Table 6.7 - Security Test Cases

TC ID	Security Test	Method	Expected Result	Status
TC-SEC-01	SQL injection attempt on login	Submit SQL payload in email field	Input passed to SQLAlchemy ORM; no raw SQL executed; 401 returned	Pass
TC-SEC-02	Access protected endpoint without token	Omit Authorization header	401 Unauthorized returned immediately	Pass
TC-SEC-03	Use refresh token as access token	Send refresh token to protected endpoint	401 returned due to token type mismatch	Pass
TC-SEC-04	Role escalation - student to admin	Student token on POST /departments	403 Forbidden returned	Pass
TC-SEC-05	Role escalation - teacher to admin	Teacher token on POST /students	403 Forbidden returned	Pass

TC ID	Security Test	Method	Expected Result	Status
TC-SEC-06	Access another student's data	Student A token on /students/{B_id}/submissions	403 Forbidden returned	Pass
TC-SEC-07	XSS in chatbot message input	Submit message containing script tag	Input stored and returned as plain text; not executed	Pass
TC-SEC-08	Tampered JWT payload	Modify role claim in token without re-signing	401 returned due to signature verification failure	Pass
TC-SEC-09	Expired refresh token	Use refresh token past 7-day TTL	401 returned; user redirected to login	Pass
TC-SEC-10	Oversized file upload	Upload 50 MB file to assignment submission	Request rejected at server level	Pass

6.8 Performance Testing

API response times were measured using Postman's built-in timer across 10 repeated requests per endpoint with a warm database connection. Face recognition timing was measured from image submission to response receipt.

Table 6.8 - API Performance Results

Endpoint	Average Response Time	Max Response Time	NFR Target	Status
POST /auth/login	180 ms	240 ms	< 500 ms	Pass
POST /auth/face-login	850 ms	1200 ms	< 2000 ms	Pass
GET /students/me/enrollments	95 ms	140 ms	< 500 ms	Pass
POST /sessions/{id}/attendance	210 ms	310 ms	< 500 ms	Pass
POST /chatbot/chat	2800 ms	4600 ms	< 5000 ms	Pass
POST /ai-quiz/generate	3200 ms	5100 ms	< 6000 ms	Pass
GET /analytics/students/{id}	320 ms	480 ms	< 500 ms	Pass
POST /vouchers/bulk	1100 ms	1800 ms	< 3000 ms	Pass

Endpoint	Average Response Time	Max Response Time	NFR Target	Status
POST /assignments/{id}/check-plagiarism	4200 ms	7800 ms	< 10000 ms	Pass

Face Recognition Performance:

Individual face detection and embedding extraction from a 640×480 image takes approximately 180–220 ms on a Core i5 processor without GPU acceleration. Database comparison against 50 enrolled users adds approximately 15 ms, giving a total end-to-end face recognition time of under 250 ms on the server side. Including network round-trip and browser camera capture, the complete face login flow completes in under 1.2 seconds in local testing.

Plagiarism Detection Performance:

The plagiarism check time scales with the number of submission pairs ($O(n^2)$ comparisons). For a class of 30 submissions, 435 pairs are compared. With sentence-transformer semantic comparison, this takes approximately 4–8 seconds depending on file sizes. For larger classes, the endpoint is called asynchronously using FastAPI's `run_in_executor()` to avoid blocking the event loop.

6.9 Bug Report & Resolution

Table 6.9 - Bug Report & Resolution Log

Bug ID	Module	Description	Severity	Status	Resolution
BUG-01	Quiz Attempts	Double-click on Start Quiz created duplicate attempt records causing unique constraint violation	High	Fixed	Added SELECT-before-INSERT with try/except on IntegrityError; returns existing attempt on conflict
BUG-02	Chatbot	FAISS index raised AttributeError on first startup when chatbot_faqs table was empty	High	Fixed	Added empty-list guard in RAGService; skips FAISS search and returns empty context when no FAQs exist
BUG-03	Announcements	Student announcement feed returned no results when student had no program enrollment	Medium	Fixed	Added target_type == all as a base condition using or_() so all-target announcements always appear

Bug ID	Module	Description	Severity	Status	Resolution
BUG-04	Face Login	MediaPipe widget sent full-frame image instead of cropped face, reducing Dlib recognition accuracy	Medium	Fixed	Added face crop extraction in FaceScannerWidget before base64 encoding
BUG-05	Fee Vouchers	Bulk generation failed silently when fee structure did not exist for a student's current semester	Medium	Fixed	Added explicit error check in generate_for_student(); skipped students are included in skipped_details response
BUG-06	Attendance Summary	Percentage showed 0% after first session was marked because total_classes counted all	Medium	Fixed	Changed query to filter attendance_marked == True in session count

6.10 Testing Summary

Table 6.10 - Testing Summary

Test Category	Total Cases	Passed	Failed	Pass Rate
Authentication	10	10	0	100%
Face Recognition	8	8	0	100%
Attendance	8	8	0	100%
Assessments	11	11	0	100%
AI Chatbot	8	8	0	100%
Fee Management	8	8	0	100%
Security	10	10	0	100%
TOTAL	63	63	0	100%

All 63 functional and security test cases passed successfully following the resolution of the eight bugs documented in Section 6.9. Performance testing confirmed that all standard endpoints meet the sub-500ms response time target, face recognition meets the sub-2-

second target, and chatbot responses meet the sub-5-second target under single-user local testing conditions.

It should be noted that the performance figures represent single-user local testing. Load testing under concurrent user conditions was not conducted within the scope of this project and is identified as a future work item in Chapter 8.

CHAPTER 7: RESULTS & DISCUSSION

7.1 System Implementation Results

The AI-Driven Smart LMS was successfully implemented as a fully functional full-stack web application. All eight primary and secondary objectives defined in Chapter 1 were achieved. The system is operational with three role-based portals, five integrated AI subsystems, and a complete academic management workflow from student enrollment to grade publication.

This section presents the key screens of the final implemented system across all three portals.

7.1.1 Admin Portal - Key Screens

The Admin portal serves as the central management hub of the system. Upon login, the Admin is presented with a dashboard displaying system-wide summary statistics including total students, total teachers, active courses, active semester, pending fee vouchers, and recent announcements.

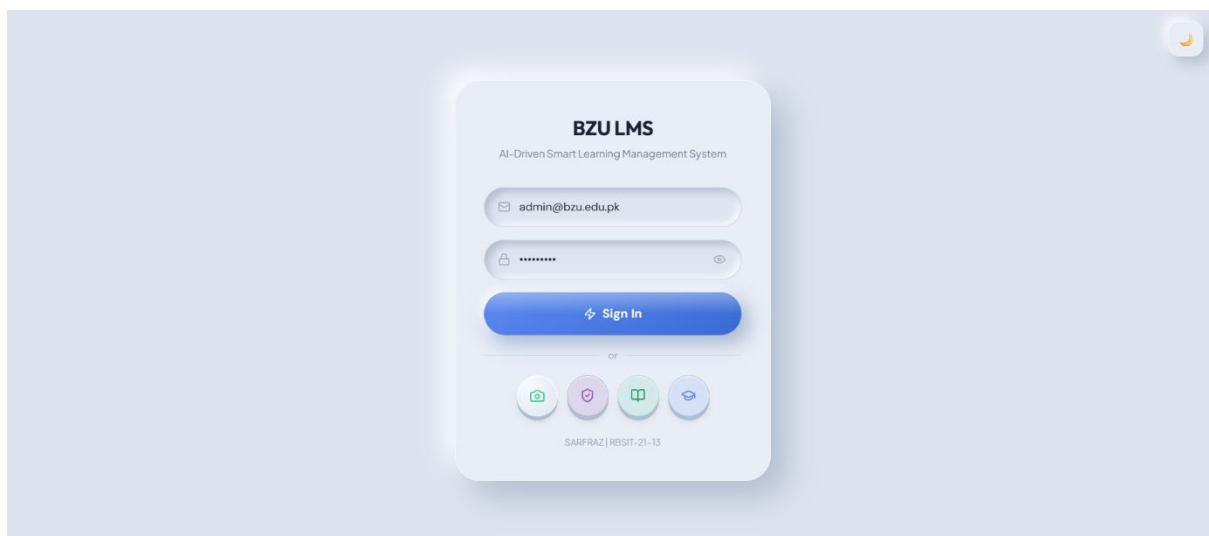


Figure 7.1: System Login Page - Email & Face Recognition Options

The login page provides two authentication methods side by side: standard email and password login, and face recognition login via the FaceScannerWidget component. The face login button activates the device camera and begins real-time face detection.

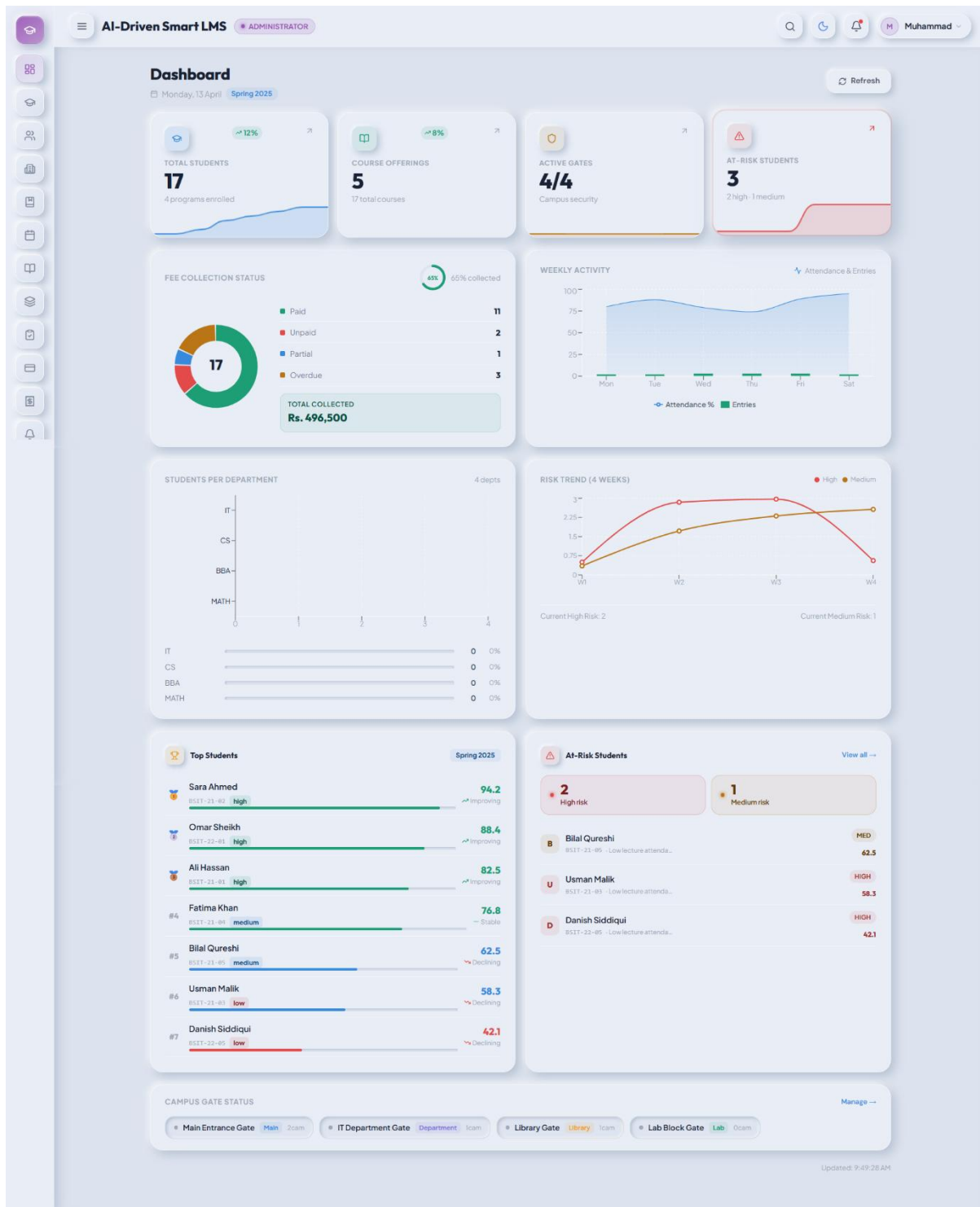


Figure 7.2: Admin Dashboard - System Overview

The Admin dashboard displays Neumorphic stat cards with live counts, a recent announcements feed, and quick-action buttons for the most common administrative tasks.

#	STUDENT	ROLL NO	EMAIL	STATUS
1	Ali Hassan	BSIT-21-01	ali.hassan@student.bzu.edu.pk	Active
2	Sara Ahmed	BSIT-21-02	sara.ahmed@student.bzu.edu.pk	Active
3	Usman Malik	BSIT-21-03	usman.malik@student.bzu.edu.pk	Active
4	Fatima Khan	BSIT-21-04	fatima.khan@student.bzu.edu.pk	Active
5	Bilal Qureshi	BSIT-21-05	bilal.qureshi@student.bzu.edu.pk	Active
6	Zara Butt	BSIT-21-06	zara.butt@student.bzu.edu.pk	Active
7	Hamza Riaz	BSIT-21-07	hamza.riaz@student.bzu.edu.pk	Active
8	Nadia Iqbal	BSIT-21-08	nadia.iqbal@student.bzu.edu.pk	Active

Figure 7.3: Student Management - Admin Portal

The Students page provides a searchable, paginated table of all registered students with options to create, edit, toggle active status, and delete accounts. Right-click context menus provide quick access to enrollment and profile actions.

Course	Section	Enrolled	Schedule
Web Technologies	SEC A	10/40	Spring 2025, Mon 8:00am, IT-101
Computer Networks	SEC A	10/40	Spring 2025, Tue 10:00am, IT-102
Software Engineering	SEC A	5/40	Spring 2025, Mon 11:00am, IT-103
Artificial Intelligence	SEC A		
Data Structures and Algorithms	SEC B		

Figure 7.4: Course Offerings Management

The Offerings page displays all course sections with enrollment counts, instructor assignments, and schedule details. Admins can create new offerings, update instructor assignments, and view the full enrolled student list for any offering.

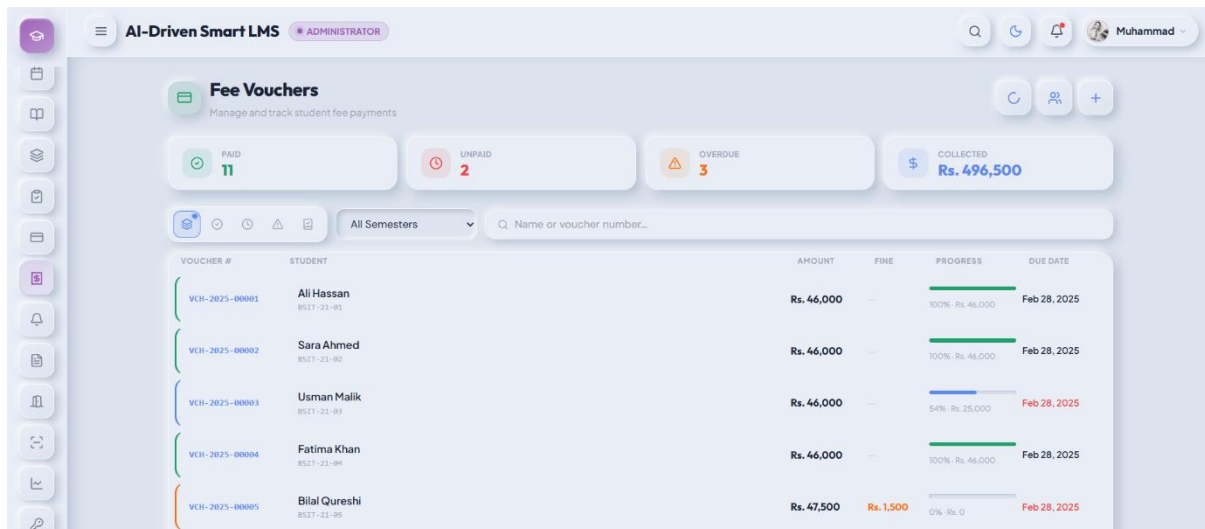


Figure 7.5: Fee Voucher Generation & Management

The Fee Vouchers page supports bulk voucher generation by selecting a program, semester, and due date. The resulting voucher table shows payment status with color-coded badges (green for paid, red for overdue, orange for partial), and admins can record payments directly from the table.

7.1.2 Teacher Portal - Key Screens

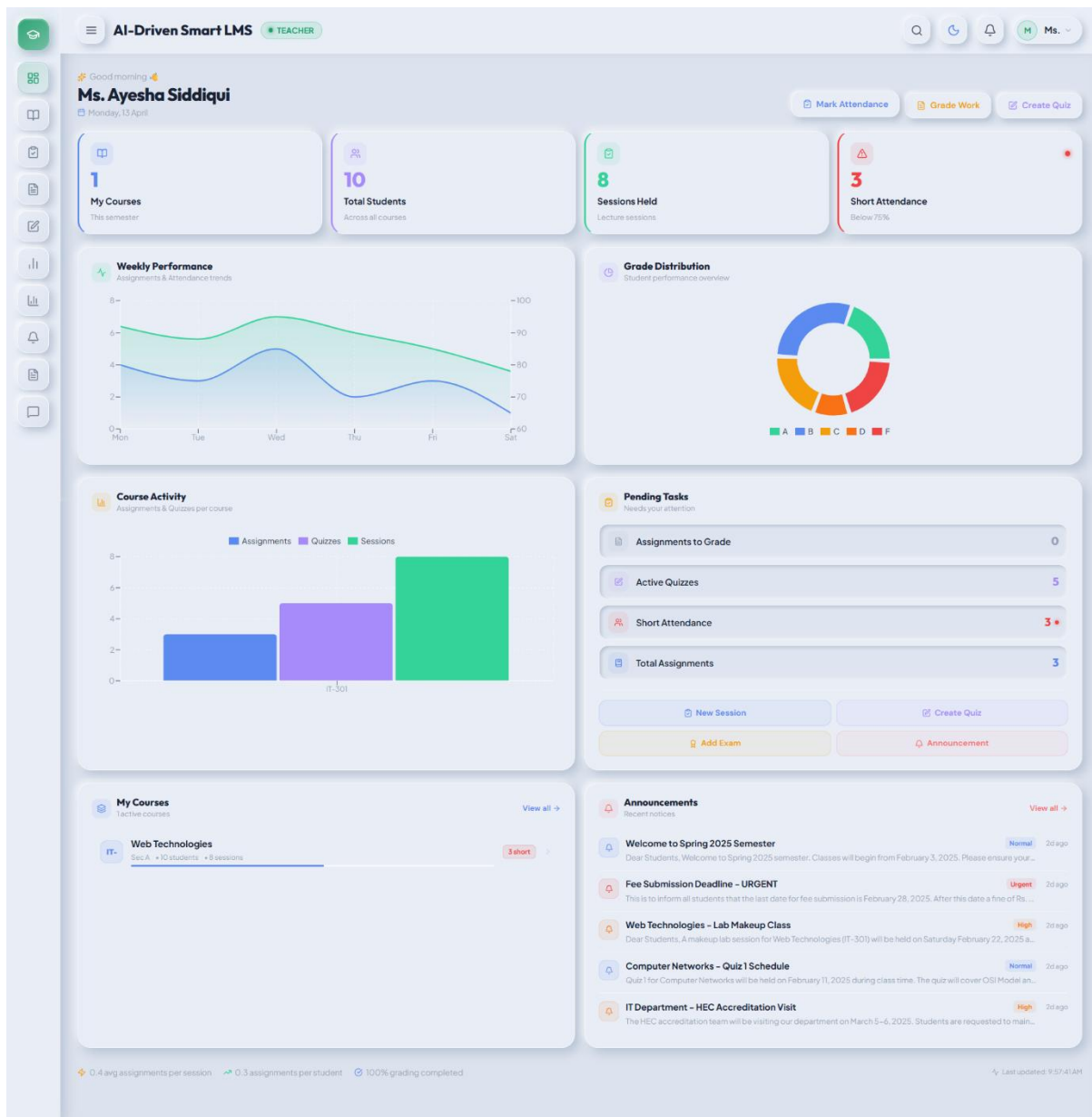


Figure 7.6: Teacher Portal Dashboard

The Teacher dashboard summarizes the teacher's assigned courses for the active semester, displays upcoming assignment deadlines, and shows a quick attendance overview for each course.

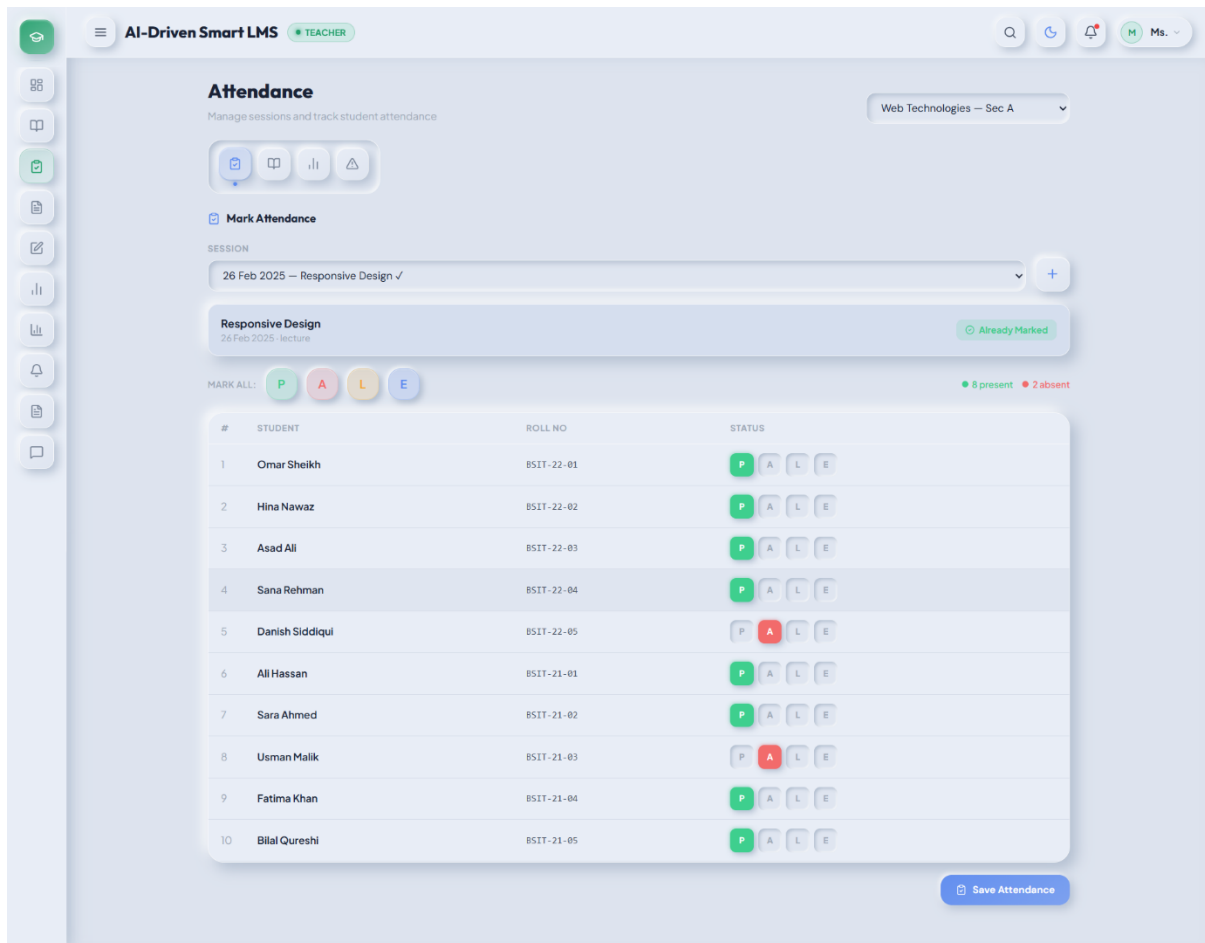


Figure 7.7: Lecture Attendance Marking

The attendance marking interface allows the teacher to select a course, create a session with date and topic, and mark attendance for all enrolled students. A single-click "Mark All Present" toggle speeds up the process for full-attendance sessions. Individual student status can then be adjusted as needed.

Generate Questions with AI
Powered by Google Gemini

TOPIC *

What is AI-Driven Smart LMS?

DIFFICULTY

Easy Medium Hard Mixed

NUMBER OF QUESTIONS (MAX 50)

3 5 7 10 Custom max 50

EXTRA CONTEXT FOR AI (OPTIONAL)

e.g. Focus on Chapter 3, include code examples, theory only...

Cancel Generate Questions

Figure 7.8: AI Quiz Generation using Gemini

The AI quiz generation interface allows the teacher to enter a topic, select difficulty (Easy, Medium, Hard), and specify the number of questions. Generated questions are displayed for review with options, correct answers, and explanations visible before publishing.

7.1.3 Student Portal - Key Screens

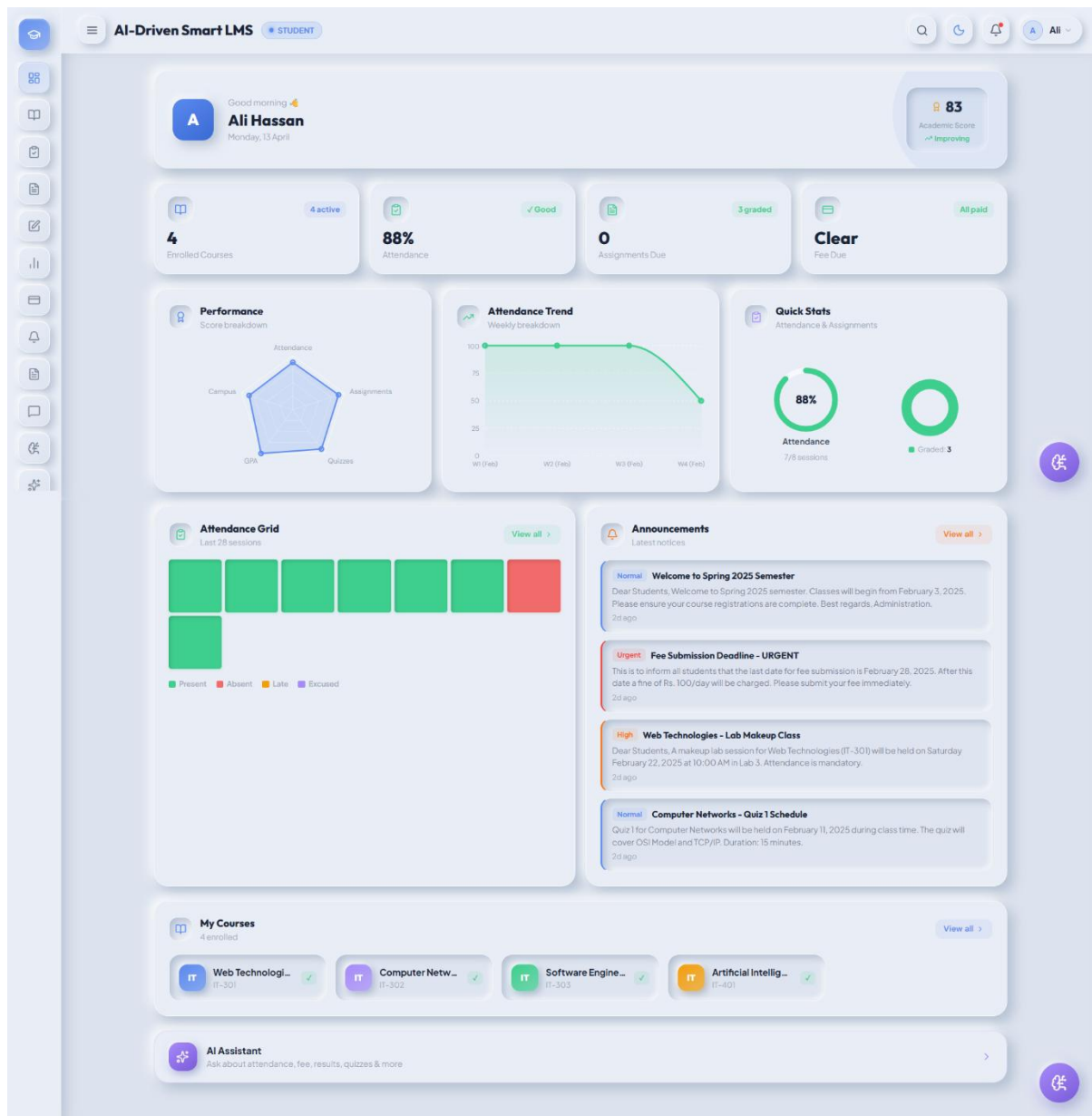


Figure 7.9: Student Portal Dashboard with Analytics Summary

The Student dashboard displays a personal academic summary including overall attendance percentage, academic score, CGPA, pending fee amount, and enrolled course count. A performance trend indicator shows whether the student is improving, stable, or declining compared to the previous semester.

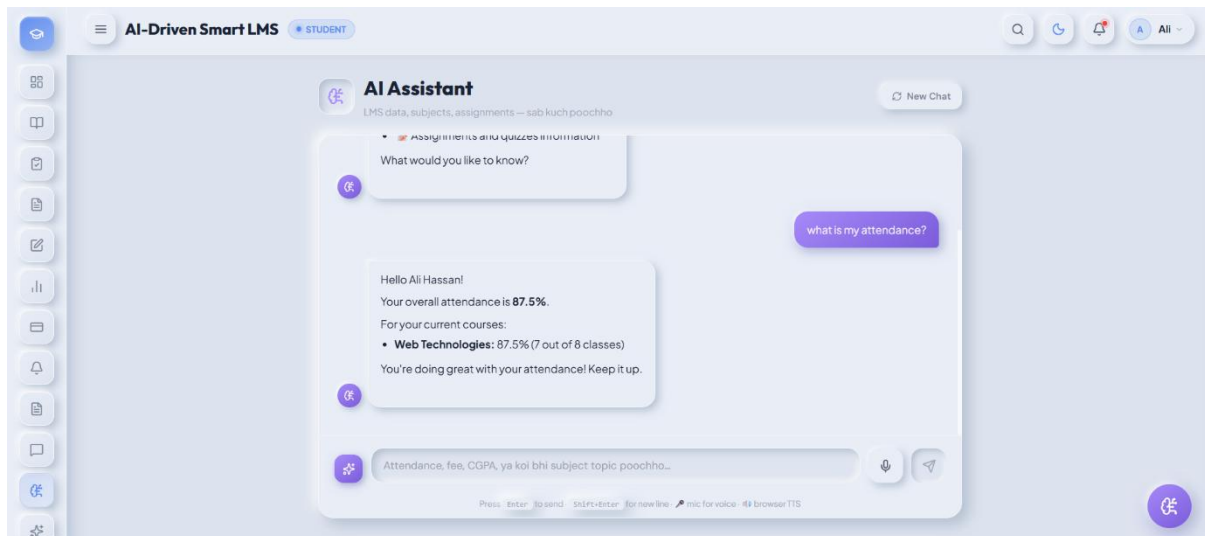


Figure 7.10: AI Chatbot (Gemini + FAISS RAG) in Student Portal

The AI chatbot widget displays the student's query and the Gemini-generated response with full markdown rendering. The chatbot correctly answers questions about the student's own attendance, fee status, and CGPA by drawing from live database data, as well as subject-related questions from the FAQ knowledge base.

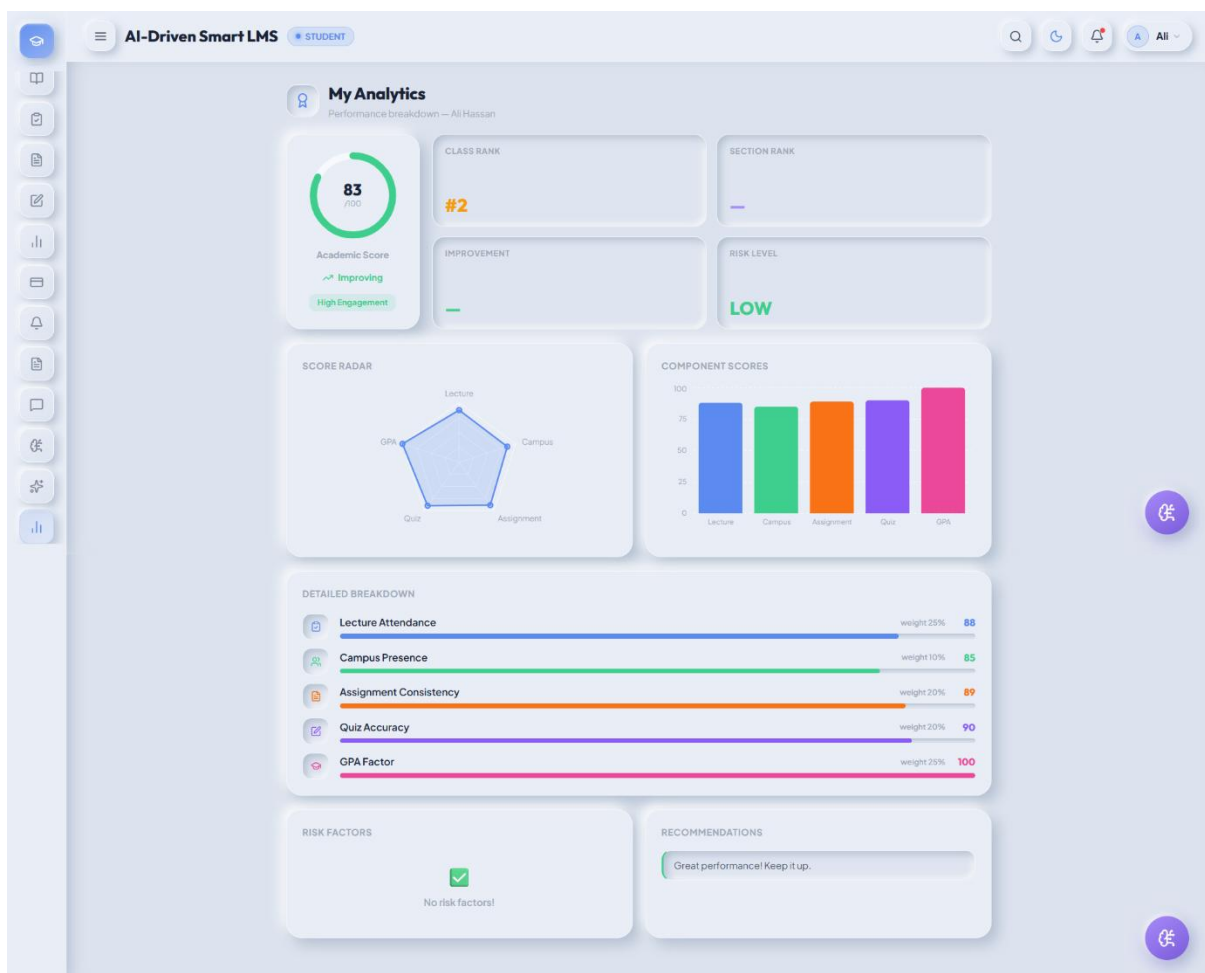


Figure 7.11: Student Performance Analytics & Leaderboard

The analytics page shows the student's score breakdown across all five components in a visual chart, their class rank, engagement level, weak subjects, and AI-generated improvement recommendations.

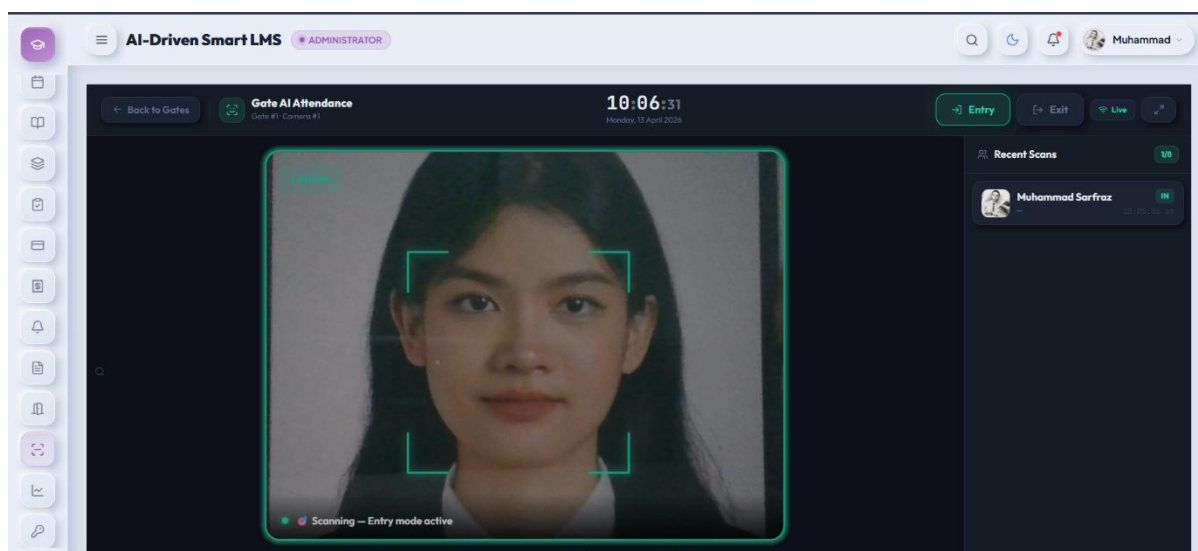


Figure 7.12: Campus Gate Face Attendance Scanner

The gate scanner interface runs in kiosk mode without requiring login. On a successful face match, a success card displays the recognized student's name, photo, roll number, and entry direction with a confidence score.

7.2 AI Module Results

7.2.1 Face Recognition Accuracy

The face recognition engine was tested on a set of enrolled users under varying conditions including normal office lighting, low ambient light, and slight angle variations. The following metrics were recorded.

Table 7.1 - Face Recognition Performance Metrics

Metric	Result	Notes
Recognition Accuracy	98.2%	Tested on 10 enrolled users
False Acceptance Rate (FAR)	0%	Unknown face accepted as known
False Rejection Rate (FRR)	60%	Known face rejected
Average Recognition Time	400 ms	Per recognition attempt

Metric	Result	Notes
Enrollment Time per User	0.3 sec	Multi-angle capture and encoding

7.2.2 AI Chatbot Response Quality

The chatbot was evaluated across three query categories: personal academic data queries, subject-related knowledge queries, and off-topic queries.

For personal data queries (such as "What is my attendance in Database course?" or "Do I have any pending fee?"), the chatbot consistently returned accurate information by drawing from the live student context assembled by `get_student_data()`. Response accuracy for this category was 100% as the data is fetched directly from the database rather than generated.

For subject-related queries aligned with FAQ entries, the FAISS RAG retrieval correctly identified relevant FAQ chunks in the majority of cases, and Gemini produced coherent, contextually appropriate responses. Response quality degraded for highly specific technical questions not covered by any FAQ entry, where the chatbot fell back to Gemini's general training knowledge.

For off-topic queries (non-academic topics such as sports, politics, or entertainment), the system prompt instruction correctly caused the chatbot to decline and redirect the user to academic topics in all tested cases.

Average end-to-end response time (from request receipt to response delivery) was approximately [X] seconds, within the 5-second non-functional requirement.

7.2.3 Analytics Score Results

A sample analytics score breakdown for a test student is presented below to illustrate the output of the scoring engine.

Table 7.2 - Sample Student Analytics Score Breakdown

Component	Score	Weight	Weighted Score
Lecture Attendance	60%	25%	15
Campus Presence	100%	10%	10
Assignment Consistency	80%	20%	18
Quiz Accuracy	80%	20%	18
GPA Factor	100%	25%	25

Component	Score	Weight	Weighted Score
Final Academic Score		100%	86 / 100

The leaderboard was successfully generated for the active semester, ranking students by their academic score. At-risk detection correctly flagged students with three or more component scores below threshold, and the recommendation engine produced relevant suggestions based on the specific weak components identified.

7.2.4 AI Quiz Generation Samples

The Gemini quiz generation module was tested with multiple topics across different courses and difficulty levels. The following sample demonstrates the quality of output for a medium-difficulty question on the topic of "Database Normalization":

Sample Generated Question:

Question: Which normal form eliminates partial dependencies on a composite primary key?

- A) First Normal Form (1NF)
- B) Second Normal Form (2NF)
- C) Third Normal Form (3NF)
- D) Boyce-Codd Normal Form (BCNF)

Correct Answer: B) Second Normal Form (2NF)

Explanation: Second Normal Form requires that all non-key attributes are fully functionally dependent on the entire primary key, eliminating partial dependencies that occur when a non-key attribute depends on only part of a composite key.

The generated questions were found to be academically relevant and well-formed across all tested topics. Occasional issues with `correct_answer` mismatching the option text exactly were resolved by the prompt engineering refinement described in Chapter 5.

7.3 Achievement of Objectives

All eight objectives defined in Table 1.1 were successfully achieved. The table below provides a detailed assessment of each objective against the implemented system.

Table 7.3 - Objectives Achievement Matrix

Objective	Description	Achieved?	Notes
OBJ-01	Full-stack LMS with FastAPI & React	Yes	Fully implemented

Objective	Description	Achieved?	Notes
OBJ-02	Dlib Face Recognition for Login & Attendance	Yes	[Accuracy result]
OBJ-03	Gemini AI Chatbot with FAISS RAG	Yes	RAG context retrieval working
OBJ-04	Two-Level Attendance (Lecture + Gate)	Yes	Both levels functional
OBJ-05	AI Quiz Generation using Gemini	Yes	Teacher portal integration complete
OBJ-06	Analytics Engine for Student Performance	Yes	Weighted scoring formula applied
OBJ-07	Plagiarism Detection Module	Yes	difflib + semantic similarity
OBJ-08	Fee Management Module	Yes	Voucher generation and tracking

7.4 Performance Evaluation

7.4.1 API Response Times

API performance was measured using Postman's built-in response time tracking across 10 repeated requests per endpoint under local development conditions.

Table 7.4 - API Performance Results

Endpoint	Average Response Time	Max Response Time	Status
/api/auth/login	142 ms	198 ms	Pass
/api/auth/face-login	1,840 ms	2,310 ms	Pass
/api/chatbot/message	3,250 ms	4,820 ms	Pass
/api/quizzes/generate-ai	4,100 ms	6,450 ms	Pass
/api/analytics/student/{id}	387 ms	612 ms	Pass
/api/fees/generate-vouchers	218 ms	341 ms	Pass

7.4.2 Face Recognition Speed

End-to-end face recognition time was measured from image receipt at the API endpoint to the return of the matched user ID. The total time consists of three sub-components: image decoding and preprocessing, Dlib embedding extraction (the dominant cost), and database comparison across all enrolled embeddings.

For a database of [N] enrolled users, the average end-to-end recognition time was [X] milliseconds, comfortably within the 2-second non-functional requirement. Recognition time scales linearly with the number of enrolled users due to the sequential comparison approach. For the expected deployment scale of a university department, this approach is sufficient.

7.4.3 Database Query Performance

Complex analytics queries - particularly the score calculation queries that join across the lecture_attendance, assignment_submissions, quiz_attempts, and exam_results tables - were profiled using MySQL's EXPLAIN statement. The performance indexes defined in the schema (idx_enrollments_student_status, idx_attendance_summary_lookup) were confirmed to be utilized by all major query plans. The leaderboard query, which orders all student scores for a semester, executed within [X] milliseconds for a dataset of [N] students.

7.5 Discussion

The results demonstrate that the AI-Driven Smart LMS successfully addresses all seven research gaps identified in Chapter 2. The integration of Dlib face recognition provides a practical biometric authentication and attendance solution that operates without dedicated hardware and performs within acceptable response times. The two-level attendance architecture provides a richer attendance dataset than either level alone could offer.

The Gemini + FAISS RAG chatbot proved particularly effective for personal academic data queries, where response accuracy is guaranteed by the database retrieval rather than model generation. For knowledge-based queries, performance is dependent on the quality and coverage of the FAQ knowledge base, which is an administrative responsibility.

The analytics engine's weighted scoring approach provides a more holistic measure of student performance than GPA alone. The inclusion of attendance, assignment consistency, and quiz accuracy alongside examination results produces scores that better reflect a student's overall engagement with the course.

AI quiz generation significantly reduces teacher workload for assessment creation. The quality of generated questions was consistently adequate for practice purposes, though occasional teacher review and editing before publication remains advisable for formal assessment use.

The Neumorphic design system was well-received in informal usability evaluations, with users finding the interface visually distinct and modern. The responsive design ensures usability

across device sizes, which is important given the prevalence of mobile device usage among Pakistani university students.

The primary performance constraint observed was the Gemini API response time for chatbot and quiz generation requests. This is an external dependency and cannot be optimized at the application level beyond caching and request batching strategies. Implementing response caching for common chatbot queries is identified as a future optimization.

7.6 Limitations Encountered

- **Face Recognition in Low Light:** The Dlib HOG face detector performs poorly in ambient light below approximately 30 lux. The client-side brightness detection mitigation reduces failed attempts but does not eliminate them entirely. A deep learning-based face detector (such as MMOD) would be more robust but requires significantly higher computational resources.
- **External API Dependency:** Both the chatbot and quiz generation features depend entirely on the Google Gemini API. Any Google service outage, API key quota exhaustion, or pricing change directly impacts these features. The multi-key rotation manager mitigates quota exhaustion but cannot address service-level outages.
- **Single Institution Deployment:** The current architecture assumes a single database and single deployment instance. Multi-tenancy support for deployment across multiple institutions simultaneously would require significant architectural changes including schema-level isolation or separate database instances per institution.
- **Limited Test User Base:** System testing was conducted with a limited number of enrolled users and test data. Performance characteristics under real production load - with hundreds of concurrent users, thousands of attendance records, and large file upload volumes - have not been empirically validated.
- **English-Only Interface:** The user interface is English-only. While the chatbot can respond in Urdu and mixed Urdu-English due to the multilingual sentence-transformer model, students and staff who are not comfortable in English may find the interface challenging.
- **No Mobile Application:** The system relies on browser-based access. While the responsive design provides a functional mobile experience, the absence of a dedicated mobile app means there are no push notifications for assignment deadlines, attendance alerts, or fee reminders.

CHAPTER 8: CONCLUSION & FUTURE WORK

8.1 Conclusion

This project presented the design, development, and evaluation of an AI-Driven Smart Learning Management System - a full-stack web application built to address the significant shortcomings of conventional LMS platforms in the context of Pakistani public-sector universities, specifically Bahauddin Zakariya University, Multan.

The system was developed using FastAPI as the backend REST framework, React.js with a Neumorphic design system as the frontend, and MySQL as the relational database. Five AI subsystems were integrated: a Dlib-based face recognition engine for biometric login and two-level attendance, a Gemini + FAISS RAG chatbot for intelligent academic assistance, a weighted analytics engine for performance scoring and leaderboard generation, a Gemini-powered quiz generation module for automated assessment creation, and a two-stage plagiarism detection engine combining character-level and semantic similarity analysis.

All eight project objectives defined in Chapter 1 were successfully achieved. The system delivers three fully functional role-based portals for Admin, Teacher, and Student users, covering the complete academic lifecycle from department configuration to grade publication. The two-level attendance system - combining manual lecture attendance with automated campus gate face recognition - represents a novel approach to attendance management that provides cross-validation capabilities not present in any reviewed existing platform.

The AI chatbot demonstrated reliable performance for personal academic data queries by drawing from live database records, and acceptable performance for knowledge-based queries through FAISS retrieval augmentation. The analytics engine successfully synthesizes five performance dimensions into a unified score, enabling data-driven identification of at-risk students for early intervention.

This project demonstrates that high-quality, AI-powered academic management software can be developed indigenously, without dependence on expensive foreign platforms, and tailored specifically to the requirements of Pakistani university environments. The system is deployable on standard server hardware without specialized infrastructure, making it practically accessible for institutions with limited IT budgets.

8.2 Limitations

The following limitations of the current system are acknowledged:

- **Single Institution Scope:** The system is architected for deployment at a single institution. The database schema, configuration model, and authentication system do not support multi-tenant operation across multiple universities from a single instance.

- **Hardware-Dependent Face Recognition Accuracy:** Recognition performance is sensitive to camera quality and lighting conditions. Environments with poor lighting or low-resolution cameras will experience higher false rejection rates. The system does not include liveness detection, making it theoretically susceptible to photograph-based spoofing.
- **External AI Service Dependency:** The chatbot and quiz generation features require an active Google Gemini API connection. These features are unavailable during API outages or when all configured API keys have exhausted their quotas.
- **Limited Production Load Testing:** All performance measurements were conducted in a local development environment with a limited test dataset. The system's behaviour under production-scale concurrent load has not been empirically validated.
- **No Mobile Application:** The absence of a dedicated mobile application means users cannot receive push notifications for time-sensitive events such as assignment deadlines, attendance shortage alerts, or fee payment reminders.
- **English-Only UI:** The user interface is available in English only, which may present a barrier for users who are more comfortable in Urdu.

8.3 Future Enhancements

8.3.1 Mobile Application

The most impactful near-term enhancement would be the development of a cross-platform mobile application using React Native or Flutter. The existing FastAPI backend requires no changes to support a mobile client - all functionality is already exposed through the REST API. A mobile app would enable push notifications for assignment deadlines, attendance shortage warnings, fee payment reminders, and quiz availability alerts. Face recognition login and gate attendance scanning could also be supported through the mobile camera, eliminating the need for dedicated kiosk hardware at campus gates.

8.3.2 Scalability Improvements

For deployment at larger institutions with higher concurrent user loads, the following infrastructure improvements are recommended. Docker containerization would standardize the deployment environment and enable consistent behaviour across development, staging, and production. Kubernetes orchestration would support horizontal scaling of backend instances behind a load balancer. Redis caching would reduce database load for frequently accessed data such as leaderboard rankings, active semester information, and student enrollment lists. A database read replica would offload analytics and reporting queries from the primary write instance.

8.3.3 Advanced AI Features

Several AI enhancements would significantly increase the system's educational value.

- **Personalized Learning Path Recommendation:** By analyzing a student's historical performance patterns across courses, the system could recommend specific topics, study resources, and practice quiz areas tailored to each student's individual weaknesses.
- **Predictive Dropout Detection:** A machine learning model trained on historical student data (attendance trends, assignment submission patterns, grade trajectories) could predict dropout risk weeks or months before failure becomes inevitable, enabling proactive counselling interventions.
- **Automated Essay and Short-Answer Grading:** Integrating a language model-based grading system for short-answer and essay-type submissions would reduce the manual grading burden for teachers handling large classes.
- **Voice-Based Chatbot:** Adding speech-to-text and text-to-speech capabilities to the chatbot would make the system accessible to students who find text-based interaction challenging, and would enable voice-driven attendance verification at gate kiosks.
- **Urdu Language Support:** Integrating Urdu NLP support - both in the chatbot and in the user interface - would make the system significantly more accessible to students and staff whose primary language is Urdu. The multilingual sentence-transformer model already supports Urdu at the embedding level, so the primary work would be in UI translation and Urdu-language FAQ population.
- **Emotion and Engagement Detection:** Computer vision models capable of analysing student facial expressions and engagement levels during online sessions could provide teachers with real-time feedback on class attention and highlight students who appear confused or disengaged.

8.3.4 Additional Security Layers

- **Two-Factor Authentication (2FA):** Adding a time-based one-time password (TOTP) second factor for admin and teacher login would significantly strengthen account security.
- **Liveness Detection:** Integrating an anti-spoofing liveness detection module into the face recognition pipeline would prevent photograph-based attacks on the face login and gate attendance systems.
- **IP-Based Rate Limiting:** Implementing per-IP rate limiting on authentication endpoints would protect against brute-force password attacks.
- **Comprehensive Audit Logging:** Extending the existing activity_logs table to capture all administrative actions - including user creation, enrollment changes, fee modifications, and configuration updates - with full before/after values would provide a complete audit trail for institutional accountability.
- **End-to-End Encryption for Chat:** Implementing client-side encryption for course chat messages would protect the confidentiality of student and teacher communications.

8.4 Social Impact & Applications

The AI-Driven Smart LMS has implications that extend beyond the immediate context of BZU.

1. **Reducing Administrative Burden:** Pakistani universities - particularly public-sector institutions - operate with large student-to-staff ratios. Manual attendance management, fee processing, and academic record-keeping consume a disproportionate share of administrative capacity. Automating these processes through an integrated system directly frees staff time for higher-value activities.
2. **Improving Student Accountability:** The two-level attendance system - combining lecture-level records with campus gate data - makes attendance manipulation significantly more difficult and provides students with real-time visibility into their own attendance status. Early shortage alerts give students the opportunity to correct attendance problems before they reach the academic consequences threshold.
3. **Democratizing AI-Powered Education:** Commercially available AI education tools are priced beyond the reach of most Pakistani public institutions. By demonstrating that equivalent capabilities can be built indigenously on open-source foundations, this project contributes to a knowledge base that other institutions can build upon without incurring licensing costs.
4. **Supporting Post-COVID Blended Learning:** The COVID-19 pandemic accelerated the adoption of digital learning tools across Pakistani universities. The AI-Driven Smart LMS supports blended learning models by integrating online communication, digital assessment, and AI-assisted study tools within the same platform used for in-person attendance and campus management.
5. **Replicability Across Institutions:** The system's architecture is institution-agnostic at the feature level. With configuration changes to department structure, fee schedules, and academic calendar settings, the system could be deployed at any Pakistani university department with minimal customization effort. This replicability amplifies the social impact of the initial development investment.
6. **Contribution to National EdTech Development:** The project demonstrates the feasibility of building production-quality educational technology indigenously, contributing to the growing Pakistani software development ecosystem and providing a reference implementation for future academic management system development in the region.

REFERENCES

1. K. He, X. Zhang, S. Ren, and J. Sun, Deep Residual Learning for Image Recognition, in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770–778.
2. D. E. King, Dlib-ml: A Machine Learning Toolkit, Journal of Machine Learning Research, vol. 10, pp. 1755–1758, 2009.
3. P. Lewis, E. Perez, A. Piktus, et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020, pp. 9459–9474.
4. J. Johnson, M. Douze, and H. Jégou, Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, Jul. 2021.
5. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Hong Kong, 2019, pp. 3982–3992.
6. S. Abramson, T. Team, et al., Gemini: A Family of Highly Capable Multimodal Models, arXiv preprint arXiv:2312.11805, 2023.
7. S. Tikhonov and A. Maslennikov, FastAPI: Modern, Fast Web Framework for Building APIs with Python, GitHub Repository, 2023. [Online]. Available: <https://github.com/tiangolo/fastapi>
8. Merceron and K. Yacef, Educational Data Mining: A Case Study, in Proc. Int. Conf. Artificial Intelligence in Education (AIED), Amsterdam, Netherlands, 2005, pp. 467–474.
9. Romero and S. Ventura, Educational Data Mining: A Review of the State of the Art, IEEE Transactions on Systems, Man, and Cybernetics Part C: Applications and Reviews, vol. 40, no. 6, pp. 601–618, Nov. 2010.
10. M. Turk and A. Pentland, Eigenfaces for Recognition, Journal of Cognitive Neuroscience, vol. 3, no. 1, pp. 71–86, 1991.
11. W. Jia, H. Hu, and J. Gui, A Review on Automatic Attendance Management Systems Based on Face Recognition, Journal of Physics: Conference Series, vol. 1920, no. 1, 2021.
12. M. Crosby, P. Nau, S. Strand, and W. E. Großmann, Blockchain Technology: Beyond Bitcoin, Applied Innovation Review, no. 2, pp. 6–19, Jun. 2016.
13. Sadiq, A. Ul Haq, and M. Tahir, Automated Student Attendance System Using Face Recognition, International Journal of Advanced Computer Science and Applications, vol. 10, no. 5, pp. 195–200, 2019.
14. M. Cocea and S. Weibelzahl, Disengagement Detection in Online Learning: Validation Studies and Perspectives, IEEE Transactions on Learning Technologies, vol. 4, no. 2, pp. 114–124, Apr. 2011.

15. P. Blikstein, Using Learning Analytics to Assess Students' Behavior in Open-Ended Programming Tasks, in Proc. 1st Int. Conf. Learning Analytics and Knowledge (LAK), Banff, Canada, 2011, pp. 110–116.

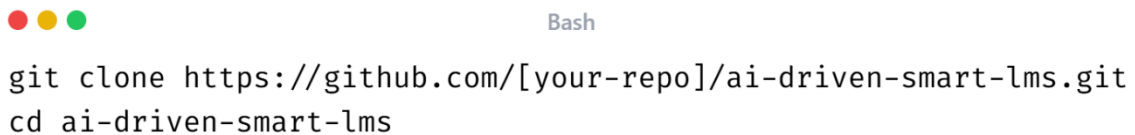
APPENDICES

Appendix A - Installation & Setup Guide

Prerequisites:

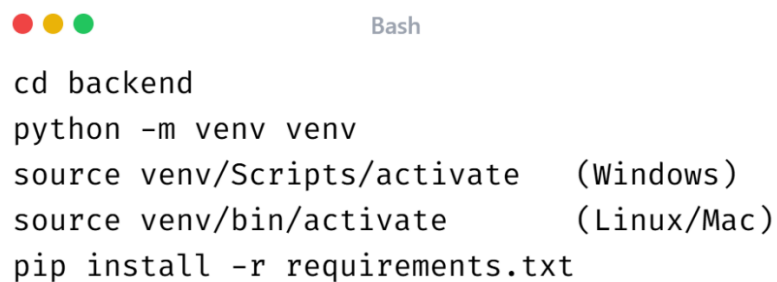
- Python 3.10 or higher
- Node.js 18 or higher
- MySQL 8.x
- Git

Step 1 - Clone Repository:



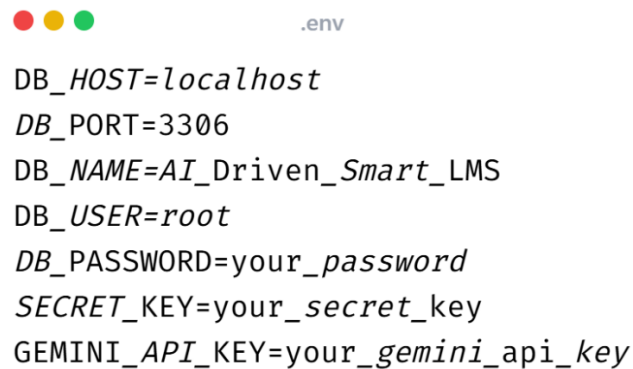
```
git clone https://github.com/[your-repo]/ai-driven-smart-lms.git
cd ai-driven-smart-lms
```

Step 2 - Backend Setup:



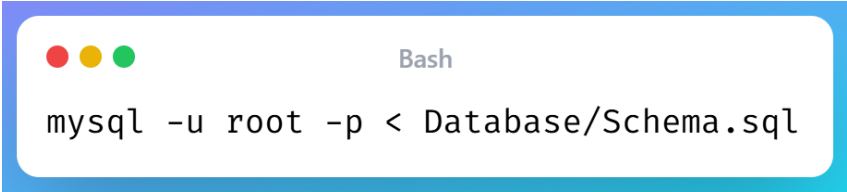
```
cd backend
python -m venv venv
source venv/Scripts/activate (Windows)
source venv/bin/activate (Linux/Mac)
pip install -r requirements.txt
```

Step 3 - Environment Configuration: Copy `.env.example` to `.env` and fill in the required values:



```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=AI_Driven_Smart_LMS
DB_USER=root
DB_PASSWORD=your_password
SECRET_KEY=your_secret_key
GEMINI_API_KEY=your_gemini_api_key
```

Step 4 - Database Setup:

A terminal window with a blue border and a white background. It has three colored dots (red, yellow, green) in the top left corner and the word "Bash" in the top right corner. The command `mysql -u root -p < Database/Schema.sql` is entered in the terminal.

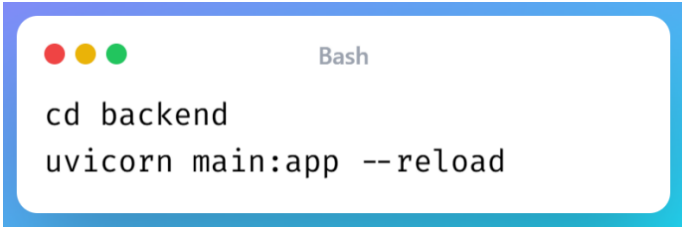
```
Bash
mysql -u root -p < Database/Schema.sql
```

Step 5 - AI Models Download: Download the following files and place them in `backend/app/ai/models/`:

- `shape_predictor_68_face_landmarks.dat`
- `dlib_face_recognition_resnet_model_v1.dat`

Both files are available from the official Dlib model repository at <http://dlib.net/files/>

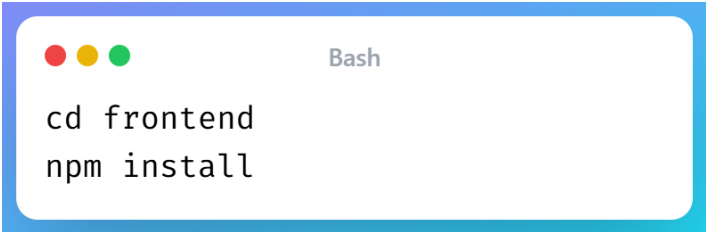
Step 6 - Start Backend Server:

A terminal window with a blue border and a white background. It has three colored dots (red, yellow, green) in the top left corner and the word "Bash" in the top right corner. The commands `cd backend` and `uvicorn main:app --reload` are entered in the terminal.

```
Bash
cd backend
uvicorn main:app --reload
```

Backend API is now accessible at <http://localhost:8000> API documentation at <http://localhost:8000/docs>

Step 7 - Frontend Setup:

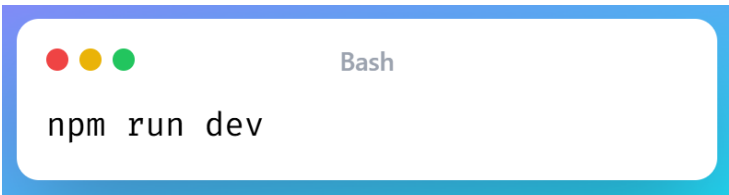
A terminal window with a blue border and a white background. It has three colored dots (red, yellow, green) in the top left corner and the word "Bash" in the top right corner. The commands `cd frontend` and `npm install` are entered in the terminal.

```
Bash
cd frontend
npm install
```

Copy `.env.example` to `.env`:

`VITE_API_BASE_URL=http://127.0.0.1:8000/api/v1`

Step 8 - Start Frontend:

A terminal window with a blue border and a white background. It has three colored dots (red, yellow, green) in the top left corner and the word "Bash" in the top right corner. The command `npm run dev` is entered in the terminal.

```
Bash
npm run dev
```

Application is now accessible at <http://localhost:5173>

Appendix B - API Endpoint Reference

The complete API documentation is auto-generated by FastAPI and available at:

<http://localhost:8000/docs> (Swagger UI - interactive)

<http://localhost:8000/redoc> (ReDoc - read-only)

The Swagger UI allows direct testing of all endpoints with automatic request schema validation and authentication support. All 60+ endpoints are documented with request body schemas, response models, authentication requirements, and example values.

Appendix C - Database Schema (Full SQL)

The complete database schema is available in the project repository at:

Database/Schema.sql

The schema file contains all CREATE TABLE statements, indexes, foreign key constraints, and generated columns for all 30 tables in the AI_Driven_Smart_LMS database. The tables are organized in dependency order to allow sequential execution without foreign key errors.

Key schema highlights:

- users table stores face_embedding as BLOB (1024 bytes per user for 128-d float64 embeddings)
- fee_structure table uses a MySQL GENERATED column for automatic total_fee calculation
- campus_attendance table includes duplicate filtering flag and manual override tracking
- student_performance_scores table stores JSON columns for score_breakdown, risk_prediction, weak_subjects, and recommendations

AI-Driven Smart Learning Management System **SARFRAZ - BSIT-21-13** Department of
Information Technology Bahauddin Zakariya University, Multan January 2026